Totally Collectibles OS Operator Manual

Authored by David Bakanas.

Software ownership: Dag Danky Holdings LLC.

Software platform: Totally Collectibles OS (TCOS).

Main TCOS website/domain: TotallyCollectibles.com.

Flagship store / Store #1: Truely Collectables.

Last updated: 2026-07-15 21:49 MDT / 2026-07-16 03:49 UTC

This is the working manual for Totally Collectibles OS (TCOS). It must stay current as features are added.

This revision includes the durable InstaComp batch queue and PaddleOCR worker, faster InstaComp image preparation and five-card queue claim mini-packs, database-pressure pause/governor behavior, 45-degree local image rotation and thumbnail zoom review, InstaComp-to-seller-draft handoff, InstaComp-to-Available-for-Trade handoff, seller inventory InstaComp lane, seller marketplace export packets, seller eBay staging and reconciliation, Stripe payment reliability controls, seller payout guards, shipping/coverage operations, and complete laptop-failure disaster recovery. Procedures labeled `dry run`, `draft`, `review`, `export`, or `not configured` are not production completion claims.

Deterministic Application Fonts

TCOS loads Geist Sans and Geist Mono from the installed `geist` package. The font files are bundled locally through `next/font/local`; production and local builds do not fetch CSS or font files from Google Fonts. This keeps `npm run build` deterministic in restricted or temporarily offline build environments while preserving the existing Geist typography and CSS variable names. The `tsx` runner used by shipping verification is also a direct development dependency rather than an accidental transitive dependency, so a clean install can run the full production verification stack.

Next.js and `eslint-config-next` are aligned on `16.2.10`. Next.js still declares PostCSS `8.4.31`, which is affected by GHSA-qx2v-qp2m-jg93, so `package.json` overrides PostCSS to `8.5.15`. Keep the override until a later verified Next.js release directly depends on a fixed PostCSS version. `npm audit --omit=dev` must report zero production vulnerabilities before removing or changing this protection. Production builds use the supported `next build --webpack` opt-out because Turbopack 16.2.10 stalled during compilation with the fixed PostCSS override; development can continue using Turbopack through `next dev`.

Tailwind source detection is explicitly bounded in `src/app/globals.css`: `source(none)` disables automatic workspace discovery and `@source "../**/*.{js,ts,jsx,tsx,mdx}"` scans the complete application source tree. This prevents cold builds on the FileProvider-backed workspace from recursively scanning generated build caches, operator artifacts, Git metadata, or dependencies without reducing application source coverage.

The production deploy helper command-pins Vercel CLI `56.2.0` through isolated `npm exec --package=vercel@56.2.0`; it does not rely on an undocumented machine-global CLI or add Vercel's deployment-

only transitive tree to the application lockfile or `node_modules`. The CLI cache lives under the operating system temporary directory, while every Vercel invocation receives `--cwd` with the TCOS repository root so isolation cannot change the deployment target. `VERCEL_SCOPE` must be a simple Vercel team slug using only lowercase letters, numbers, and hyphens. Flag-like, URL-like, whitespace, dotted, slashed, at-sign, uppercase, or secret-shaped scope values fail before quota status, preflight, Git fetch, or Vercel CLI work. `npm run preflight:production` runs the exact command and verifies its reported version before Git fetch or any possible Vercel upload. Missing npm registry access or a mismatched CLI fails closed with `No Vercel upload was started`.

Production target overrides are strict. `VERCEL_CLEAN_DOMAIN` and `VERCEL_UNWANTED_ALIAS` accept only valid bare DNS hostnames or root HTTP(S) URLs. Credentials, ports, paths, queries, fragments, IP addresses, localhost/single-label names, empty labels, underscores, and leading/trailing hyphens fail before npm exec, Git fetch, alias commands, or Vercel upload. Error messages name the environment variable but never echo the rejected value, preventing embedded credentials from leaking into logs.

Production smoke targets are equally strict. `SMOKE_BASE_URL` and `SMOKE_UNWANTED_ALIAS_URL` accept only valid bare DNS hostnames or root HTTP(S) URLs. The same unsafe URL components and malformed hostname shapes fail before admin authentication, Git fetch, or any HTTP request, and rejected values are not echoed. This prevents smoke from silently discarding a supplied credential, port, path, query, or fragment and validating a different origin than the operator intended.

Use `npm run status:production` during recurring build blocks to inspect the local Vercel quota cooldown without fetching Git, building, uploading, or starting a deployment. The output includes the exact block and retry timestamps, approximate remaining cooldown, marker path, whether a retry is locally allowed, configured Vercel deploy timeout, and the explicit line `Vercel upload started: no`. Use `npm --silent run status:production:json` for raw quota evidence with schema `tcos.productionQuotaStatus.v1`; it includes the same retry state, marker path, `deployTimeoutMs`, `deployTimeout`, `deployTimeoutEnv`, `vercelUploadStarted: false`, next action, and read-only guarantee without scraping text. The environment-flag equivalent is `TCOS_PRODUCTION_QUOTA_STATUS_ONLY=true node scripts/deploy-production.mjs`.

The normal deploy path checks this local cooldown before command-pinned npm exec, Git fetch, build, Vercel upload, or deployment. An active, invalid, or invalidly configured cooldown therefore stops external launch work at the earliest gate. `npm run preflight:production` intentionally remains quota-independent so operators can validate the CLI and Git state while waiting.

A malformed or unreadable marker fails closed as `state: invalid_marker`: the helper starts no Vercel upload and blocks deployment. Inspect or restore the marker before continuing. Use `TCOS_VERCEL_QUOTA_RETRY_OVERRIDE=true` or `--force-quota-retry` only after independently confirming that the rolling quota window has reset.

A zero, negative, or nonnumeric cooldown value also fails closed as `state: invalid_configuration`; it cannot disable the deployment guard. Set `TCOS_VERCEL_QUOTA_COOLDOWN_HOURS` to a positive number. The explicit retry override remains the only intentional bypass and should be used only after independently confirming the quota reset.

The quota marker is success-cleared, not attempt-cleared. A failed override retry, unparseable Vercel response, or clean-alias failure preserves the marker. The helper removes it only after Vercel returns a parsed deployment URL and the clean production alias succeeds.

The helper also requires `vercel --prod` to exit successfully before it parses the deployment URL, runs either alias command, or clears the quota marker. A URL printed by a failed Vercel command is diagnostic output, not a deployable result.

The actual `vercel --prod` process is bounded by `TCOS_VERCEL_DEPLOY_TIMEOUT_MS`, defaulting to 15 minutes and accepting only integer milliseconds from `60000` through `3600000`. If the deploy stalls or times out, the helper rejects any printed URL, starts no alias commands, preserves the local quota marker, and requires an operator to inspect Vercel deployments and aliases before retrying.

Unwanted-alias cleanup is also fail closed. Before moving the clean production domain, the helper requires `vercel alias rm truely-collectables-tt3b.vercel.app` to succeed or return Vercel CLI's explicit `Alias not found by` result. Authentication, scope, network, or other cleanup failures stop the launch before clean-domain aliasing and preserve the local quota marker.

The shared deploy-safety contract exposes `quotaStatusCommand` and its read-only description in launch-readiness JSON and Markdown, the launch handoff bundle, the Launch Readiness page, and the Production Smoke Report. Production smoke verifies those surfaces retain `npm run status:production`, so an operator does not have to rely on chat history to decide whether a deployment retry is safe.

The quota cooldown self-test must never use the production marker path. The helper refuses `--self-test-quota-cooldown` unless `TCOS_VERCEL_QUOTA_MARKER_PATH` points to an explicit temporary test file, preventing a validation run from deleting or replacing the actual quota record.

TCOS means Totally Collectibles OS. It is the multi-store software platform, admin system, order system, inventory engine, marketplace layer, and pricing/helper system. Truely Collectables is the flagship store inside TCOS, not a separate rebuild.

Ownership And Account Separation

David Bakanas is the owner of Dag Danky Holdings LLC.

Dag Danky Holdings LLC owns and administers the Totally Collectibles OS software platform.

David Bakanas is an admin/operator of Dag Danky Holdings LLC and TCOS.

Truely Collectables LLC is the collectables storefront operating company and should be treated as Store #1 inside TCOS. It should become its own platform seller/buyer account when seller, buyer, collection, wishlist, trade, or payout accounts are built.

Dag Danky Shoes is the footwear storefront/operator for the shoe and sneaker portion of the platform. It should be treated as its own storefront seller/buyer account when footwear seller, buyer, inventory, collection, wishlist, trade, or payout accounts are built.

David Bakanas is also the admin/operator of the Truely Collectables LLC account.

Dag Danky Holdings LLC platform revenue should come from:

- platform commission/rake from purchases completed through the TCOS website checkout
- the commission/rake is calculated from total sale amount, including item sale price plus buyer-paid shipping
- advertising revenue
- sponsorship revenue

- other platform-level service fees only after they are defined and disclosed

Truely Collectables LLC revenue and activity should stay separate from Dag Danky Holdings LLC platform revenue. Truely Collectables LLC is the storefront seller/buyer account for its own buying, selling, collecting, inventory, offers, trades, messages, payouts, and account history.

Truely Collectables LLC must have completely separate login credentials, account profile, seller settings, buyer settings, payout setup, and audit history from Dag Danky Holdings LLC platform-admin access.

Future account roles should stay separate:

- `platform_owner`: Dag Danky Holdings LLC
- `platform_admin_user`: David Bakanas and any future authorized TCOS admins
- `storefront_operator`: Truely Collectables LLC
- `storefront_seller_account`: Truely Collectables LLC selling inventory on TCOS
- `storefront_buyer_account`: Truely Collectables LLC buying/trading/collecting on TCOS when needed
- `footwear_storefront_operator`: Dag Danky Shoes
- `footwear_seller_account`: Dag Danky Shoes selling footwear inventory on TCOS
- `footwear_buyer_account`: Dag Danky Shoes buying/trading/collecting footwear on TCOS when needed
- `external_seller_account`: future third-party sellers
- `external_buyer_account`: future customers/collectors

Important rule:

Dag Danky Holdings LLC admin authority must not be mixed with Truely Collectables LLC seller/buyer activity. David may administer both, but the software should track platform-admin actions separately from seller/buyer actions so audits, payouts, disputes, chargebacks, taxes, and permissions stay clean.

Product North Star

TCOS should be built toward the ultimate collecting experience.

The collector should feel:

- confident they know exactly what the item is
- informed about rarity, demand, condition, and market value
- protected by clear policies, secure checkout, and honest evidence
- excited when the item arrives
- proud enough to share the pickup, collection, story, or mail day with the community

Every customer-facing item page should help answer:

- What exactly is this collectable?
- Why does it matter?
- How rare is it?
- What condition is it in?
- What data supports the price?
- What similar items have sold for?

- Is there population, certification, serial-number, autograph, patch, variant, or provenance information?
- What should a collector know before buying?
- What makes this item fun to own?
- What can the collector do with it after purchase, such as add it to a collection board or brag session?

Product data should be layered:

1. TCOS local inventory data
2. seller/admin-entered facts
3. AI extraction and description support
4. catalog/reference matching
5. sales comps
6. pricing guides
7. grading/certification data when available
8. rarity/population/print-run data when available
9. community context after moderation features exist

The tone should be collector-first: excited, knowledgeable, honest, and brand-safe. Public copy should invite collectors to celebrate their items without using sexual, hateful, private, or unsafe language.

Future: Collectable Megahub And Sneaker Expansion

TCOS should be designed to become a collectable megahub, not only a sports-card storefront.

Long-term ambition:

- become a premier destination for sneaker collectors
- support shoes, cards, memorabilia, comics, toys, TCG, autographs, graded items, sealed products, and other collectables
- help collectors research, find, request, buy, trade, show off, and track what they care about
- make AI search, collection tracking, and wish lists the best collector discovery experience possible
- turn Truely Collectables into a trusted place to understand collectables, not only transact on them

Sneaker marketplace goals:

- support sneaker brands such as Nike, Jordan, Adidas, New Balance, Puma, Reebok, Asics, Converse, Vans, and future brands
- support shoe model, colorway, SKU/style code, size, gender sizing, release date, condition, box status, authenticity status, defects, and provenance
- track deadstock, new with box, used, tried on, restored, custom, sample, player exclusive, collaboration, limited release, and graded/authenticated shoe states
- store multiple photos including box label, outsole, insole, size tag, heel, toe box, stitching, defects, and receipt/provenance when available
- support authentication workflow before high-value sneaker sales go public
- support size-specific pricing and comps because sneaker value changes heavily by size
- show comparable listings and sales only when the size/model/colorway match is close enough

AI collection and wish list goals:

- users can create a wish list for any collectable category
- users can describe what they want in natural language
- AI should translate wish-list language into structured search fields
- AI should match wish-list items against TCOS inventory, seller inventory, want ads, trades, auctions, and approved outside data sources
- AI should notify users only through consented notification channels
- users can mark items as grails, priority targets, set fillers, size targets, gift ideas, or research-only
- wish lists should support budget range, condition preference, grading preference, size, category, player, team, character, franchise, era, brand, colorway, and urgency

Set builder checklist goals:

- users can create checklists for complete sets, partial sets, team sets, player runs, character runs, release runs, parallel runs, graded runs, shoe colorway runs, or custom collector goals
- checklist templates should support year, brand, set, subset, card number, player, team, character, franchise, variant, parallel, serial number, autograph, relic/patch, grade target, condition target, and notes
- users can mark each checklist item as owned, needed, upgraded, incoming, watching, trade target, or not interested
- users can attach owned items from their collection to checklist slots
- users can add missing checklist items to wish lists or want ads
- AI should help build a checklist from natural language, uploaded lists, scans, set names, catalog sources, or manufacturer checklists when allowed
- TCOS should show checklist progress by percentage, count owned, count missing, estimated value when available, and highest-priority missing items
- set builders should be able to filter by missing cards, rookies, inserts, parallels, autographs, patches, graded targets, and price range

Checklist acquisition links:

- each missing checklist item should first search TCOS inventory
- if not in TCOS inventory, show approved outside lookup links
- outside links should include eBay sold/active search, CollX research, COMC search, Sportlots search, 130point sales search, Google source search, PriceCharting/SportsCardsPro, PSA when relevant, and other approved category sources
- outside links should preserve the structured checklist query as much as possible, including year, set, card number, player, team, grade, variant, size, colorway, or brand
- TCOS should clearly label outside links as external research or external marketplace links
- TCOS should not imply outside marketplaces are partners unless a partnership exists
- users should be able to save outside finds back to a checklist as `watching`, `found externally`, or `research note`

Megahub information goals:

- product pages should become research pages
- category pages should explain brands, sets, releases, pop reports, size markets, rarity, and recent demand

- collector intelligence should apply to all categories, not only cards
- buyer decisions should be supported by sources, comps, photos, condition proof, authenticity data, and clear risk notes
- AI should help summarize the collectable's story while showing source links and confidence

Competitive standard:

- TCOS should compete on trust, depth of information, buyer confidence, seller tools, AI discovery, and collector experience
- TCOS should not copy another marketplace's user experience blindly
- TCOS should avoid fake scarcity, fake hype, fake sold data, hidden fees, or misleading authenticity claims
- every expansion category should have its own data model, condition standards, authenticity rules, pricing logic, and dispute rules

Future data tables should store:

- collectable_categories
- sneaker_products
- sneaker_sizes
- sneaker_condition_reports
- sneaker_authentication_events
- sneaker_release_data
- sneaker_price_snapshots
- user_collections
- user_collection_items
- user_wish_lists
- user_wish_list_items
- wish_list_matches
- wish_list_notifications
- set_builder_checklists
- set_builder_checklist_items
- set_builder_templates
- set_builder_item_matches
- set_builder_external_links
- set_builder_progress_snapshots
- collectable_reference_sources
- collectable_category_attributes

Downloadable PDF copy:

```
docs/TCOS_OPERATOR_MANUAL.pdf
```

Future mobile app manual:

- the mobile app must have its own separate operator manual and downloadable PDF

- shared TCOS rules, store policies, security requirements, checkout behavior, and account requirements must stay consistent with this main site manual
- mobile-only screens, app-store release steps, push notification behavior, device permissions, and mobile troubleshooting must live in the mobile manual instead of being mixed into the web manual

The PDF is regenerated with:

```
npm run manual:pdf
```

The manual PDF generator looks for local Chrome, Edge, Chromium, and Brave binaries on macOS, Linux, and Windows. If the browser lives somewhere custom, set `TCOS_MANUAL_BROWSER_PATH` to the executable path before running `npm run manual:pdf`. If a headless browser writes the PDF but hangs during shutdown, set `TCOS_MANUAL_PDF_BROWSER_TIMEOUT_MS` to a shorter timeout; the generator treats a freshly written PDF as success and exits cleanly.

Links in this manual are clickable in both Markdown and PDF form. Use them for direct lookup, examples, API docs, and troubleshooting references.

Current V2 UI Baseline

The public storefront has been moved away from the default scaffold look.

Current UI baseline includes:

- sticky storefront navigation with TCOS/admin entry
- production metadata for Truely Collectables
- premium dark homepage hero with clear storefront positioning
- homepage feature blocks for Collector Intelligence, secure checkout, and fulfillment control
- shop page header, search/filter panel, active inventory count, and cleaner product cards
- product detail pages styled as collector research pages
- cart page with a structured order summary, shipping selector, TOS acceptance, and secure checkout action
- consistent neutral/off-white storefront background
- `/admin` now renders as a live TCOS command center with revenue metrics, fulfillment queues, offer desk, inventory watch, eBay sync policy decisions, blocked sync reasons, store settings status, evidence health, operator alerts, and fast links
- `/admin/launch-readiness` now resolves the active store primary domain, evidence email, and eBay sync settings from store settings before marking launch checks ready or warning
- `/admin/launch-readiness` now also checks `SUPABASE_SERVICE_ROLE_KEY`, and admin settings plus Stripe/order webhooks now prefer the service-role key for admin-only writes instead of relying only on anon-key access
- `/admin/launch-readiness` now checks the Seller Protection Financial Adjustments database capability from `20260712174000_add_seller_protection_financial_adjustments.sql`, so missing TCOS internal `seller_protection_reimbursement` support is visible before payout or reconciliation work relies on it
- admin settings, admin eBay control surfaces, admin payout workflows, admin files downloads, order review case packet generation, brag weekly reports, and admin login audit paths now also prefer the

service-role key for admin-side Supabase access

- checkout, offer, order-fulfillment, eBay sync, account-auth, and public rate-limit server paths now also prefer the service-role key so launch-critical writes and audits do not depend on public-key table permissions
- authenticated account dashboard/order APIs, collector binding offers, seller inventory/order/payout APIs, and seller marketplace connection routes now also prefer the service-role key for server-side account workflows
- collector profile, collection, wish-list, messaging, social, import, and export APIs now also prefer the service-role key for server-side collector workflows
- account login and signup now also use the shared server Supabase helper for consistency; those auth routes still use the normal anon-key auth flow because they are calling Supabase Auth directly rather than doing admin-only table writes
- shipment emails, offer emails, and admin packing slips now render the active store display name instead of a hardcoded storefront label
- shared public storefront surfaces now read brand/legal text from the centralized legal constants, including layout metadata, navbar, homepage eyebrow, buyer terms, seller terms, cart TOS copy, and success-page branding
- brag-share links, collector research guidance, seller marketplace messaging, and default inventory description text now also read shared store/platform branding constants instead of hardcoded storefront strings
- the remaining admin placeholder and explanatory copy now also read shared platform/store legal constants, including store email placeholders, eBay account label placeholder, admin platform heading, and launch-readiness platform/storefront separation text
- store-settings fallback contact addresses now derive from the active store primary domain when available, or from the TCOS platform domain, instead of falling back to a legacy single-store email domain

1. Quick Start

Daily operator path:

1. Open `/admin/login`.
2. Log in with the admin password.
3. Open `/admin/products/new` to scan a card lot with InstaComp or add one product manually.
4. Open `/admin/products` to review drafts, pricing, descriptions, status, and listing readiness.
5. Open `/admin/orders` and `/admin/shipping` to handle paid orders, labels, coverage, tracking, and exceptions.
6. Open `/admin/offers` to handle customer offers.
7. Open `/admin/financial-reconciliation`, `/admin/seller-payouts`, and `/admin/order-review-cases` to handle money exceptions, holds, disputes, and cash-out work.
8. Open `/admin/launch-readiness` before enabling any production payment or shipping change.

Most day-to-day work starts at:

```
/admin
```

Daily production safety order:

1. `/admin/launch-readiness` - inspect missing migrations, secrets, and provider blockers.
2. `/admin/financial-reconciliation` - verify the previous UTC day and clear unmatched-money alerts only after correction.
3. `/admin/order-review-cases` - handle disputes, returns, authenticity, shipping, payment, and payout holds.
4. `/admin/seller-payouts` - refresh Connect, release only eligible rows, and process cash-out work.
5. `/admin/shipping` - clear blocked purchases, dry-run records, missing tracking, missing Coverage policy IDs, and claims.
6. `/admin/orders` - pack, record real labels/coverage, save tracking, and mark shipped.
7. `/seller/marketplaces` and `/admin/ebay` - inspect stale connections, failed staging, outside orders, and reconciliation.
8. Run `/admin/payment-simulations` and `/admin/shipping/simulations` before changing payment or shipping behavior.
9. Run `/admin/launch-gate-drill` to confirm the runtime locks match the current launch reports without charging cards or buying postage.
10. Use `/admin/live-payment-launch` only for controlled launch approval or emergency revocation.

2. Routes

Public Storefront

Route	Purpose
`/`	Home page
`/shop`	Product grid with search and sport filtering
`/product/[id]`	Product detail page
`/cart`	Customer cart
`/account`	Customer account home
`/account/login`	Customer email/password login
`/account/signup`	Customer email/password signup
`/success`	Purchase confirmation page with rotating collector sayings
`/terms`	Customer Terms of Service
`/seller-terms`	Seller Terms of Service required before seller payout onboarding
`/seller`	Seller home redirect to marketplace connections
`/seller/orders`	Seller-owned order, payout, and hold activity workspace
`/seller/orders/[id]`	Seller-owned order detail drilldown
`/seller/marketplaces`	Seller marketplace connection dashboard for Store #1 sync foundation and future seller-safe connectors
`/seller/inventory`	Seller-owned draft, active, paused, and archived inventory workspace
`/seller/payouts`	Seller cash-out requests, holds, Connect readiness, and payout history

Admin

Route	Purpose
`/admin/login`	Admin login
`/admin`	Admin dashboard
`/admin/accounts`	Customer account lookup and linked order/offer activity
`/admin/products`	Product list
`/admin/products/new`	InstaComp lot scanner plus manual product entry
`/admin/products/[id]`	Edit product and pricing tools
`/admin/instacomp`	Dedicated InstaComp scan lab and batch workflow
`/admin/inventory`	Inventory operations workspace
`/admin/orders`	Fulfillment center
`/admin/settings`	Store operations and marketplace integration controls for the active TCOS store
`/admin/orders/[id]`	Order detail and tracking
`/admin/orders/[id]/packing-slip`	Printable packing slip
`/admin/order-review-cases`	Global chargeback, return, authenticity, shipping, payment-risk, and seller-dispute case queue
`/admin/files`	Transaction evidence files
`/admin/launch-readiness`	Live payment and production readiness checklist
`/admin/launch-gate-drill`	No-money runtime drill for live payment and shipping launch locks
`/admin/live-payment-launch`	Auditable dual-lock live payment approval/revocation gate
`/admin/live-shipping-launch`	Auditable dual-lock live shipping approval/revocation gate
`/admin/payment-simulations`	Payment reliability, webhook, refund, dispute, and checkout drill lab
`/admin/financial-reconciliation`	Stripe-versus-TCOS reconciliation queue and resolution controls
`/admin/seller-payouts`	Seller Connect readiness, ledger holds, and cash-out administration
`/admin/shipping`	Label, tracking, coverage, claim, exception, and shipping priority queue
`/admin/shipping/simulations`	Shipping-policy and dry-run provider simulation lab

Route	Purpose
`/admin/inventory/category-review`	eBay import category confidence and review queue
`/admin/ebay`	Store eBay connection and import operations
`/admin/ebay/sync-control`	Controlled eBay batch sync launcher
`/admin/offers`	Offer review
`/admin/security`	Admin login audit and lockout review

API

Route	Purpose
`/api/admin/login`	Sets `admin_auth` cookie
`/api/admin/logout`	Clears admin cookie
`/api/admin/files/[id]/download`	Downloads transaction evidence PDF
`/api/admin/order-review-cases`	Opens and updates order review cases and writes case audit events
`/api/admin/order-review-cases/[id]/packet`	Downloads an order review case packet PDF
`/api/admin/order-review-cases/[id]/payout-resolution`	Resolves related seller payout rows after a case decision
`/api/admin/order-review-cases/[id]/stripe-evidence`	Stages or submits the case evidence supported by Stripe
`/api/admin/launch-gate-drill`	Runs the no-money payment and shipping runtime gate drill
`/api/admin/live-payment-launch`	Reports, approves, or revokes the database half of the live payment gate and returns the live-money JSON evidence contract
`/api/admin/live-shipping-launch`	Approves or revokes the database half of the live shipping gate
`/api/admin/payment-simulations`	Runs signed webhook, refund, dispute, idempotency, and related payment simulations
`/api/admin/payment-simulations/checkout-e2e`	Runs the isolated storefront checkout end-to-end drill
`/api/admin/financial-reconciliation`	Loads and resolves Stripe-versus-TCOS financial exceptions
`/api/admin/seller-payouts/connect-refresh`	Refreshes seller Stripe Connect readiness
`/api/admin/seller-payouts/ledger`	Applies payout ledger review and hold actions
`/api/admin/seller-payouts/requests`	Reviews and resolves seller cash-out requests

Route	Purpose
`/api/admin/orders/[id]/shipping-labels`	Plans, purchases in approved mode, records, or voids shipping labels
`/api/admin/orders/[id]/shipping-claims`	Creates a shipping coverage claim draft for an order
`/api/admin/shipping-labels/[id]/coverage-policy`	Records or updates a Coverage policy reference
`/api/admin/shipping-labels/[id]/packet`	Downloads a shipping label audit packet
`/api/admin/shipping-claims/[id]`	Updates shipping claim status and provider references
`/api/admin/shipping-claims/[id]/packet`	Downloads a shipping claim evidence packet
`/api/admin/shipping/exceptions`	Exports the ranked shipping exception queue as CSV, with `X-TCOS-Shipping-Exceptions-*` headers summarizing total/critical/warning/watch counts
`/api/admin/shipping/simulations`	Runs shipping eligibility, dry-run adapter, and provider purchase-attempt audit simulations
`/api/account/signup`	Creates customer account through Supabase Auth
`/api/account/login`	Logs customer account in through Supabase Auth
`/api/account/orders`	Returns logged-in customer order history for the active store
`/api/account/dashboard/preferences`	Saves account sports/team and market watchlist preferences
`/api/account/collector/profile`	Loads and saves collector bio, social URLs, visibility, and message preference
`/api/account/collector/items`	Saves collection shelf items, wish list items, want ads, set needs, and trade targets

Route	Purpose
`/api/account/collector/exports`	Downloads the logged-in collector collection as CSV or full catalog JSON
`/api/account/collector/messages`	Creates and lists collector conversation records
`/api/account/collector/binding-offers`	Starts a card-required binding offer through Stripe setup checkout
`/api/account/seller/payout-onboarding`	Starts or checks Stripe-hosted seller payout/bank verification
`/api/account/seller/payout-requests`	Returns seller cash-out balance, request history, review-blocked context, and linked order routing summaries for the logged-in seller
`/api/account/seller/inventory`	Returns seller-owned inventory summary, full seller inventory workspace data, and recent promoted draft output for the active store
`/api/account/seller/inventory/bulk-status`	Runs seller-scoped bulk activation or archive actions across selected inventory rows using the same guardrails as single-item controls
`/api/account/seller/inventory/[inventoryItemId]`	Updates a seller-owned listing's title, price, quantity, and description without using the admin product screen
`/api/account/seller/inventory/[inventoryItemId]/activate`	Activates a seller-owned draft after readiness and payout verification checks pass
`/api/account/seller/inventory/[inventoryItemId]/archive`	Archives or pauses a seller-owned listing without deleting it from the active store records
`/api/account/seller/inventory/[inventoryItemId]/description`	Regenerates or AI-writes a seller-owned listing description through the shared inventory engine
`/api/account/seller/orders`	Returns seller-owned order activity, payout state, and hold context for

Route	Purpose
	the logged-in seller
`/api/account/seller/orders/[id]`	Returns seller-owned order detail for a single routed order
`/api/account/seller/marketplace-connections`	Loads or saves logged-in seller marketplace connection records for the active store
`/api/account/seller/marketplace-connections/ebay/auth`	Starts seller-safe eBay OAuth and returns the authorization URL
`/api/account/seller/marketplace-connections/ebay/status`	Refreshes the logged-in seller's eBay token status and updates connection health
`/api/account/seller/marketplace-connections/ebay/disconnect`	Securely revokes and disconnects a seller eBay account
`/api/account/seller/marketplace-connections/ebay/import-preview`	Builds a resumable seller eBay staging preview
`/api/account/seller/marketplace-connections/ebay/staged-items`	Reviews and updates staged seller eBay rows
`/api/account/seller/marketplace-connections/ebay/staged-items/promote`	Promotes approved staged rows into seller-owned TCOS drafts
`/api/account/seller/marketplace-connections/ebay/sync-control`	Pauses or resumes seller eBay sync activity
`/api/account/seller/marketplace-connections/ebay/reconcile`	Reconciles seller eBay quantities and listing state
`/api/account/seller/marketplace-connections/ebay/orders`	Imports outside eBay order effects for inventory reconciliation
`/api/instacomp/scan`	Runs OCR, AI identification, comp lookup, and scan persistence
`/api/instacomp/draft-listings`	Creates seller-owned TCOS draft listings from reviewed scan rows
`/api/checkout`	Creates Stripe checkout session
`/api/webhook`	Main Stripe webhook handler
`/api/stripe/webhook`	Alternate Stripe webhook handler
`/api/offers/create`	Customer offer submission

Route	Purpose
`/api/offers/update-status`	Accept/decline offer
`/api/offers/counter`	Send counter offer
`/api/orders/update-tracking`	Save carrier/tracking
`/api/orders/mark-shipped`	Mark shipped and send email if configured
`/api/ebay/auth`	Start eBay OAuth
`/api/ebay/callback`	Store eBay refresh token
`/api/ebay/import-listings`	Import one eBay inventory page
`/api/ebay/full-sync`	Batch import eBay inventory
`/api/ebay/notifications`	Receives and verifies eBay account-deletion/revocation notifications
`/api/cron/seller-ebay-reconciliation`	Scheduled seller eBay reconciliation endpoint
`/api/cron/stripe-reconciliation`	Scheduled daily Stripe financial reconciliation endpoint

3. Admin Login

Admin login uses:

```
ADMIN_PASSWORD=
```

Successful login sets a cookie:

```
admin_auth=<issued_timestamp>.<signature>
```

Cookie behavior:

- HTTP-only
- Secure
- Same-site lax
- Max age: 24 hours

Protected admin routes redirect to `/admin/login` if the cookie is missing.

Admin sessions are signed by `ADMIN_SESSION_SECRET` when configured. If `ADMIN_SESSION_SECRET` is missing, TCOS falls back to `ADMIN_PASSWORD` for signing. Production should use a separate strong

`ADMIN_SESSION_SECRET`.

Admin login hardening:

- password verification uses fixed-length hashed comparison instead of plain string equality
- every login attempt is evaluated through `src/lib/admin-login-security.ts`
- failed and successful attempts are written to `admin_login_attempts` when the table is available
- login attempts store store ID, IP address, user agent, result, failure reason, identity risk, header evidence, and lockout timestamp when applicable
- five failed attempts from the same IP inside 15 minutes triggers a 15-minute lockout
- active-lockout hits are audited but do not count as additional invalid-password attempts
- locked-out login attempts return HTTP `429`
- masked or blocked identity attempts return HTTP `403`
- if the audit table has not been migrated yet, login still works but audit/lockout storage is unavailable

Admin login policy:

Setting	Value
Failed-attempt window	15 minutes
Failed-attempt limit	5
Lockout duration	15 minutes

Admin security page:

```
/admin/security
```

This page shows recent admin login attempts, successful logins, failed logins, active lockouts, unique IP count, identity risk, failure reason, lockout expiration, and user agent. If the audit table has not been migrated yet, the page shows an unavailable warning instead of crashing.

Launch readiness also checks whether `admin_login_attempts` is available. If the table is missing or unavailable, `/admin/launch-readiness` marks Admin Login Audit as blocked.

The same `/admin/security` page also shows public money-path rate-limit events from `public_endpoint_rate_limit_events`, including checkout attempts, public offer attempts, collector binding-offer setup, seller payout onboarding, blocked status, endpoint, IP address, subject key, identity risk, block reason, policy window, and header evidence summary.

Launch readiness checks whether `public_endpoint_rate_limit_events` is available. If the table is missing or unavailable, `/admin/launch-readiness` marks Public Endpoint Rate Limits as blocked.

The same `/admin/security` page also lists saved IP investigations from `security_ip_investigations`. These cases show the IP address, status, severity, updated/reviewed/resolved timestamps, and internal notes.

Launch readiness checks whether `security_ip_investigations` is available. If the table is missing or unavailable, `/admin/launch-readiness` marks Security IP Investigations as blocked.

Suspicious IP drilldown:

```
/admin/security/ip/[ip]
```

IP addresses on `/admin/security` link to a focused IP dossier. The dossier combines admin login attempts, public money-path rate-limit events, TOS acceptance evidence, orders, offers, and transaction evidence reports tied to that server-observed IP. Use it when reviewing blocked checkout attempts, offer spam, suspicious account behavior, chargebacks, or repeat abuse.

The IP dossier includes an investigation form. Admins can mark the IP as `watch`, `review`, or `resolved`, set severity as `low`, `medium`, `high`, or `critical`, and save internal notes. Saving the form updates `last_reviewed_at`; resolving the case stores `resolved_at`.

4. Product And Inventory Basics

TCOS currently keeps two inventory layers in sync:

1. Legacy `products`
2. TCOS V2 `inventory_items`

The legacy `products` table still supports older screens and relations. The V2 `inventory_items` table is the newer inventory authority for status, price, and quantity.

The sync layer is:

```
src/modules/inventory/engine.ts
```

Do not bypass the engine for normal admin inventory changes.

Inventory V2 bridge screen:

```
/admin/inventory
```

Use this screen to verify that every legacy storefront product has a matching TCOS V2 `inventory_items` record. The page shows total legacy products, V2 bridged items, missing inventory rows, mismatch counts, active items, sold-out items, and eBay-linked items.

The `Backfill V2 Inventory` button runs an idempotent backfill. It can be run more than once. It scans Store #1 products, creates missing `inventory_items`, updates existing inventory rows by legacy product ID or SKU, mirrors quantity/price/title/description/status, and adds product images into `inventory_images` when they are not already present.

Reconciliation labels:

Label	Meaning
`OK`	Legacy product and V2 inventory row are aligned
`MISSING INVENTORY ITEM`	Product exists but no V2 inventory row exists yet
`SKU LINK ONLY`	V2 row matched by SKU but still needs the legacy product bridge filled
`QUANTITY MISMATCH`	Legacy quantity and V2 quantity differ
`PRICE MISMATCH`	Legacy price and V2 price differ
`SOLD OUT`	Product quantity is zero; not a failure by itself

eBay import safety:

The eBay import path is store-scoped. It first updates by Store #1 eBay listing ID, then by Store #1 SKU, then inserts a new product if no store-scoped match exists. It does not perform a global SKU upsert across all stores.

4A. Multi-Store Platform Foundation

TCOS is being converted from a single-store Truely Collectables app into a multi-store Totally Collectibles OS platform without breaking Store #1.

Current Store #1:

```
Truely Collectables
Truely Collectables LLC
store_id: 00000000-0000-4000-8000-000000000001
```

Foundation migration:

```
supabase/migrations/20260628110000_create_tcos_stores.sql
supabase/migrations/20260628113000_create_store_settings.sql
```

The migration:

1. Creates `stores`.
2. Inserts Store #1 = Truely Collectables.
3. Adds defaulted `store_id` columns to current core tables.
4. Backfills existing rows to Store #1.
5. Adds store indexes for future filtering.

Tables prepared with `store_id`:

- `products`
- `inventory_items`
- `orders`

- `order_items`
- `offers`
- `ebay_tokens`
- `sales_comp_snapshots`
- `tos_acceptance_events`
- `transaction_evidence_reports`
- `account_collector_profiles`
- `account_conversations`
- `account_conversation_messages`
- `account_binding_offers`
- `account_collection_export_jobs`

Safety rule:

Current Store #1 constants live in:

```
src/lib/stores.ts
```

Current active store context is resolved through `src/lib/stores.ts`. Store #1 is still the only active store, but app code now calls active-store helpers instead of importing the Store #1 ID across the app.

Current write paths pass the active store ID when creating products, inventory items, orders, order items, offers, eBay tokens, sales comp snapshots, TOS acceptance events, and transaction evidence reports. Current inventory, eBay token, sales comp, fulfillment, offer, evidence, admin, and success-page read/update paths are also scoped to the active store. The database default still points new rows to Store #1 as a safety net.

The inventory repository and inventory engine now carry store context internally. Future multi-store work should replace the current Store #1 resolver with request/account/domain-selected store context.

Store operational settings live in:

```
src/lib/store-settings.ts
```

The `store_settings` table stores per-store support, sales, offer, evidence, order email, Stripe mode, eBay environment, eBay account label, and seller commission settings. If the table is missing or empty, TCOS falls back to current environment variables and Store #1 defaults so production behavior does not break.

`/admin/launch-readiness` shows the active store settings and whether they came from the database or fallback defaults.

5. Inventory Status Values

Allowed status values:

Status	Meaning	Checkout allowed?
`draft`	Not ready to sell	No
`active`	Ready to sell	Yes, if quantity is enough
`reserved`	Held back	No
`sold`	Sold out	No
`archived`	Removed from active workflow	No

Checkout requires:

- status is `active`
- quantity is greater than or equal to cart quantity
- price is greater than zero

6. Add A Product

Open:

```
/admin/products/new
```

The page has two paths.

InstaComp lot scanner

Use the scanner at the top for card images and card lots. It accepts up to 500 card rows, pairs fronts and backs, performs OCR and AI identification, searches configured comp sources, and can create non-public TCOS draft listings. The exact operating procedure is in `Section 32: InstaComp Production Operation`.

Nothing from InstaComp should be treated as publicly verified merely because a scan completed. Review the player, year, set, card number, parallel, serial number, autograph/relic signals, condition, comps, title, price, and photos before activation or cross-listing.

Manual product entry

Use the form below the scanner when one product is already fully known.

Fields:

- Title
- Player
- Sport
- Price
- Quantity
- Image URL
- Description

Description can be left blank. If blank, TCOS generates a description from product data.

When `Add Manual Product` is saved, TCOS:

1. Creates a row in `products`.
2. Creates a row in `inventory_items`.
3. Creates an `inventory_images` primary image row if image URL exists.
4. Redirects to `/admin/products`.

7. Edit A Product

Open:

```
/admin/products
```

Click `Edit`.

Edit page:

```
/admin/products/[id]
```

Fields editable:

- Title
- Player
- Sport
- Price
- Quantity
- Status
- Image URL
- Description

Click `Save Product`.

Save behavior:

1. Updates legacy `products`.
2. Ensures V2 `inventory_items` exists.
3. Updates V2 title/description/category/status/quantity/price.
4. If image URL changed, adds a new primary `inventory_images` row.
5. Publishes an in-memory inventory event.

8. Quick Status Buttons

On `/admin/products/[id]`:

- `Set Active`

- `Reserve`
- `Mark Sold`
- `Archive`

`Mark Sold` sets quantity to `0`.

Status changes update through `inventoryEngine.setStatus`.

9. Product Description Tools

On `/admin/products/[id]`, the Description panel has:

- `Auto-Fill Description`
- `AI Write Description`

Auto-Fill Description

Uses only local TCOS product data.

Generated from:

- title
- player
- sport
- price
- quantity
- status
- SKU
- eBay listing ID

This is deterministic and does not call outside APIs.

AI Write Description

Uses OpenAI API docs if configured:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
```

Default model:

```
gpt-5.5
```

If OpenAI is not configured or the request fails, TCOS falls back to local auto-fill.

The AI prompt forbids inventing:

- year
- set

- grade
- condition
- autograph
- patch
- serial number
- rookie status
- scarcity

Product Detail Collector Intelligence

Product pages currently show a Collector Intelligence panel on:

```
/product/[id]
```

Implementation files:

```
src/app/product/[id]/page.tsx
src/lib/collector-intelligence.ts
```

Current behavior:

- lays out `/product/[id]` as a collector research page with image, status badge, price, purchase actions, description, and a collector snapshot
- the collector snapshot shows category, player/subject, availability, status, SKU, and eBay linkage when available
- shows an honest trend badge
- defaults to `Not Enough Verified Data`
- creates market research links for eBay sold search and 130point
- creates acquisition/research links for TCOS search, eBay active search, COMC, Sportlots, CollX, PriceCharting, SportsCardsPro, PSA Auction Prices, and Google marketplace search
- creates news/source links for Google News and official-source searches
- creates an X search link for the player, item, team, character, or franchise query
- shows a `Complete The Set Or Run` helper with TCOS, eBay, COMC, Sportlots, CollX, manufacturer-checklist, and 130point research links
- shows an `Exact Match Signals` panel using deterministic title extraction
- detects title-level serial numbering, card number, parallel/finish words, autograph, relic/patch, rookie, grading company, and grade signals when present
- marks title-level variant/parallel evidence as needing checklist/source confirmation before TCOS claims an exact rare variation
- detects PSA, SGC, CGC, Beckett, or BGS names in the title when present
- detects grade-like title text when present
- detects cert-number-like title text when present
- links to official grading lookup pages

- gives a watch list of things collectors should check before treating the item as hot or rare

TCOS does not currently store live trend snapshots or verified grading population counts for storefront display. Until those sources are saved and verified, the public trend state remains `Not Enough Verified Data`.

10. Sales Comps

Sales comps live on:

```
/admin/products/[id]
```

Click:

```
Check eBay Sold Comps
```

The page reloads with:

```
?comps=true
```

TCOS tries these sources:

1. eBay Marketplace Insights API
2. eBay completed-items fallback
3. Google Programmable Search JSON API, if configured
4. PriceCharting / SportsCardsPro API, if configured

TCOS also gives research links for:

- CollX
- 130point sales search
- Google sold search example
- PriceCharting example search
- SportsCardsPro
- PSA Auction Prices
- Card Ladder
- ALT
- COMC
- eBay sold search example

Lookup Examples

Use the product title, player, year, set, card number, grade, and autograph/patch terms when known.

Example searches:

- eBay sold: Michael Jordan 1991 Fleer
- Google sold-card search: Michael Jordan 1991 Fleer

- PriceCharting: Michael Jordan 1991 Fleer
- PSA Auction Prices
- 130point sales search

When checking comps, ignore listings that are not the same card or same grade. A raw card, PSA 10, BGS 9.5, autographed card, patch card, serial-numbered parallel, and base card are different markets.

11. Suggested Price

Suggested price uses a CollX-style method:

1. Average of up to 10 recent sold comps from the last six months.
2. If unavailable, use the last known sold comp.
3. If unavailable, use median of available comps.
4. If no comps exist, no suggested price is shown.

Displayed values:

- Suggested
- Count
- Median
- Average
- Range
- Recent comps used
- Source status

12. Apply Suggested Price

Click:

```
Apply Suggested Price
```

TCOS does not trust stale browser data. It recalculates comps server-side, saves a new snapshot, then updates the product price.

Flow:

1. Load current product from `inventoryEngine`.
2. Run `getSalesComps`.
3. Save a `sales_comp_snapshots` row if table exists.
4. If suggested price exists, update product through `inventoryEngine.updateProduct`.
5. Redirect back to product edit page with comps loaded.

13. Comps History

Comps history shows on product edit pages.

Table:

```
sales_comp_snapshots
```

Migration:

```
supabase/migrations/20260627160000_create_sales_comp_snapshots.sql
```

If this table is missing, the page still works but history will show a Supabase error. Apply the migration to enable saving.

14. eBay Sync

Connect eBay

Start OAuth:

```
/api/ebay/auth
```

Callback stores refresh token into:

```
ebay_tokens
```

Reference links:

- eBay Developers Program
- eBay Sell Inventory API
- eBay Marketplace Insights API

Import Listings

Single page:

```
/api/ebay/import-listings
```

Batch sync:

```
/api/ebay/full-sync
```

Controlled admin launcher:

```
/admin/ebay/sync-control
```

Import behavior:

1. Reads latest eBay refresh token.

2. Gets eBay access token.
3. Pulls eBay inventory items.
4. Fetches offer data per SKU.
5. If listing is active, updates `products` and `inventory_items`.
6. Maps the eBay title/aspects into a TCOS category.
7. Saves useful eBay aspects as generated inventory attributes.
8. If listing is not active, marks local quantity zero.

It does not delete eBay inventory.

Current eBay category mapper output:

- `sports_cards`
- `trading_cards`
- `shoes`
- `comics`
- `memorabilia`
- `toys`
- `sealed_wax`
- `autographs`
- `coins`
- `other_collectable`

The mapper stores category confidence in generated attributes. Low-confidence imports remain in inventory but receive `tcos_review_required = true` so admin review can identify listings that need better categorization instead of silently guessing wrong.

Admin review:

```
/admin/inventory/category-review
```

This page shows mapped eBay imports, TCOS category confidence, review-required flags, mapping evidence, and sample eBay aspects. Review-required, low-confidence, and `other_collectable` rows appear first and link back to the product edit screen.

Safe sync workflow:

1. Open `/admin/ebay/sync-control`.
2. Run a small batch, usually 10 or 25 listings.
3. Review imported categories on `/admin/inventory/category-review`.
4. Continue with the next offset if results look clean.
5. Use larger batch sizes only after the category mapper is behaving well.

The full-sync API accepts optional `limit` and `maxBatches` query parameters. Allowed batch sizes are 10, 25, 50, and 100. `maxBatches` is capped at 25.

Current sync implementation:

```
src/lib/ebay-sync.ts
src/lib/ebay-category-mapper.ts
```

Both single-page import and full sync use this shared server-side importer. Full sync calls the importer directly instead of calling TCOS through its own protected HTTP route, so it does not depend on an admin browser cookie or `NEXT_PUBLIC_SITE_URL` to keep running. This keeps Truely Collectables eBay sync more reliable when the toggle is enabled.

eBay Sync Policy Decisions

Open:

```
/admin/ebay/sync-control
```

The controlled sync page now shows public inventory totals, the last batch result, current-run policy decisions, and blocked policy summaries.

Inventory stats shown:

- public products
- in-stock products
- sold-out products
- eBay-linked products
- missing SKU products

Decision labels:

Decision	Meaning
`ALLOWED`	TCOS allowed the sync action
`NEEDS REVIEW`	TCOS imported the listing but flagged it for admin category/title review
`BLOCKED BY TCOS POLICY`	TCOS refused the local import/update because required listing evidence was unsafe or incomplete

Current blocked reasons:

- missing SKU
- missing eBay listing ID
- invalid listing price
- invalid listing quantity

Current review reasons:

- missing product title
- category review required
- low category confidence

Important rule:

Blocked sync decisions only protect TCOS local inventory. They do not delete, revise, or end eBay-side inventory.

Decision events are stored in:

```
ebay_sync_decision_events
```

Summary views:

```
tcos_ebay_snapshot_import_decision_summary
tcos_ebay_missing_sync_decision_summary
tcos_public_inventory_stats
```

eBay Health

Open:

```
/admin/ebay
```

This page is the local eBay reconciliation board. It does not call eBay on page load. It reads TCOS product and inventory data to show whether Store #1 inventory looks ready for eBay sync.

The page shows:

- total products
- eBay-linked products
- healthy linked products
- products needing attention
- missing SKU count
- never-synced count
- stale-sync count
- sold-out count
- latest eBay import timestamp

Health labels:

Label	Meaning
`OK`	Linked product has no local sync warning
`MISSING SKU`	Product cannot safely sync to eBay inventory without a SKU
`NOT LINKED`	Product is local-only and has no eBay listing ID
`NEVER SYNCED`	Product has an eBay listing ID but no `last_seen_at` import timestamp
`STALE SYNC`	Product has not been seen by eBay import in the configured stale window
`SOLD OUT`	Local TCOS quantity is zero

Current stale window:

```
12 hours
```

Buttons on the page:

Button	Purpose
`Test Route`	Confirms the eBay test route responds
`Import Batch`	Runs one import page from eBay
`Full Sync`	Runs the batch eBay import loop
`Reconnect`	Starts eBay OAuth again

Rule:

Use `/admin/ebay` before and after large imports. Missing SKU and stale sync warnings should be reviewed before assuming local inventory and eBay inventory agree.

eBay Sync Toggle

Open:

```
/admin/settings

- store operations now let admins update the active store primary domain,
support/sales/offers/evidence emails, evidence/order sender labels, eBay environment, eBay
account label, and seller commission rate
- those settings feed launch readiness, evidence delivery, support/contact display, and
marketplace environment checks for the active store
```

The `Enable eBay Sync` toggle controls whether the active store can use eBay integration.

When eBay sync is enabled:

- `/api/ebay/import-listings` can import an eBay inventory page
- `/api/ebay/full-sync` can run the batch import loop
- `/api/ebay/auth` can reconnect the store's eBay OAuth token
- completed sales can push quantity changes back to eBay

When eBay sync is disabled:

- import is blocked
- full sync is blocked
- reconnect/OAuth is blocked
- post-sale eBay quantity updates are skipped
- `/admin/ebay` shows a disabled warning

Storage:

```
store_settings.metadata.ebay_sync_enabled
```

Default behavior:

```
enabled
```

This keeps Store #1 working unless an admin intentionally turns eBay sync off. For TCOS/future stores, turn this off when eBay sync should not help an outside seller or competitor.

15. Checkout

Checkout route:

```
/api/checkout
```

Customers must accept the Terms of Service before checkout can start.

Current Terms of Service:

```
/terms
```

Current TOS version:

```
2026-06-28
```

Checkout validates:

- cart is not empty
- duplicate cart lines are combined by product before quantity checks
- product IDs and quantities must be positive whole numbers
- shipping method is valid

- shipping is currently limited to United States addresses
- Terms of Service was accepted
- cart metadata fits inside Stripe metadata limits
- each product exists
- each product is `active`
- each product has enough quantity
- each product has a price greater than zero

Then it creates a Stripe checkout session.

Stripe checkout is configured with `shipping_address_collection.allowed_countries = ["US"]` for cart checkout, accepted offer checkout, and counter-offer checkout. Truely Collectables does not currently accept shipments outside the United States. Stripe webhooks also verify the collected shipping country before marking an order ready to ship; missing or non-US shipping evidence is stored as `paid_shipping_review` / `shipping_review`.

Successful cart checkout sends the buyer to:

```
/success?type=cart&session_id={CHECKOUT_SESSION_ID}
```

Stripe metadata includes:

- compact cart item/quantity metadata
- shipping method
- shipping name
- shipping amount
- subtotal
- item count
- TOS accepted flag
- TOS version
- TOS accepted timestamp

Reference:

- Stripe Checkout docs

16. Stripe Webhooks

Main webhook:

```
/api/webhook
```

Alternate webhook:

```
/api/stripe/webhook
```

On `checkout.session.completed`:

1. Verify Stripe signature.
2. Create or update order.
3. Insert order items if not already inserted.
4. Decrement inventory through `inventoryEngine`.
5. Sync eBay quantity after sale.
6. If accepted offer, mark offer paid.
7. Create or update transaction evidence report.
8. Email evidence PDF if evidence email is configured.

Accepted-offer sessions use `product_id` metadata if cart metadata is missing.

Safety rule:

Webhook cart metadata is normalized the same way checkout cart input is normalized. Duplicate product lines are combined before availability checks. Current cart checkout stores compact cart metadata in Stripe so large cart JSON cannot break checkout; webhooks still accept older JSON cart metadata for existing sessions.

Webhook order fulfillment status is also shipping-policy aware. If Stripe does not provide a US shipping country, TCOS still records the paid order and transaction evidence, but the order is held as `paid_shipping_review` / `shipping_review` instead of being released as `ready_to_ship`. Review-held orders appear in the admin Fulfillment Center under the Needs Review tab and cannot be marked shipped until the review status is resolved.

Inventory decrements run through the Supabase RPC `tcos_decrement_inventory_after_sale`, which locks the product row, refuses insufficient quantity, updates `products.quantity`, mirrors the result into `inventory_items`, and returns the before/after quantity. If inventory disappears between checkout creation and Stripe webhook completion, TCOS marks the paid order as `paid_inventory_review` / `inventory_review` instead of silently overselling the item.

Order webhooks store TOS/IP acceptance fields and create a transaction evidence report. The evidence report is intended for chargeback defense, fraud review, and legal dispute support.

Offer and counter-offer Stripe success URLs send buyers to:

```
/success?type=offer&session_id={CHECKOUT_SESSION_ID}
/success?type=counter&session_id={CHECKOUT_SESSION_ID}
```

The confirmation page uses the Stripe checkout session ID server-side to read checkout metadata, find the purchased product, and personalize the customer experience. When product data is available, it shows the purchased item image/title and themes the page with inferred team, character, franchise, or collectable colors. If product data is unavailable, it falls back to the Truely Collectables black/gold theme.

The confirmation page rotates short collector-focused sayings such as `Welcome to the family.` so the purchase feels like a meaningful addition to the customer's collection instead of a plain receipt page.

Reference:

- Stripe webhooks docs

Live Payment Readiness

Open:

```
/admin/launch-readiness
```

This page is server-rendered for admins only. It checks production configuration without exposing secret values.

It reports:

- public site URL
- Supabase configuration
- admin password/session secret configuration
- Stripe live/test/mixed key mode
- Stripe webhook secret
- IP intelligence/VPN blocking configuration
- transaction evidence email configuration
- eBay production sync configuration
- AI product helper configuration
- platform/storefront account separation status

Live buyer payments should stay closed until:

1. `NEXT_PUBLIC_SITE_URL` is the final HTTPS production domain.
2. Supabase production variables are set.
3. `ADMIN_PASSWORD` and `ADMIN_SESSION_SECRET` are set.
4. `STRIPE_SECRET_KEY` is a live key.
5. `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` is a matching live key.
6. `STRIPE_WEBHOOK_SECRET` is the live webhook signing secret for `/api/webhook`.
7. `IP_INTELLIGENCE_REQUIRED=true` and the IP intelligence provider is configured.
8. A real low-dollar checkout succeeds.
9. The order appears in admin.
10. The transaction evidence PDF is created.
11. eBay inventory quantity sync is confirmed.
12. The live test transaction is refunded in Stripe.

Platform/storefront account separation remains a readiness warning until real account roles exist. Before seller accounts, footwear operations, or third-party sellers go live, TCOS must enforce separate logins, roles, audit trails, payout profiles, and seller/buyer records for Dag Danky Holdings LLC, Truely Collectables LLC, Dag Danky Shoes, and outside sellers/customers.

17. Orders And Fulfillment

Open:

```
/admin/orders
```

Tabs:

- Ready to Ship
- Shipped
- All Orders

Order detail:

```
/admin/orders/[id]
```

Order detail shows:

- payment status
- fulfillment status
- customer name/email
- shipping address
- item list
- totals
- carrier/tracking
- packing slip link
- TOS/IP chargeback evidence
- evidence packet download if created

18. Transaction Evidence Files

Open:

```
/admin/files
```

Every completed Stripe transaction should create one evidence packet in:

```
transaction_evidence_reports
```

Evidence packets include:

- order ID
- order creation timestamp
- customer name and email
- shipping address
- carrier and tracking when saved
- shipped timestamp when marked shipped
- item list
- quantities and prices
- subtotal
- shipping paid

- total paid
- Stripe checkout session ID
- Stripe payment intent ID when available
- Stripe webhook event ID
- Stripe payment status
- accepted TOS version
- TOS accepted timestamp
- TOS acceptance audit event ID
- server-observed IP address
- user agent
- IP risk status
- IP block reason if applicable
- raw Stripe metadata saved with the transaction
- report generation timestamp

Admin actions:

- download PDF packet
- open related order
- download packet directly from the order detail page

Email behavior:

- if `RESEND_API_KEY` and `TRANSACTION_EVIDENCE_EMAIL` are set, TCOS emails the evidence PDF after the transaction report is created
- if email is not configured, the report is still saved in Admin Files
- if email fails, the error is saved on the report

Refresh behavior:

- saving tracking refreshes the saved evidence packet
- marking shipped refreshes the saved evidence packet
- the original transaction email is not resent during tracking/shipment refreshes
- the Admin Files PDF download uses the latest saved packet text

Evidence files:

```
src/lib/evidence-pdf.ts
src/lib/transaction-evidence.ts
src/app/admin/files/page.tsx
src/app/api/admin/files/[id]/download/route.ts
```

19. Tracking And Shipment

Tracking update API:

```
/api/orders/update-tracking
```

Required:

- order ID
- carrier
- tracking number

Mark shipped API:

```
/api/orders/mark-shipped
```

Requirements:

- order exists
- carrier exists
- tracking number exists

When marked shipped:

- `fulfillment_status` becomes `shipped`
- `shipped_at` is set
- the transaction evidence packet refreshes with shipment details if one exists
- email is sent if `RESEND_API_KEY` and customer email exist

Supported tracking links:

- USPS
- UPS
- FedEx

20. Packing Slips

Open:

```
/admin/orders/[id]/packing-slip
```

Use this when packing shipments.

21. Offers

Product pages include offer submission.

Customers must accept the Terms of Service before submitting an offer.

Offer creation route:

```
/api/offers/create
```

Admin page:

```
/admin/offers
```

Admin actions:

- accept
- decline
- counter

Accept/counter creates a Stripe checkout link.

Offer checkout payment decrements inventory through TCOS V2.

Accepted and counter offers must pass `inventoryEngine.requireAvailableCartItems()` before TCOS creates the Stripe checkout session. That means the product must exist, be active, have enough quantity, and have a positive checkout price.

Accepted and counter offer Stripe sessions include Store #1 metadata, cart metadata, subtotal, item count, and zero-dollar offer-checkout shipping metadata so the webhook can create a normal order record.

Accepted and counter offer Stripe sessions carry the TOS acceptance metadata from the original customer offer when available.

Accepted and counter offer Stripe sessions are also limited to United States shipping addresses.

22. Seller Accounts And Auctions

The seller account, seller inventory, seller marketplace connection, Stripe Connect readiness, seller order, and cash-out foundations are current. Auctions remain a future-build feature. Do not create fake auction workflows or bypass the existing seller ownership, payout, inventory, and audit controls.

Account signup direction:

- TCOS accounts should use email and password as the baseline login method.
- Buyer/customer accounts, seller/store accounts, and platform-admin accounts must stay separate.
- Platform admin access for Dag Danky Holdings LLC must not be mixed with Truely Collectables LLC seller/buyer activity.
- Future seller accounts must have their own profile, verification status, payout-provider IDs, TOS acceptance, audit trail, and permissions.
- Future buyer accounts must have their own profile, order history, TOS acceptance, saved addresses, collection, wishlist, want ads, and communication preferences.

Current account foundation:

```
/account
/account/login
/account/signup
/api/account/login
/api/account/signup
```

```
/api/account/orders
/api/account/dashboard/preferences
/api/account/collector/profile
/api/account/collector/items
/api/account/collector/exports
/api/account/collector/messages
/api/account/collector/binding-offers
```

Current behavior:

- customer accounts use Supabase Auth email/password
- signup requires buyer Terms of Service acceptance
- signup requires Stripe card and billing address verification unless `ACCOUNT_CARD_VERIFICATION_REQUIRED=false`
- password must be at least 10 characters
- signup creates or updates `account_profiles` with `payment_verification_required` until Stripe setup verification completes
- signup assigns a Store #1 buyer membership in `account_store_memberships`; pending accounts use `payment_verification_required`
- pending card-verification accounts cannot log in or use authenticated account APIs as active customers
- signup/login writes `account_auth_events` when the migration exists
- signup may proceed through Stripe card and US billing address verification even when IP intelligence marks the request as VPN, proxy, Tor, relay, hosting, or anonymous; login and money-path activity remain subject to identity controls
- signup/login checks recent failed attempts by IP and email before calling Supabase Auth
- six failed signup or login attempts inside 15 minutes triggers a 15-minute account auth lockout
- account auth failures store `failure_reason` and `lockout_until` when the lockout migration exists
- `/api/account/login` and `/api/account/signup` responses include `X-TCOS-Account-Auth-Action`, `X-TCOS-Account-Auth-Status`, `X-TCOS-Account-Auth-Card-Verification`, `X-TCOS-Account-Auth-Session`, and `X-TCOS-Account-Auth-Membership` headers so support can distinguish active, blocked, pending-verification, and failed auth outcomes without exposing emails, account IDs, auth sessions, Stripe session IDs, or card data
- logged-in checkout and offer flows attach the account ID to Stripe metadata
- completed Stripe webhooks save `orders.account_id` when account metadata is present
- customer-created offers save `offers.account_id` when the customer is logged in
- `/account` shows recent linked orders for the logged-in customer
- `/api/account/orders` responses include `X-TCOS-Account-Orders`, `X-TCOS-Account-Orders-Dry-Run-Shipping-Blocked`, and `X-TCOS-Account-Orders-Seller-Item` headers so account order history can be reconciled without exposing hidden dry-run tracking/carrier values
- `/account` lets customers save a collector handle, bio, collecting focus, location label, social URLs, visibility, and message preference

- `/account` lets customers save owned collection items with category, condition, grade, estimated value, and notes
- `/account` lets customers save wish list items, 30-day want ads, set needs, and trade targets
- `/account` lets customers download their collection as CSV or a full catalog JSON backup
- `/account` lets customers save favorite teams/sports and market watchlist items
- `/api/account/dashboard/preferences` list responses include `X-TCOS-Dashboard-Sports-Favorites` and `X-TCOS-Dashboard-Market-Watchlist`; create/archive responses include `X-TCOS-Dashboard-Preference-Kind`, `X-TCOS-Dashboard-Preference-Mutation`, and `X-TCOS-Dashboard-Preference-Id`
- sports dashboard preferences are stored locally first; live news, scores, schedules, and odds require approved data providers later
- market watchlist preferences support stocks, ETFs, indexes, crypto, NFTs, commodities, collectable indexes, and other assets
- `/admin/accounts` shows customer accounts, linked order counts, offer counts, TOS status, and linked revenue
- `/admin/orders` and `/admin/orders/[id]` show whether an order is linked to a TCOS account or was a guest checkout
- `/admin/offers` shows whether an offer is linked to a TCOS account or was a guest offer
- guest checkout still works and leaves `account_id` empty
- account sessions are browser-local and separate from admin login cookies
- admin login still uses `/admin/login` and `admin_auth`

Anti-fraud signup requirement:

- buyer/customer account creation requires a valid payment card and billing/address evidence through Stripe unless disabled by environment override
- the card is saved through a Stripe setup flow before the account becomes fully active
- TCOS activates the buyer account only when Stripe returns a card payment method with complete United States billing address evidence
- TCOS must not store raw card numbers, CVV, or payment credentials
- failed or canceled verification prevents account activation and login
- this requirement is intended to reduce scam accounts, chargeback risk, fake offers, and abusive collector/social activity

Migration:

```
supabase/migrations/20260628190000_create_tcos_accounts.sql
supabase/migrations/20260628193000_link_accounts_to_orders_offers.sql
supabase/migrations/20260628201500_add_account_auth_lockouts.sql
supabase/migrations/20260628213000_create_sports_dashboard_tables.sql
supabase/migrations/20260628220000_create_collector_dashboard_tables.sql
supabase/migrations/20260628223000_create_collector_profiles_messaging_exports.sql
supabase/migrations/20260630100000_add_account_card_verification.sql
supabase/migrations/20260701074500_add_account_billing_address_evidence.sql
```

Current: Collection Shelf, Wish List, And Want Ads

The collector dashboard now has the foundation for owned collections and hunting targets.

Collection Shelf supports:

- title
- category
- item type
- condition
- grade company
- grade value
- estimated value
- ownership status
- privacy/visibility
- favorite flag
- notes

Wish List and Want Ads support:

- wish list items
- 30-day want ads
- set needs
- trade targets
- category and item type
- player/character, team/franchise, brand, set, year, card number, and variant fields
- desired condition and grade
- budget range
- priority levels including grail
- status tracking
- future match records

Current behavior:

- collector records are account-scoped and store-scoped
- removing a collection item soft-archives it
- removing a wish list item cancels it
- want ads default to a 30-day expiration
- matching, AI identification, image uploads, alerts, and outside marketplace links are future layers on this foundation

Post-current-goal collection cockpit backlog:

- after the current go-live/live-money goal is finished, build the Collection Shelf into a one-click collection cockpit where each owned item can be marked `Hold`, `Trade`, or `Sell`
- `Hold` should keep the item in the collector shelf as not available while preserving estimated value and collection history

- `Trade` should mark the item trade-available and connect it to the trade-target/want-ad layer without creating payment, payout, shipping, or order activity
- `Sell` should start a seller-inventory draft handoff from the collection item, not a live listing, checkout, auction, postage purchase, or payout release
- the same cockpit should surface integrated pricing context from the saved estimated value, imported value confidence, InstaComp/collector-intelligence research links, and future comp refresh jobs
- set-builder work should evolve from set needs and `Complete The Set Or Run` research links into checklist progress, missing-card filters, and pricing-aware upgrade/trade/sell decisions

Foundation tables:

```
account_collection_items
account_wish_list_items
account_wish_list_matches
```

Current: Collector Bio, Social, Messaging, Binding Offers, And Backups

Collector profiles support:

- collector handle
- bio
- collecting focus
- location label
- website URL
- social marketplace URLs for Instagram, Facebook, X, TikTok, YouTube, Whatnot, and eBay
- profile visibility
- message opt-in flag

Collector social supports:

- discovering public/community collector profiles
- following collectors
- sending friend requests
- accepting incoming friend requests
- showing a following/friends brag feed on the account dashboard
- posting a purchase brag directly from order history
- one-click default brag posting with an optional customize path
- brag visibility of private, friends, followers, community, or public
- generated share links under `/brag/[slug]`
- brag-link click tracking through `account_brag_post_clicks`
- source-tagged share actions for feed links, X, Facebook, and copied links
- weekly brag performance report foundation through `/api/admin/brag-weekly-report`

Collector social API:

- `/api/account/collector/social` list responses include `X-TCOS-Collector-Social-Collectors`, `X-TCOS-Collector-Social-Following`, `X-TCOS-Collector-Social-Friends`, `X-TCOS-Collector-Social-Incoming-Friend-Requests`, `X-TCOS-Collector-Social-Outgoing-Friend-Requests`, and `X-TCOS-Collector-Social-Feed` headers so the dashboard arrays can be reconciled without parsing the body
- follow, friend-request, accept-friend, create-brag, and remove-connection responses include `X-TCOS-Collector-Social-Action`, `X-TCOS-Collector-Social-Status`, `X-TCOS-Collector-Social-Connection-Type`, and `X-TCOS-Collector-Social-Resource-Id` headers without exposing target collector account IDs

Brag post share links:

- redirect to `/shop?brag=[slug]`
- preserve `src` so traffic can be attributed to feed, X, Facebook, copied links, direct links, or future channels
- increment `account_brag_posts.click_count`
- save click audit data with source, referrer, user agent, observed IP, and timestamp
- redirect responses include `X-TCOS-Brag-Share-Slug`, `X-TCOS-Brag-Share-Source`, `X-TCOS-Brag-Click-Tracking`, and `X-TCOS-Brag-Redirect-Destination` headers so share-link tracking can be verified without exposing collector account IDs
- display the TCOS/TotallyCollectibles.com link in the brag feed so shared posts can bring customers back to the marketplace

Weekly brag stats:

- configured by `BRAG_REPORT_EMAIL`
- uses `RESEND_API_KEY` when available
- falls back to saving the weekly report row if email is not configured or email fails
- stores report history in `account_brag_weekly_reports`
- includes tracked traffic by source so weekly email can show which social/link channel brought visitors back
- `/api/admin/brag-weekly-report` success responses include `X-TCOS-Brag-Weekly-Report-Id`, `X-TCOS-Brag-Weekly-Posts`, `X-TCOS-Brag-Weekly-Clicks`, `X-TCOS-Brag-Weekly-Emailed`, and `X-TCOS-Brag-Weekly-Email-Status` headers so report jobs can be reconciled with saved rows and email configuration
- should be scheduled once per week by the deployment scheduler or admin automation

Collection dashboard API:

- `/api/account/collector/items` lists the logged-in collector's private collection shelf and active/matched/renewed wish-list rows
- the same endpoint creates `collection_item` and `wish_list_item` rows, archives collection rows, and cancels wish-list rows without creating storefront products, orders, checkout rows, or Stripe activity
- list responses include `X-TCOS-Collector-Items` and `X-TCOS-Collector-Wish-List` headers so the dashboard payload can be reconciled with the returned counts
- create/archive/cancel responses include `X-TCOS-Collector-Item-Kind`, `X-TCOS-Collector-Mutation`, and `X-TCOS-Collector-Item-Id` headers so browser traces can identify the exact collector mutation

without parsing the JSON body

Collector profile API:

- `/api/account/collector/profile` loads and upserts the logged-in collector's profile without exposing account IDs in response headers
- profile responses include `X-TCOS-Collector-Profile-Present`, `X-TCOS-Collector-Profile-Visibility`, `X-TCOS-Collector-Profile-Messages`, and `X-TCOS-Collector-Profile-Mutation` headers so operators can confirm profile state and message opt-in from browser traces

Collection exports:

- `/api/account/collector/exports?format=csv` downloads a spreadsheet-friendly collection backup
- `/api/account/collector/exports?format=catalog_json` downloads profile, collection items, wish list items, pricing fields, descriptions/notes, image URLs, and a media manifest
- each export writes an `account_collection_export_jobs` audit row when the migration is available
- export responses include `X-TCOS-Collector-Export-Format`, `X-TCOS-Collector-Export-Items`, `X-TCOS-Collector-Export-Wish-List`, and `X-TCOS-Collector-Exported-At` headers so a saved backup can be matched to its export metadata without parsing the file body

Collection imports:

- `/api/account/collector/imports` imports CSV rows into the logged-in collector's private collection shelf
- the account dashboard supports source-labeled CSV uploads for eBay, COMC, CollX, Sportlots, Whatnot, Shopify, generic CSV, and other outlets
- CSV import accepts common headers such as title, category, condition, grader, grade, certification number, image URL, listing URL, price/value, price paid/cost, source ID, SKU, and notes
- imports write only to `account_collection_items`
- imports do not create storefront products, sellable TCOS inventory, eBay listings, orders, offers, checkout rows, or Stripe activity
- duplicate checks use source marketplace plus source item ID when available, then title/category/certification fallback matching
- `account_collection_import_jobs` stores row, import, skip, and error counts when the migration is available
- successful import responses include `X-TCOS-Collector-Import-Source`, `X-TCOS-Collector-Import-Rows`, `X-TCOS-Collector-Import-Imported`, `X-TCOS-Collector-Import-Skipped`, `X-TCOS-Collector-Import-Errors`, and `X-TCOS-Collector-Import-Job` headers so the browser response can be reconciled with the import job audit row

Messaging foundation:

- `account_conversations` stores account-to-account collector threads
- `account_conversation_messages` stores regular messages, binding-offer messages, and system messages
- the account page does not yet expose the full inbox UI; the API and schema foundation are in place
- `/api/account/collector/messages` list responses include `X-TCOS-Collector-Conversations`

- message-send responses include `X-TCOS-Collector-Conversation-Id`, `X-TCOS-Collector-Message-Id`, and `X-TCOS-Collector-Message-Action` so browser traces can distinguish new conversation sends from replies without parsing the JSON body

Binding offer rule:

- a binding offer starts in `payment_required`
- the buyer must accept buyer TOS
- masked identity checks still apply
- Stripe Checkout runs in setup mode before the offer is submitted
- `/api/account/collector/binding-offers` success responses include `X-TCOS-Collector-Binding-Offer-Id`, `X-TCOS-Collector-Binding-Offer-Conversation`, `X-TCOS-Collector-Binding-Offer-Conversation-Action`, `X-TCOS-Collector-Binding-Offer-Status`, and `X-TCOS-Collector-Binding-Offer-Payment-Required` headers so the setup handoff can be traced without exposing Stripe secrets or account IDs
- after Stripe confirms the payment method, the webhook updates the offer to `submitted`
- the later seller-acceptance slice must charge the saved payment method, mark the offer paid or failed, and create/lock the order

Foundation tables:

```
account_collector_profiles
account_social_connections
account_brag_posts
account_brag_post_clicks
account_brag_weekly_reports
account_conversations
account_conversation_messages
account_binding_offers
account_collection_export_jobs
```

One Or Two Click Inventory Imports From Other Sales Outlets

Collectors and seller accounts should eventually import inventory from other sales outlets with as few steps as possible.

The first collector CSV import path is implemented for private collection shelves. Future connector work should extend this foundation to official APIs, OAuth flows, provider exports, and seller inventory import jobs.

Goal:

- connect or upload from an outside sales outlet
- preview detected items
- import into TCOS inventory or the collector's private collection
- preserve source IDs, source marketplace, listing URLs, images, descriptions, prices, condition, quantity, and category evidence
- map items through the same Universal Inventory Engine category and attribute system

- prevent duplicate imports by source listing ID, SKU, and normalized title

Candidate outlets:

- eBay seller inventory
- COMC
- CollX
- Sportlots
- Whatnot
- Shopify or CSV export
- future shoe/collectable marketplaces where terms allow import

Implementation rule:

- use official APIs, OAuth, account export files, or user-uploaded CSV/templates where available
- do not scrape accounts or bypass marketplace terms
- each outlet needs its own connector status, last sync time, import job log, and duplicate-detection report

Future: Sports, Scores, Schedules, Odds, And Market Watchlists

The account dashboard now has the data foundation for favorite teams and market watchlists.

Sports dashboard goals:

- users can save favorite teams, leagues, and sports
- future provider jobs can populate news, scores, schedules, league links, and official-team references
- odds data must be provider-backed, jurisdiction-aware, age-gated where required, and display-only unless a future legal/compliance review approves anything more
- TCOS should prefer official league/team sites or licensed sports data providers instead of scraping

Market dashboard goals:

- users can save stocks, ETFs, indexes, crypto, NFTs, commodities, collectable indexes, and other assets
- future provider jobs can populate quotes, price history, news, NFT floor pricing, and alerts
- market data must be informational only and must not be presented as financial advice
- provider terms, licensing, freshness, and attribution must be reviewed before live display

Foundation tables:

```
account_sports_favorites
sports_data_sources
sports_event_snapshots
sports_news_snapshots
sports_odds_snapshots
account_market_watchlist_items
market_data_sources
market_price_snapshots
market_news_snapshots
```

Current seller legal page:

```
/seller-terms
```

Seller-account requirements:

- Truely Collectables LLC should be modeled as the storefront seller/buyer account
- David Bakanas can administer the Truely Collectables LLC account, but those seller/buyer actions must be recorded separately from Dag Danky Holdings LLC platform-admin actions
- Truely Collectables LLC must use separate login credentials and account information from Dag Danky Holdings LLC platform-admin access
- Dag Danky Shoes should be modeled as the footwear storefront seller/buyer account
- Dag Danky Shoes must use separate login credentials and account information from Dag Danky Holdings LLC platform-admin access and from Truely Collectables LLC
- sellers must accept the Seller Terms of Service before listing, auction submission, payout setup, or seller account use
- seller bank and payout information must be verified by an approved third-party payment, banking, or identity provider
- TCOS must not store raw bank credentials directly
- seller payout status must be gated by third-party verification status
- Dag Danky Holdings LLC charges an 8% platform commission/rake on purchases completed through the TCOS website checkout
- Dag Danky Holdings LLC does not earn the 8% platform commission/rake when an item uploaded from TCOS sells on eBay or another outside marketplace; outside marketplace sales follow that marketplace's own fees, payout rules, and seller obligations
- the 8% commission is calculated from total sale amount, including item sale price plus buyer-paid shipping
- seller acceptance should be stored with seller TOS version and timestamp when seller accounts are implemented
- seller payouts must follow the approved payment processor's timing, reserve, debit, chargeback, instant payout, bank-transfer, and recovery rules unless Dag Danky Holdings LLC approves a different processor or payout method
- sellers may list unverified autographs, in-person autographs, through-the-mail autographs, fan-club returns, or other provenance-supported items only if the listing clearly discloses the true verification status
- autograph listings must clearly label whether they are third-party certified, seller-pass-guaranteed, provenance-supported but unverified, or sold as-is and unverified
- if a seller relies on provenance evidence such as envelopes, letters, postmarks, signing photos, event tickets, or correspondence, that evidence must be clearly called out in the listing description and shown in listing photos when available
- if a seller claims or implies that an item will pass JSA, PSA DNA, Beckett, or another named third-party authenticator, that seller claim becomes part of the transaction record and refund/dispute evidence
- if an autograph or authenticity-sensitive item is sold as unverified, sold as-is, and not guaranteed to pass third-party authentication, the buyer assumes that disclosed authentication risk and a later failure alone

does not force a refund unless the seller made a false or misleading authenticity representation

- seller inventory activation now blocks autograph-sensitive drafts when the listing still lacks an explicit authenticity status, required certification provider, required pass-guarantee authenticator, or required provenance evidence
- seller eBay staging now carries forward title/aspect autograph clues into draft authenticity defaults when possible, so certified or autograph-sensitive imports start closer to a compliant disclosure state before activation
- seller marketplace preview rows and staged rows now show imported authenticity badges, cert clues, category hints, and disclosure notes directly in the seller UI before promotion into draft inventory
- seller marketplace staged rows now also include a review editor so the seller can correct category hint, authenticity status, autograph source, cert details, pass-guarantee names, provenance evidence, and disclosure notes before promoting the row into draft inventory
- seller marketplace staged rows now also show the projected draft activation outlook, including future activation blockers such as missing authenticity disclosure, missing cert provider, missing provenance evidence, missing SKU, missing image, or missing quantity before the seller promotes the row
- seller cash-out requests can only be made against payout ledger amounts marked eligible; held funds remain unavailable until fulfillment, dispute, fraud, and review rules release them
- seller payout ledger rows can be manually released to eligible, placed back on fulfillment hold, placed on dispute/review hold, reversed, or cancelled by admin review
- seller payout ledger rows tied to an active or paid cash-out request must not be destructively changed without resolving the cash-out request first
- admin payout review can mark seller cash-out requests requested, approved, processing, paid, rejected, or cancelled for audit tracking
- every admin payout ledger or cash-out request status change must write an append-only audit event with previous status, new status, admin note, IP, user agent evidence, and timestamp
- marking a cash-out request paid records TCOS status only; actual payout movement must be completed through the approved payout processor until automated transfers are explicitly built and tested
- marking a cash-out request paid must record the provider payout reference plus final processor fee and final seller net amount so TCOS can reconcile the cash-out against the payment processor
- seller cash-out or payout processor fees are separate from the Dag Danky Holdings LLC 8% platform rake and may reduce the seller's final net payout according to the payout provider's rules
- when a return, dispute, chargeback, authenticity case, or item-not-as-described claim is opened against a seller item, related seller funds must be held until the case and all available appeals are finally decided
- if the case is decided against the seller, TCOS policy should support recovery from held funds, future payouts, or the seller's verified payout/bank method according to payment processor rules, including recovery within three business days when supported by the provider and allowed by law
- a first confirmed counterfeit or authenticity-breach case may require refund, formal seller warning, payout hold, and case-history logging
- a repeat confirmed counterfeit or authenticity-breach case may trigger seller suspension, removal, or permanent ban review
- order review cases live on the admin order detail page and cover chargebacks, returns, authenticity issues, item-not-as-described claims, payment risk, shipping issues, seller disputes, and other order problems

- opening an order review case can automatically move related seller payout ledger rows into `hold_dispute_or_review`
- opening an order review case can optionally move an unshipped order into `shipping_review`
- order review case updates write append-only case events with previous status, new status, admin note, IP, user agent evidence, and timestamp
- closing a case records the outcome summary, but seller payout release or recovery must still follow the payout ledger controls and the approved payout processor's rules

Current seller payout verification foundation:

- `/account` includes a Seller Verification panel for logged-in, active accounts
- `/api/account/seller/payout-requests` now returns dispute-blocked request context so sellers can see when an open cash-out request is still tied to active review cases or held payout rows
- `/api/account/seller/payout-requests` now also returns linked order routing summaries for each request so sellers can jump straight into the exact routed orders tied to a cash-out
- `/api/account/seller/payout-requests` list responses include `X-TCOS-Seller-Payout-Requests`, `X-TCOS-Seller-Payout-Open-Requests`, `X-TCOS-Seller-Payout-Blocked-Requests`, `X-TCOS-Seller-Payout-Eligible-Rows`, `X-TCOS-Seller-Payout-Pending-Fulfillment`, `X-TCOS-Seller-Payout-Dispute-Holds`, `X-TCOS-Seller-Payout-Review-Guard`, `X-TCOS-Seller-Payout-Protection-Status`, and `X-TCOS-Seller-Payout-Protection-Rows`; successful cash-out creation responses include `X-TCOS-Seller-Payout-Request-Mutation`, `X-TCOS-Seller-Payout-Request-Status`, and `X-TCOS-Seller-Payout-Request-Allocated-Rows` headers so payout pressure can be reconciled without exposing payout request IDs, payout references, seller notes, admin notes, ledger IDs, or seller account IDs in headers
- `/api/account/seller/orders` responses include `X-TCOS-Seller-Orders`, `X-TCOS-Seller-Orders-Active-Cases`, `X-TCOS-Seller-Orders-Held`, `X-TCOS-Seller-Orders-Open-Cash-Out`, and `X-TCOS-Seller-Orders-Dry-Run-Shipping-Blocked` headers; `/api/account/seller/orders/[id]` responses include `X-TCOS-Seller-Order-Detail`, `X-TCOS-Seller-Order-Items`, `X-TCOS-Seller-Order-Active-Cases`, `X-TCOS-Seller-Order-Held-Payout-Rows`, `X-TCOS-Seller-Order-Cash-Out-Requests`, and `X-TCOS-Seller-Order-Dry-Run-Shipping-Blocked` headers so seller order workspaces can reconcile order pressure without exposing tracking values, customer names, payout request IDs, payout ledger IDs, or seller account IDs in headers
- `/admin/financial-reconciliation` now includes a TCOS Internal Money Context card for seller-protection reimbursement adjustments, showing latest reconciliation-run seller-protection reimbursement totals, recent internal credits, protected item reimbursed total, shipping excluded from reimbursement, allocation counts, and the payout-review handoff
- `/admin/launch-readiness` now includes a Seller Protection Financial Adjustments database readiness row so operators can confirm TCOS internal seller-protection reimbursement credits and reimbursement-plan metadata are supported before launch
- `/api/admin/launch-readiness`, its Markdown export, and its hand-off bundle now include an Under-$20 Seller Protection section with the internal-only coverage model, 2% seller reserve, `$20.00` item cap, shipping-excluded reimbursement rule, LetterTrack/USPS IMb delivery-evidence rule, and `seller_protection_reimbursement` ledger path

- `/admin/production-smoke` now also lists Under-$20 Seller Protection launch handoff coverage and links operators straight to the handoff bundle, seller-protection reconciliation view, and shipping claims cockpit after smoke passes
- the Under-$20 Seller Protection launch handoff contract is now centralized in `src/lib/seller-protection-launch-contract.ts`, so `/api/admin/launch-readiness` and `/admin/production-smoke` share the same reserve, cap, evidence, ledger, migration, and handoff-link source
- `/admin/shipping` now keeps the Seller Protection Refund Proof Missing and Seller Protection Payout Blocked guardrails visible even when there are no current matching claims, so production smoke can verify the seller-protection control plane on an empty queue
- production guardrails now protect the always-visible `/admin/shipping` Under-$20 Seller Protection Guardrails text, keeping empty-queue smoke coverage tied to source code instead of only the smoke runner
- production smoke now reports missing expected text for the `/admin/shipping` LetterTrack and seller-protection controls check, so failed smoke output names the absent launch strings instead of only showing a clipped HTML snippet
- production smoke now also reports missing expected text for the production-smoke report page, launch handoff bundle, and shipping simulation lab checks, covering the string-heavy launch surfaces most likely to drift after a delayed deploy
- `/api/admin/launch-readiness` now includes a Deployment Source fingerprint in JSON, Markdown, and handoff exports with Vercel environment, deployment URL, Git commit SHA/ref/repo, clean production domain, and the operator instruction to compare that SHA against `origin/main`
- production smoke now parses the launch-readiness Deployment Source JSON and fails if production's reported Git commit SHA/short SHA/ref/domain does not match the refreshed `origin/main` deployment target
- production smoke failure output now includes a `diagnostic` column for the launch-readiness Deployment Source check, showing the exact production Git SHA/ref/domain versus the refreshed `origin/main` values when Vercel is behind GitHub
- `/admin/seller-payouts` now shows an admin Under-$20 Protection Reserve view across loaded payout ledger rows that carry TCOS under-$20 protection metadata, including 2% reserve withheld, protected item amount, protected/liability row counts, and shipping excluded from reimbursement
- `/admin/seller-payouts` seller payout ledger rows now also show row-level under-$20 protection chips for Standard Envelope rows with protection metadata, so operators can see protected/liability status and reserve math before releasing or holding a payout row
- the Seller Cash-Out panel now breaks held funds into pending-fulfillment holds, dispute holds, reserved open requests, and cancelled/reversed rows so sellers can see what is truly cash-out ready
- blocked seller cash-out requests now link into a seller-side Hold Context section on `/account` so the affected order numbers and hold reasons are easy to trace without admin access
- `/seller/payouts` now gives sellers a dedicated payout workspace with Stripe verification status, cash-out readiness metrics, request history, and blocked hold context links into seller orders
- `/seller/payouts` now also supports seller-side request filters for blocked, open, paid, and attention-needed cash-out requests plus text search across request IDs, order references, notes, provider status, and payout references

- `/seller/payouts` now surfaces linked order cards inside each seller cash-out request, including payment status, fulfillment status, request amount by order, active case counts, held payout row counts, and direct links into seller order detail
- `/seller/payouts` now also shows TCOS Under-$20 Seller Protection reserve visibility from the payout ledger, including protected/liability row counts, 2% reserve withheld, protected item amount, and shipping excluded from reimbursement at the workspace, request, and linked-order levels
- `/account` now also shows TCOS Under-$20 Seller Protection reserve visibility inside the Seller Cash-Out panel and recent cash-out request cards, so sellers can see the 2% reserve, `$20.00` item cap, protected/liability row counts, and shipping-excluded amount before leaving the account dashboard
- `/seller` now acts as the seller command center with cross-workspace metrics for inventory readiness, payout pressure, seller order volume, recent seller signals, and operational radar panels for draft blockers, blocked cash-out requests, and action-required orders
- `/seller` now also surfaces the TCOS Under-$20 Seller Protection reserve in the command-center metrics and payout-pressure card, including protected/liability row counts, protected item amount, 2% reserve withheld, and shipping excluded from reimbursement
- the seller command center now also deep-links each workspace card, pressure panel, and recent seller signal into the exact needs-work, blocked, action, shipping, or signal-focused view when a smarter handoff exists
- seller inventory, orders, and payout pages now honor incoming URL filter/search parameters so dashboard and workspace handoffs open the intended view immediately
- seller inventory, orders, and payout filters now also keep the browser URL in sync so the current view is refresh-safe, bookmarkable, and shareable after the seller changes filters
- seller order detail links now preserve originating seller order or payout context, and the detail page returns sellers to the right view instead of dumping them back into the generic order list
- seller order detail now also includes direct order and payout action cards plus timeline handoffs so one order can launch the seller back into the exact workflow view that needs attention
- seller order detail payout rows and review cases now also expose contextual order or payout links so the seller can jump straight from a specific issue into the matching workflow view
- seller order cash-out request links now open the payout workspace in the correct request view with the specific request pre-searched instead of relying on a loose anchor jump
- seller payout links into order detail now preserve the originating payout view, and order detail can return the seller back to that payout workspace instead of only the generic order list
- seller inventory handoff buttons now route into action-order, shipping, or blocked-payout views when the current listing state or bulk failure reason makes a more precise seller workflow jump available
- seller payout navigation and empty-state recovery now route into action-order, cash-out, or completed order views based on the active payout request view so sellers can jump into the matching order workspace without resetting context first
- seller order navigation and empty-state recovery now route into blocked, attention, open, or paid payout views based on the active order workspace so the seller can cross from order work into the matching payout view without losing workflow context
- the seller command-center header now routes Inventory, Payouts, and Orders into the hottest needs-work, blocked, open, action, or shipping view instead of dropping sellers at generic workspace roots

- the seller inventory header now routes Orders and Payouts into shipping, action, blocked, or open views based on active listing focus, draft cleanup pressure, and payout-verification fallout
- the seller order-detail header now routes Payouts into blocked, attention, open, paid, or general payout views based on the live payout pressure tied to that routed order
- blocked cash-out order chips and action-order cards on the seller command center now jump straight into seller order detail with the right order or payout return context
- payout-related seller signals on the command center now open the matching payout view directly, and their order-detail links return sellers back to blocked or open payout work instead of only the order workspace
- seller order workspace shortcut cards now include direct jumps into their matching blocked, attention, open, or paid payout view instead of making the seller open orders first and payouts second
- seller payout workspace shortcut cards now include direct jumps into their matching action-order, cash-out, or completed order view so sellers can pivot across workspace states without resetting context
- seller payout request cards and blocked-hold summaries now route directly into the matching action, shipping, cash-out, or completed order view based on the request's real order pressure instead of only linking into order detail
- seller order detail now routes cash-out claims into action, shipping, cash-out, or completed order views based on live request pressure, and seller-facing payout actions now use clearer cash-out wording
- seller home blocked cash-out cards now include direct jumps into action-order and blocked-payout request views, and seller payout summary labels now use clearer cash-out wording
- seller order workspace cash-out request cards now include direct jumps into action, shipping, cash-out, or completed order views based on live request pressure, and seller home payout summary labels stay aligned with the cash-out wording
- seller signal cards on the command center and seller order detail now label payout actions with the actual blocked or cash-out view they open instead of using generic payout wording
- seller marketplace import draft-output links now open the seller draft inventory workspace directly, and draft-output guidance now calls that handoff by name instead of sending sellers to a generic inventory root
- seller marketplace draft-output links now choose the most useful draft view, and recent promoted inventory cards can jump straight into the matching seller inventory workspace for that item
- seller home draft blockers now include direct seller-workspace jumps for each item, and seller-home inventory or order recovery buttons now name the exact needs-work or action view they open
- seller order cards now turn recent signal rows into direct order, payout, and seller-detail actions so sellers can react from the order list instead of opening the detail page first
- top-level seller order signals now include direct blocked-payout or cash-out-payout actions when the signal is payout-related, and their seller-detail label now matches the rest of the workspace
- seller cash-out request buttons inside order list and order detail now name the exact payout view they open, including blocked, cash-out, paid, or attention views
- seller home and top-level seller order signals now label action jumps with the exact action, shipping, cash-out, completed, or seller-order view they open, and seller-home signal detail links now use the same seller-detail wording as the rest of the workspace
- seller inventory order buttons now carry the current listing title into the target seller order workspace so shipping, action, or seller-order jumps land on the relevant collectible instead of a broad search

- remaining seller views that route into the open payout-request view now call it the cash-out payout view instead of the older generic open-payout wording
- seller inventory can now open marketplace review with staged-row search context from the current listing, and the marketplace workspace can initialize its filter/search state from those incoming query parameters
- seller inventory summary and toolbar marketplace shortcuts now carry readiness or search context into seller marketplace review so needs-work, ready-stage, and item-search jumps land in the right staged view
- the seller command center now reads the latest staged import summary and turns its marketplace card and header shortcut into direct blocked, needs-review, ready, mapped, or general marketplace links based on live sync pressure
- seller marketplace draft-output links now reuse the smart seller draft workspace target, and promotion-result controls now distinguish between showing promoted rows inside marketplace review versus opening the seller draft inventory workspace
- promotion-result rows now include direct seller draft-workspace links using the promoted title or SKU, so sellers can open the matching seller-owned inventory record without detouring through admin first
- blocked, failed, and already-promoted marketplace conflict matches now expose seller-workspace links whenever the duplicate belongs to the same seller or store-owned inventory, while keeping the admin product links for deeper review
- seller inventory bulk success and failure follow-up cards now include direct seller-workspace links for the affected listing, and remaining admin product links use the same open-in-admin wording as the rest of the seller workspace
- the seller inventory header marketplace shortcut now carries the current inventory search and readiness context into seller marketplace review, and marketplace admin-only exits now consistently use the same open-in-admin wording as the rest of the seller workspace
- seller orders and seller payouts now turn their inventory header shortcuts into context-aware seller inventory links, using the current order or payout filter plus usable search text to land on needs-work drafts, active inventory, or a focused seller inventory search
- the seller order detail header now routes its inventory shortcut into active or general seller inventory with single-item search context when available, instead of always dropping sellers at the inventory root
- seller orders, seller payouts, and seller order detail now turn their marketplace header shortcuts into search-aware seller marketplace links, carrying usable listing text or single-item context instead of always opening the generic marketplace root
- the seller order detail login gate now preserves the page's return order or payout context instead of always sending logged-out sellers back to the generic seller orders root
- seller command center action cards now reuse the same smart workspace routing as the header shortcuts, so idle states fall back to seller inventory, seller payouts, or seller orders instead of forcing empty ready, shipping, or cash-out views
- the seller marketplace page now uses its live store counts to steer inventory and payout shortcuts toward active inventory, seller drafts, or payout setup instead of treating every workspace jump as a generic root link
- the account page now routes seller payout entry points into blocked or cash-out payout views when pressure exists, and hold-context shortcuts now open the seller action-order workspace instead of the generic seller order root

- blocked payout request chips and hold-context cards on the account page now open seller order detail with the blocked-payout return view preserved, so the seller can inspect an order and still jump back into the right payout workspace
- seller payout request cards on the account page now include direct jumps into the matching blocked, cash-out, paid, or general payout view plus the corresponding seller order view, instead of only showing request status in place
- the account page seller-verification card now reads the latest staged import summary and routes its marketplace button into blocked, needs-review, ready, mapped, or general seller marketplace review instead of always opening the generic marketplace root
- seller orders and seller payouts now steer their marketplace header shortcut into needs-review marketplace cleanup when the seller is already working blocked or attention-heavy views, while clean views keep the broader marketplace search handoff
- seller order detail now uses live blocked payout and review pressure to send its marketplace shortcut into needs-review cleanup when that order is under active hold pressure, while clean order detail pages keep the broader marketplace search handoff
- seller command-center empty-state buttons for draft blockers, blocked cash-outs, and action orders now reuse the same smart workspace fallbacks as the header and action cards, instead of dropping sellers into views that may already be empty
- the account page seller payout shortcut now shows payout setup when seller verification is still incomplete, while still routing straight into blocked or cash-out payout views when live request pressure exists
- draft blockers on the seller command center now include direct needs-review marketplace search links for the affected collectible, so sellers can pivot from a blocked draft into marketplace cleanup without first opening seller inventory
- seller order detail item rows now include direct seller inventory and marketplace search links for each collectible, and blocked orders steer those marketplace item links into needs-review cleanup instead of a broad marketplace search
- seller order workspace item rows now include direct seller inventory and marketplace search links for each collectible, and review-heavy orders steer those marketplace item links into needs-review cleanup instead of a broad marketplace search
- seller order detail payout rows now include direct seller inventory and marketplace search links for the affected collectible, and blocked orders steer those marketplace row links into needs-review cleanup instead of a broad marketplace search
- review pressure cards on the seller order workspace now include direct action, blocked-payout, and order-detail links for each case, so sellers can jump from a live case summary into the exact workflow without opening the order first
- action-order cards on the seller command center now include direct order-view, payout-view, and order-detail buttons, so homepage pressure can jump straight into action, shipping, cash-out, blocked payout, or order detail without forcing a single generic click path
- blocked seller payout request cards now turn their affected-order list into direct action-order and order-detail handoffs, preserving blocked payout return context instead of treating those orders as detail-only chips
- blocked cash-out cards on the seller command center now turn their linked-order list into direct action-order and order-detail handoffs, so homepage payout pressure can jump straight into the right order

workflow without a generic detour

- blocked hold-context cards on the seller payout page now include direct blocked-payout view links with the specific order search context preserved, so held cash-out pressure can reopen the exact payout workspace instead of only relying on local focus state
- top blocker rows on the seller inventory page now act as direct focus controls, jumping sellers into draft needs-work inventory with the affected listings preselected instead of leaving blocker counts as passive summary text
- seller inventory item cards now include direct payout workspace handoffs, sending active listings into cash-out payouts and other listings into seller payouts with item search context preserved
- recent seller inventory cards on the marketplace page now also include direct seller-order and seller-payout workspace handoffs, so imported listings can pivot into shipping, action, seller orders, cash-out payouts, or seller payouts without leaving the marketplace workspace first
- recent seller inventory cards on the marketplace page now include direct marketplace-row search links using eBay item ID, SKU, or title, with blocked drafts steering into needs-review cleanup, ready drafts steering into the ready stage, and non-draft items keeping the broader marketplace search handoff
- seller inventory bulk success and failure follow-up cards now include direct marketplace review links for the affected listing, and draft items with blockers steer those links into needs-review cleanup instead of a broad marketplace search
- main seller inventory row actions now reuse the same smart marketplace routing as the follow-up cards, so blocked drafts open needs-review cleanup, ready drafts open the ready stage, and non-draft listings keep the broader marketplace search handoff
- seller payout linked-order cards now include direct workspace handoffs for the specific order, steering into action, shipping, cash-out, completed, or general seller-order views based on the live payout and fulfillment pressure already shown on the card
- `/seller/orders` now gives sellers a dedicated order and payout activity workspace scoped to their own routed items, including payout statuses, active case pressure, tracking state, open cash-out claim counts, and a fresh seller signals feed that summarizes recent order, payout, shipping, and review movement
- `/seller/orders` now also supports seller-side order filters for action-required orders, shipping-needed orders, cash-out-linked orders, completed orders, cash-out request summaries with payout deep links, and text search across order IDs, item titles, case titles, carriers, tracking numbers, and request references
- the seller order workspace now also includes shortcut cards plus recent-signal focus actions that can jump straight into action-required, shipping, cash-out, or completed order views
- recent activity cards on seller order detail now name the exact order view they open, so shipment, cash-out, completed, and action signals match the explicit order wording already used on seller home and the order workspace
- the seller home payout-pressure card now reuses the same live payout workspace shortcut logic as the rest of the dashboard, so its CTA opens blocked payouts, cash-out payouts, or the general seller payout workspace based on real request pressure instead of a stale two-branch shortcut
- the seller home inventory-pulse card now reuses the same live inventory workspace shortcut logic as the rest of the dashboard, so its CTA opens needs-work drafts, ready drafts, or the general seller inventory workspace based on real listing pressure instead of a stale two-branch shortcut
- seller home and seller order signal cards now consistently label their drilldowns as order detail links, matching the rest of the seller workspace instead of mixing in older seller-detail wording

- draft blocker cards on the seller command center now also include direct action-order handoffs scoped by listing title, so seller cleanup can pivot from a blocked draft into the surrounding order view without stopping at inventory first
- seller order and seller payout shortcut cards now label their primary buttons with the exact order or payout view they open instead of generic shortcut wording, matching the explicit handoff style used everywhere else in the seller workspace
- blocked hold-context cards on the seller payout page now use the same Open Order Detail wording as the rest of the seller workspace instead of keeping a shorter one-off order label
- seller inventory bulk follow-up controls now use explicit inventory workspace labels such as Open Seller Inventory instead of vaguer fallback wording, keeping bulk recovery actions aligned with the naming used across the seller workspace
- seller home section footer links now use the same Open wording as the rest of the seller workspace, replacing older Review labels on needs-work drafts, blocked payouts, and action orders
- seller inventory and seller marketplace cleanup panels now name their failed follow-up actions directly as failed inventory or failed promotions instead of sharing vague fallback wording
- marketplace diagnostics now label their view buttons with explicit Open wording, so ready, review, blocked, and mapped jump actions match the rest of the seller workspace instead of using shorter stage-only labels
- seller payout filters and empty-state recovery buttons now refer to blocked, cash-out, paid, and attention requests directly instead of calling those request views generic buckets
- blocked hold-context cards on the seller payout page now refer to focusing blocked requests in the plural, matching the summary card's multi-request hold context instead of implying only one blocked request exists
- the seller home inventory workspace shortcut now uses the fuller Needs Work Drafts label, so dashboard calls-to-action match the rest of the seller inventory language instead of shortening that workspace name
- the seller inventory sidebar now uses Open Action Orders alongside its other explicit footer actions, instead of leaving the order handoff as the shorter Action Orders label
- failed-promotion cleanup controls now use ready, review, and conflict view wording directly, so marketplace recovery buttons match the same language used by the rest of the staging diagnostics
- seller home and seller payout action buttons now render with explicit Open wording for order and payout handoffs, so direct workspace jumps read like actions instead of unlabeled stage names
- seller order list and order detail actions now use Action Orders, Shipping Orders, Cash-Out Orders, and Completed Orders wording instead of the older workflow phrasing, so order handoffs read the same way as the rest of the seller workspace
- seller inventory item cards now use explicit Open Shipping Orders, Open Action Orders, Open Seller Orders, and payout handoff wording, so item-level jumps match the action language used throughout the rest of the seller workspace
- seller workspace header shortcuts now render inventory, payout, order, and marketplace destinations with explicit Open wording unless they are already search or return actions, so the top navigation chips read like actions across the whole seller surface
- seller order, payout, and marketplace handoff controls now also prefix their remaining visible workspace jump buttons with Open wording, removing the last raw Seller Orders and payout workspace labels from seller-facing controls

- seller marketplace inventory preview cards now use explicit Open Shipping Orders, Open Action Orders, Open Seller Orders, and Open Seller Payouts wording, so preview-card jumps match the rest of the seller workspace action language
- seller inventory and marketplace cards now use Open Seller Inventory and Open Admin Product wording for product-level handoffs, so item drilldowns read more clearly than the older Open In Seller Workspace and Open In Admin labels
- generic seller marketplace jumps now use Marketplace Rows wording across seller home, inventory, orders, payout, and order-detail surfaces, so the unscoped marketplace destination keeps one clear name while stage-specific review and ready labels stay intact
- the seller marketplace dashboard now uses Seller Payout Setup as its payout fallback label, so that header shortcut reads like a proper seller workspace destination instead of the shorter Open Payout Setup phrasing
- seller inventory and marketplace section headings now use Seller Inventory Workspace and Seller-Safe Build Progress wording, so those surface titles match the rest of the seller UI instead of keeping older workflow phrasing
- seller home marketplace workspace labels now use Blocked Marketplace Rows, Ready Marketplace Rows, and Mapped Marketplace Rows, so the stage-specific seller-home chips stay aligned with the broader Marketplace Rows naming
- `/seller/orders/[id]` now gives sellers a scoped drilldown with their item rows, payout ledger rows, deduplicated cash-out request summaries, cross-order payout request links, shipping/tracking state, linked seller-visible review cases, and a recent activity timeline for that order
- `/seller/marketplaces` shows the seller marketplace connection dashboard with live Store #1 inventory/eBay stats and the seller-safe connector build progress panel
- `/admin/order-review-cases` shows the global admin case queue across chargebacks, returns, authenticity issues, payment risk, shipping issues, and seller disputes
- `/admin/orders/[id]` shows order review cases and can open chargeback, return, authenticity, payment-risk, shipping, and seller-dispute case files against an order, download case packet PDFs, and apply seller payout release/reversal/appeal resolution directly from the order screen
- case packet PDF downloads compile order details, TOS/IP evidence, shipping evidence, order items, case notes, case event history, seller payout hold context, Dag Danky Holdings LLC fee rows, and transaction evidence report references
- downloading a case packet saves or refreshes an `order_review_case_packets` record so the packet appears under `/admin/files`
- `/api/admin/order-review-cases/[id]/payout-resolution` can release held seller payout rows to eligible after a seller-favorable decision, reverse/cancel held rows after a buyer-favorable decision, or keep related rows held for appeal; release skips rows whose order only has dry-run shipping proof
- `/api/admin/seller-payouts/requests` now blocks approve, processing, and paid transitions whenever the request is tied to active order review cases, held/cancelled payout rows, or dry-run shipping rows
- `/api/account/seller/payout-onboarding` starts or resumes Stripe-hosted Express onboarding
- the seller payout workspace now also includes request-view shortcut cards, stronger empty-state recovery, and hold-context focus actions that jump straight into blocked request cleanup
- `/api/account/seller/marketplace-connections` returns seller-scoped marketplace connection records and saves seller connection requests for the logged-in account

- `/api/account/seller/marketplace-connections` list responses include `X-TCOS-Seller-Marketplace-Connections`, `X-TCOS-Seller-Marketplace-Connected`, `X-TCOS-Seller-Marketplace-Requested`, `X-TCOS-Seller-Marketplace-Sync-Errors`, and `X-TCOS-Seller-Marketplace-Providers`; save responses include `X-TCOS-Seller-Marketplace-Connection-Mutation`, `X-TCOS-Seller-Marketplace-Connection-Provider`, `X-TCOS-Seller-Marketplace-Connection-Status`, and `X-TCOS-Seller-Marketplace-Sync-Status` headers so seller connection health can be reconciled without exposing connection IDs, provider account IDs, provider account labels, OAuth scopes, token timestamps, sync error text, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/auth` starts seller-safe eBay OAuth for the logged-in account
- `/api/account/seller/marketplace-connections/ebay/auth` responses include `X-TCOS-Seller-Marketplace-Ebay-Auth-Mutation`, `X-TCOS-Seller-Marketplace-Ebay-Auth-Status`, `X-TCOS-Seller-Marketplace-Ebay-Auth-Provider`, `X-TCOS-Seller-Marketplace-Ebay-Auth-Store-Sync`, `X-TCOS-Seller-Marketplace-Ebay-Auth-Connection-Status`, and `X-TCOS-Seller-Marketplace-Ebay-Auth-Sync-Status` headers so OAuth-start outcomes can be reconciled without exposing authorization URLs, signed OAuth state, eBay client IDs, OAuth scopes, connection IDs, provider account IDs, provider account labels, token data, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/status` refreshes the seller eBay token and updates connection health without touching the Store #1 `ebay_tokens` path
- `/api/account/seller/marketplace-connections/ebay/status` responses include `X-TCOS-Seller-Marketplace-Ebay-Status-Mutation`, `X-TCOS-Seller-Marketplace-Ebay-Status`, `X-TCOS-Seller-Marketplace-Ebay-Identity-Verified`, and `X-TCOS-Seller-Marketplace-Ebay-Identity-Warning` headers so token-refresh health can be reconciled without exposing access tokens, refresh tokens, provider account IDs, provider account labels, OAuth scopes, token timestamps, identity usernames, identity user IDs, raw eBay error text, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/sync-control` responses include `X-TCOS-Seller-Marketplace-Sync-Control-Mutation`, `X-TCOS-Seller-Marketplace-Sync-Control-Action`, `X-TCOS-Seller-Marketplace-Sync-Control-Result`, `X-TCOS-Seller-Marketplace-Sync-Control-Unchanged`, `X-TCOS-Seller-Marketplace-Sync-Control-Connection-Status`, and `X-TCOS-Seller-Marketplace-Sync-Control-Sync-Status` headers so pause/resume outcomes can be reconciled without exposing connection IDs, provider account IDs, provider account labels, token IDs, token timestamps, store settings details, sync error text, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/disconnect` responses include `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Mutation`, `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Result`, `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Already`, `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Connection-Status`, `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Sync-Status`, and `X-TCOS-Seller-Marketplace-Ebay-Disconnect-Credentials-Deleted` headers so disconnect/revoke outcomes can be reconciled without exposing connection IDs, provider account IDs, provider account labels, OAuth scopes, token IDs, token timestamps, stored token keys, provider metadata, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/import-preview` responses include `X-TCOS-Seller-Marketplace-Import-Preview-Status`, `X-TCOS-Seller-Marketplace-Import-Preview-Requested-Limit`, `X-TCOS-Seller-Marketplace-Import-Preview-Sampled`, `X-TCOS-Seller-`

`Marketplace-Import-Preview-Total-Available`, `X-TCOS-Seller-Marketplace-Import-Preview-Has-More`, `X-TCOS-Seller-Marketplace-Import-Preview-Write-Blocked`, `X-TCOS-Seller-Marketplace-Import-Preview-Ready`, `X-TCOS-Seller-Marketplace-Import-Preview-Needs-Review`, `X-TCOS-Seller-Marketplace-Import-Preview-Missing-SKU`, `X-TCOS-Seller-Marketplace-Import-Preview-Missing-Listing-ID`, `X-TCOS-Seller-Marketplace-Import-Preview-Missing-Price`, and `X-TCOS-Seller-Marketplace-Import-Preview-Missing-Image` headers so import quality and write-block pressure can be reconciled without exposing preview listing IDs, SKUs, titles, image URLs, prices, provider account IDs, connection IDs, token data, raw eBay error text, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/reconcile` responses include `X-TCOS-Seller-Marketplace-Reconcile-Mutation`, `X-TCOS-Seller-Marketplace-Reconcile-Status`, `X-TCOS-Seller-Marketplace-Reconcile-Linked`, `X-TCOS-Seller-Marketplace-Reconcile-Recent-Runs`, `X-TCOS-Seller-Marketplace-Reconcile-Scanned`, `X-TCOS-Seller-Marketplace-Reconcile-Matched`, `X-TCOS-Seller-Marketplace-Reconcile-Quantity-Reduced`, `X-TCOS-Seller-Marketplace-Reconcile-Sold`, `X-TCOS-Seller-Marketplace-Reconcile-Review`, `X-TCOS-Seller-Marketplace-Reconcile-Failed`, `X-TCOS-Seller-Marketplace-Reconcile-Has-More`, and `X-TCOS-Seller-Marketplace-Reconcile-Reset-Cursor` headers so inventory reconciliation progress can be reconciled without exposing run IDs, connection IDs, provider account IDs, listing IDs, SKUs, titles, inventory item IDs, cursor offsets, token data, raw eBay error text, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/orders` responses include `X-TCOS-Seller-Marketplace-Order-Import-Mutation`, `X-TCOS-Seller-Marketplace-Order-Import-Status`, `X-TCOS-Seller-Marketplace-Order-Import-Orders`, `X-TCOS-Seller-Marketplace-Order-Import-Paid`, `X-TCOS-Seller-Marketplace-Order-Import-Refunded`, `X-TCOS-Seller-Marketplace-Order-Import-Recent`, `X-TCOS-Seller-Marketplace-Order-Import-Imported-Orders`, `X-TCOS-Seller-Marketplace-Order-Import-Imported-Items`, `X-TCOS-Seller-Marketplace-Order-Import-Inventory-Reduced`, `X-TCOS-Seller-Marketplace-Order-Import-Sold`, `X-TCOS-Seller-Marketplace-Order-Import-Unmatched`, `X-TCOS-Seller-Marketplace-Order-Import-Review`, `X-TCOS-Seller-Marketplace-Order-Import-Failed-Items`, `X-TCOS-Seller-Marketplace-Order-Import-Has-More`, and `X-TCOS-Seller-Marketplace-Order-Import-Reset-Cursor` headers so outside-order import pressure can be reconciled without exposing order IDs, provider order IDs, event keys, listing IDs, SKUs, buyer data, order totals, cursor windows, connection IDs, token data, or seller account IDs in headers
- `/api/admin/order-review-cases` opens and updates admin order review cases, logs identity evidence, and can hold related seller payout rows
- the seller must accept Seller Terms before payout onboarding starts
- seller TOS acceptance is recorded through `tos_acceptance_events`
- Stripe collects and verifies bank/payout details; TCOS does not collect raw checking account or routing numbers
- `seller_payout_accounts` stores Stripe Connect account ID, onboarding status, payout flags, due requirements, disabled reason, and seller TOS evidence
- `seller_marketplace_connections` stores marketplace provider, seller account label, connection status, sync status, token reference/expiry metadata, last sync timing, and sync error state; it does not store raw OAuth secrets in this first foundation slice
- seller eBay OAuth reuses `/api/ebay/callback` through signed state so the existing Store #1 eBay app redirect can support seller-safe connections without touching the global `ebay_tokens` path

- encrypted seller marketplace tokens are stored separately from `ebay_tokens`
- seller eBay staging now supports a seller-side review view with readiness counts, stage-status filters, staged-row search, warning badges such as missing SKU/listing ID, and direct admin links to already promoted draft products
- staged seller rows are loaded through `/api/account/seller/marketplace-connections/ebay/staged-items?limit=100`, keeping review work seller-scoped without writing directly into live store inventory
- `/api/account/seller/marketplace-connections/ebay/staged-items` list responses include `X-TCOS-Seller-Marketplace-Staged-Rows`, `X-TCOS-Seller-Marketplace-Staged-Ready`, `X-TCOS-Seller-Marketplace-Staged-Draft-Cleanup`, `X-TCOS-Seller-Marketplace-Staged-Needs-Review`, `X-TCOS-Seller-Marketplace-Staged-Mapped`, `X-TCOS-Seller-Marketplace-Staged-Skipped`, `X-TCOS-Seller-Marketplace-Staged-Blocked`, `X-TCOS-Seller-Marketplace-Staged-Promoted`, and `X-TCOS-Seller-Marketplace-Import-Jobs` headers; stage-batch and review-edit responses include `X-TCOS-Seller-Marketplace-Staged-Mutation`, `X-TCOS-Seller-Marketplace-Staged-Count`, `X-TCOS-Seller-Marketplace-Staged-Skipped`, `X-TCOS-Seller-Marketplace-Staged-Updated`, `X-TCOS-Seller-Marketplace-Staged-Target-Status`, and `X-TCOS-Seller-Marketplace-Staged-Has-More` headers so staging cleanup pressure can be reconciled without exposing staged row IDs, source listing IDs, SKUs, titles, image URLs, duplicate product IDs, import job IDs, or seller account IDs in headers
- `/api/account/seller/marketplace-connections/ebay/staged-items/promote` responses include `X-TCOS-Seller-Marketplace-Promote-Mutation`, `X-TCOS-Seller-Marketplace-Promote-Mode`, `X-TCOS-Seller-Marketplace-Promote-Requested`, `X-TCOS-Seller-Marketplace-Promote-Succeeded`, `X-TCOS-Seller-Marketplace-Promote-Failed`, `X-TCOS-Seller-Marketplace-Promote-Partial`, and `X-TCOS-Seller-Marketplace-Promote-Status` headers so promotion success, partial failure, and conflict pressure can be reconciled without exposing staged row IDs, source listing IDs, SKUs, titles, draft product IDs, inventory item IDs, or seller account IDs in headers
- the Seller Connections page converts those safe response headers into `Latest Marketplace API Receipt` and `Recent Marketplace API Receipts` cards; operators can use `Copy Safe Receipt` or `Download Safe Receipt` for one event, or `Copy Trail` / `Download Trail` for the current browser-tab trail without exposing OAuth tokens, seller account IDs, row IDs, listing IDs, SKUs, titles, order IDs, buyer data, or raw provider errors
- the receipt trail is stored only in browser `sessionStorage`, limited to five safe receipt summaries, and can be intentionally removed with `Clear Trail`; it is an operator handoff aid, not an audit ledger, payment record, fulfillment proof, or provider reconciliation source of truth
- the staged-items API and seller marketplace page now also surface recent seller import job history so the seller can review row counts, staged counts, skipped counts, error counts, and latest run timing without needing admin access
- recent seller import jobs now also store diagnostic metadata, including skip-reason rollups, quality-signal counts, request limits, and returned eBay totals so cleanup pressure is visible right on `/seller/marketplaces`
- each recent import run can now also focus the staging table to that specific job, making it easier to inspect only the rows touched by that run and then clear back to all staged rows
- focused import-run review now reloads staged rows by `import_job_id` from the API, so large seller imports are not limited to the default staged-row fetch window during cleanup

- each recent seller import run now also shows a live outcome snapshot with ready, review, blocked, mapped, promoted, and skipped counts so the seller can see where that run stands after cleanup begins
- those run snapshots now also include one-click shortcuts into the exact ready, review, blocked, or mapped view for that specific import job
- those run view shortcuts now also preselect the matching rows from that import job so seller bulk actions can start without another selection pass
- import run outcome cards now also show cleanup state and resolved progress so sellers can scan which import batches are complete, in progress, or still waiting on work
- import run controls now also include a one-click remaining-work selection that gathers the unresolved rows for that exact run into the active bulk selection
- the staged-row bulk action bar now shows how the current selection is split across ready, review, blocked, mapped, and skipped rows before the seller runs a bulk action
- the staged-row bulk bar now also explains the safest next move for the current selection, especially when a run-level shortcut loads a mixed selection of ready, review, and blocked rows
- when that mixed-selection guidance appears, the seller can now trim the current selection down to just ready, review, or blocked rows without rebuilding the selection manually
- if a selection mixes active work with already completed mapped/skipped rows, the guidance panel can now reduce it to active work only or completed rows only in one click
- bulk action buttons now show the exact number of rows they will touch and skip rows that are already in the requested status, so the seller gets cleaner feedback on mixed queues
- bulk promotion now clears only the rows that actually promoted into draft inventory, leaving failed or still-needed rows selected for the next seller cleanup action
- if some rows fail during bulk promotion, the seller now gets a follow-up panel with error snippets and a one-click way to keep only the failed rows selected
- that failed-promotion follow-up panel can now also move failed rows directly into `needs_review`, making it easier to park problem rows for cleanup right away
- when a promotion failure is caused by an existing product conflict, the follow-up panel now shows the conflict reason and direct admin product links for faster cleanup
- that same follow-up panel now also summarizes how many failed rows are conflicts, review rows, or still-ready rows, and can isolate conflict-only failures in one click
- conflict-only failures can now jump directly into the blocked staging view with those failed rows preselected for faster duplicate cleanup
- failed promotion rows can now also jump directly into ready-retry or review views, so the seller can move each failure type into the right workflow without rebuilding selections
- if some failed rows are still clean enough to promote, the follow-up panel can now retry just those ready failures directly from the panel
- the same follow-up panel can now reopen the entire failed subset in the staged workspace before the seller splits it into ready, review, or conflict cleanup groups
- successful promotions now also surface a seller-side results panel with promoted draft links, mapped-row handoff, and a direct jump into `/seller/inventory`
- the seller marketplace page now also includes a post-import action board with one-click focus for ready rows, needs-review rows, blocked conflicts, and promoted draft output so the seller can move straight from import results into cleanup or promotion work

- staged seller rows now include pre-promotion conflict guards that check for existing store products by `ebay_item_id` and `sku`, so sellers can see duplicate blockers and jump straight to the conflicting admin product before promotion
- seller draft promotion now requires the staged row status to be `staged`; `needs_review`, `mapped`, and `skipped` rows must be intentionally moved back into `staged` before promotion is allowed
- staged seller review now supports bulk selection with batch moves to `staged`, `needs_review`, or `skipped`, making large seller import cleanup faster without touching live store inventory
- staged seller review now also supports bulk promotion of selected ready rows, using the same single-item promotion safeguards for each row and refreshing the staged board after the batch completes
- the seller marketplace page now includes a conflict review dashboard that groups blocked reasons, lists recent blocked rows, and links straight to conflicting admin products for faster duplicate cleanup
- `/api/account/seller/inventory` and the seller marketplace page now show draft-output metrics plus recent seller-owned inventory created through staged promotion, so sellers can see what the import workflow actually produced
- `/api/account/seller/inventory` responses include `X-TCOS-Seller-Inventory-Items`, `X-TCOS-Seller-Inventory-Drafts`, `X-TCOS-Seller-Inventory-Draft-Ready`, `X-TCOS-Seller-Inventory-Draft-Needs-Work`, `X-TCOS-Seller-Inventory-Active`, `X-TCOS-Seller-Inventory-Archived`, `X-TCOS-Seller-Inventory-InstaComp-Drafts`, `X-TCOS-Seller-Inventory-InstaComp-Ready`, `X-TCOS-Seller-Inventory-Standard-Envelope`, and `X-TCOS-Seller-Inventory-Protection-Opt-In` headers so seller inventory pressure can be reconciled without exposing inventory item IDs, SKUs, titles, image URLs, marketplace IDs, or seller account IDs in headers
- `/seller/inventory` gives sellers a dedicated inventory workspace with readiness filters, blocker rollups, and quick links back to marketplace review and seller orders
- `/seller/inventory` can now activate a ready seller draft directly into live inventory after payout verification is active and readiness blockers are clear
- `/seller/inventory` can now also pause live listings, archive seller drafts, and reactivate archived listings without deleting the linked product record
- `/seller/inventory` now includes a seller-safe editor for title, price, quantity, and description, plus shared regenerate and AI description tools without requiring admin access
- `/seller/inventory` now supports bulk row selection, select-visible shortcuts, bulk activation for ready listings, and bulk archive controls for faster seller cleanup
- seller inventory bulk actions now keep failed rows selected after partial success, surface row-level failure details, and offer one-click follow-up into active, archived, needs-work, or seller payout views
- the seller inventory workspace now also summarizes mixed selections, offers one-click trimming to ready, needs-work, draft, active, or archived rows, and only sends eligible listings into each bulk action
- seller inventory listing editor now captures authenticity status, autograph source, certification provider/number, seller pass-guarantee authenticators, provenance evidence, and buyer-facing authenticity notes
- admin product editing now stores the same authenticity/provenance disclosure fields, and product pages render those disclosures as storefront trust badges plus a buyer-facing authenticity callout
- seller draft output now highlights draft activation readiness, including blockers such as missing SKU, missing price, missing quantity, or missing primary image before a seller tries to move a draft toward live inventory

- Stripe `account.updated` webhooks refresh seller payout status
- `account_store_memberships` gets a `seller` role with `payout_verification_required` until Stripe reports the seller payout account active

Seller constants live in:

```
src/lib/legal.ts
```

Current: InstaComp AI Collectable Scan Assist

The sports-card scanning foundation is implemented as InstaComp. The dedicated scanner is at `/admin/instacomp`, and the same production scanner is embedded at `/admin/products/new`.

Current behavior:

- accepts up to 500 card rows in one durable batch job
- accepts front-only cards but performs better with front/back pairs
- pairs files by explicit front/back filename signals when available; otherwise it pairs upload 1 with 2, 3 with 4, and so on
- registers and confirms rows in chunks of at most 50 and uploads original images directly to private Supabase Storage
- can recover a saved server job after a page reload or browser restart and resume eligible unfinished rows
- creates targeted contrast, inverted, edge, band, and serial-stamp crops in the browser for the multipart fallback path; the local PaddleOCR service creates its own targeted serial crops for durable jobs that supply only the original card sides
- sends no more than the configured OCR image limit through the PaddleOCR path
- uses PaddleOCR first when `PADDLEOCR_API_URL` is configured
- can use Google Vision as an optional OCR fallback
- uses OpenAI vision for structured card identification and a dedicated serial-number pass
- prefers printed back evidence for year, set, card number, and manufacturer when front/back evidence conflicts
- searches configured TCOS, eBay, sold/marketplace providers, and broader research providers; COMC is kept as checklist/reference context and is not used in the active comp equation
- displays player, year, brand, set, card number, parallel, serial number, team, sport, condition clue, confidence, comps, and OCR diagnostics
- shows `Buy Me on TCOS` and `Trade For Me on TCOS` lookup actions so the detected card can be searched against TCOS sale and trade surfaces
- creates seller-owned drafts when a valid seller session owns the job, or store-owned drafts when an admin session owns the job
- lets a seller-owned scan row be added to Available for Trade instead of creating a sell draft; a row cannot do both
- refreshes a stored seller session before long-running authenticated queue and draft requests when the token is close to expiration
- never activates or publicly publishes those drafts automatically

- keeps low-confidence, incomplete identity, uncertain-parallel, front-only, weak-pairing, and missing-comp rows in `review_required`; Auto-Pilot does not silently create drafts from those rows
- uses item leases and bounded retry attempts so an interrupted row can be reclaimed without two workers completing the same attempt
- supports thumbnail zoom review and 45-degree left/right local image rotation before retry; rotation uses the original local file plus cumulative degrees so filenames do not stack repeated `rotated-right-45` suffixes and repeated clicks do not shrink the preview
- pauses on database connection pressure, backs browser concurrency down by one, and resumes with five-card claim mini-packs to reduce queue round-trips

The browser currently acts as the queue worker. Uploaded jobs and results survive a reload, but OCR and AI work do not continue while every InstaComp browser tab is closed. This is a durable, resumable browser-driven queue, not a detached background worker.

The complete operator procedure, local PaddleOCR startup, diagnostics, and failure recovery are in `Section 32: InstaComp Production Operation`.

The system must not rely on AI guessing alone. AI and OCR propose facts; the operator must compare those facts with both card images and trusted references before activation, pricing, or cross-listing. A displayed confidence score is evidence for triage, not a guarantee of exact identity.

Future scanner catalog identity requirement:

- next build priority as of July 15, 2026: build `InstaComp™ Multi-Scanner Consensus` before the final 100-card tester pass
- each card should be submitted to independent AI/OCR readers that produce structured findings for year, manufacturer, set, card number, player, team, parallel/insert, visible variation clues, serial text, front/back OCR, and confidence evidence
- readers should not simply copy one another; TCOS should compare independent findings, detect disagreements, and escalate only the uncertain cards to an additional reader or operator question
- adaptive consensus escalation is now the required speed posture: obvious complete cards stay on the `fast_lane` / `fast_lane_council`, while low-confidence cards, missing critical fields, front-only/weak pairings, serial-numbered signals, autograph/relic signals, uncertain identity text, insert-style card-number prefixes such as `O-8`, `C-369`, `UD3-28`, or `S-50` that are still being treated as generic Base, or printed variant cues such as Outliers/Clear Cut/Limited Red escalate to `escalated_multi_ai` / `full_council` and run a second independent AI identity reader before comps are trusted
- the consensus layer must use checklist/catalog truth as the referee, not majority vote alone, so two scanners calling a card `Base` cannot override a confirmed `Outliers`, `Clear Cut`, `Canvas`, `Limited Red`, `Future Watch`, serial run, autograph, relic, or other printed/cataloged variation
- the review packet should preserve a concise reason trail showing which reader saw which clue, which checklist/catalog candidate matched, why the final identity won, and what still needs review
- consensus confidence should unlock speed on easy cards and stricter review on valuable, rare, serial-numbered, or conflicting cards; if the consensus/catalog evidence is incomplete, the row must stay `Needs Review`
- InstaComp should connect to one or more approved online card catalog/checklist sources so scanned cards can be matched against structured card identity data before comps are trusted

- catalog provider adapters must build a lookup plan from scanned identity fields, filter out any source that lacks API availability or TCOS-approved commercial/storage/display permissions, and keep exact comp trust blocked until at least one approved source can be queried
- this catalog layer should show known cards and their variations, including base cards, parallels, refractors, short prints, image variations, autographs, relics/patches, serial-numbered print runs, inserts, promos, grading/certification identifiers when available, and manufacturer checklist identifiers
- every approved source policy must declare variation coverage for base cards, parallels, refractors, short prints, image variations, autographs, relics/patches, and serial-numbered print runs so the scanner can show where a catalog is strong or incomplete
- provider lookup results must be aggregated before identity resolution: candidates from approved providers may enter the exact-match pool, candidates from unapproved or unknown providers must be ignored as trusted evidence, and provider failures, timeouts, empty results, and ignored-source reasons must remain visible for operator review
- the scanner review packet should show the selected catalog match, alternate matches, provider summaries, warnings, review reasons, suggested operator action, and safe-use boundary; exact comps, public rare-variant claims, auto-pricing, and trade-value recommendations remain blocked unless the packet is `ready_for_exact_comps`
- scan rows and seller drafts should save a compact catalog evidence snapshot with schema `tcos.instacomp.catalogEvidence.v1`, selected source attribution, action-permission flags, review blockers, and audit flags so later activation, exact comps, pricing, and trade decisions can prove why catalog identity was trusted or blocked
- catalog identity matching should happen before or alongside pricing lookup, so InstaComp can use catalog-confirmed year, brand, set, card number, player, team, parallel/variant, serial-number range, and checklist ID when pulling comps
- the scanner should compare OCR/vision facts, front/back image clues, card-number text, back-of-card fine print, serial numbers, color/foil/refractor pattern, autograph/relic markers, and visible logos against the catalog candidate list
- when one catalog candidate is strongly supported, InstaComp should attach the catalog/source ID, source label, source URL, match explanation, and confidence evidence to the scan row and draft listing
- when catalog evidence cannot prove the exact variation, InstaComp must keep the row in `review_required` and ask for one targeted operator confirmation instead of guessing
- catalog identity data and pricing/comps data must remain separate; catalog data answers “what exact card is this?” while InstaComp/comps answer “what is this confirmed card worth?”
- exact comp search should be trusted only behind a catalog identity comp gate: `catalog_confirmed` rows may use catalog-normalized fields for exact comps, while `review_required` rows must preserve review reasons and block trusted exact-comp claims until an operator confirms the identity
- every catalog source must be evaluated for API availability, licensing, caching limits, attribution rules, and whether TCOS may store, display, or use the data commercially
- the operator UI should clearly show the catalog-confirmed match, alternate plausible matches, mismatched clues, and why the system accepted or rejected each candidate
- no public listing, exact rare-variant claim, auto-price, or trade-value recommendation should depend on catalog data unless the selected source is permitted and the match evidence is saved with the scan

- the product goal is practical catalog-confirmed identity, not unsupported “100% certainty”; if the source data or image evidence is incomplete, TCOS should say `Needs Review` rather than overclaim

Future category expansion must support all collectables, not only sports cards.

Collectable categories to support over time:

- sports cards
- trading card games
- comics
- coins
- currency
- stamps
- autographs
- memorabilia
- sealed wax/product
- toys
- graded collectables
- limited edition collectables
- other cataloged collectables

Data-source research checklist:

- find official APIs first
- confirm licensing and allowed use before storing or displaying data
- prioritize card catalog/checklist identity sources that expose structured card lists, set/checklist IDs, card numbers, variation/parallel names, image references where allowed, and print-run/serial-number metadata
- prefer catalog IDs, cert numbers, set names, card numbers, issue years, print runs, population data, and verified images
- separate catalog identity data from pricing data
- record source names, source URLs, lookup timestamps, confidence scores, and raw evidence
- never auto-list a collectable from AI recognition without admin confirmation

Candidate source categories to evaluate:

- eBay sold/active marketplace data
- PriceCharting/SportsCardsPro pricing data
- PSA certification, population, CardFacts, and auction data
- TCGplayer catalog/pricing data for trading card games
- COMC card marketplace/catalog data
- Beckett/card checklist references
- customer-authorized PSAcard.com account linking so collectors can import owned PSA-graded cards, cert data, population/card facts when permitted, and any supported account collection data
- customer-authorized Beckett.com account/subscription linking so collectors can use permitted Beckett pricing/checklist data inside their own TCOS collection dashboard

- manufacturer checklists when available
- grading-company certification lookups
- comic, coin, stamp, toy, and memorabilia catalog databases
- broad collectable catalog/reference sites such as Colnect-style catalogs
- Google Programmable Search as a fallback discovery layer

Future linked-provider portal requirements:

- build a secure connected-accounts page for collector-owned provider logins, tokens, or authorized import sessions
- never store raw third-party passwords when OAuth, API tokens, or import files are available
- encrypt any provider access tokens and separate them from public profile data
- log provider name, account owner, scopes, sync status, last sync time, and revocation status
- let collectors disconnect a provider and stop future syncs
- keep provider data source labels visible anywhere pricing or collection data is displayed
- respect each provider's licensing, subscription, caching, and redistribution rules

Future collection analytics goals:

- import PSA-owned card data into the user's collection shelf when permitted
- use Beckett subscription pricing/checklist data for the user's own dashboard when permitted
- track what the collector paid, current estimated value, realized sale price, and date sold
- show collection profit/loss, unrealized gain/loss, realized gain/loss, cost basis, current value, and performance over time
- chart collection value by category, player/character, team/franchise, set, grader, grade, and acquisition source
- support CSV/catalog exports that include provider source, paid price, current value, sold price, and profit/loss fields

Scan Assist output should include:

- likely collectable title
- category
- manufacturer/brand
- year or era
- set or series
- card/comic/coin/stamp/catalog number when available
- player/character/person/franchise
- condition clues visible from scan
- grade and cert number when visible
- serial number, autograph, patch, variant, parallel, or limited mark when visible
- rarity notes
- value range with source breakdown
- confidence score
- missing information warnings

- recommended next action

Autograph, certification, and provenance disclosure policy:

- use plain, buyer-facing labels such as `Verified Cert`, `Seller Pass Guarantee`, `Provenance Evidence Included`, and `Unverified Autograph - Sold As-Is`
- require autograph source disclosure when relevant, such as in-person, through-the-mail (TTM), fan club return, private signing, inherited, estate-sourced, or acquired secondhand
- if a seller includes provenance evidence without third-party certification, the listing must not imply that the item is third-party authenticated
- if a seller has an envelope, letter, event ticket, signing photo, receipt, or other provenance support, the listing should identify that evidence near the top of the description and show it in listing photos when available
- if a seller offers a pass guarantee, the named authenticator(s), claim basis, and refund consequence must be stored with the listing and transaction
- if the listing is unverified and sold as-is, that risk warning must be shown as a badge, a seller-side required field, and a product-page warning instead of being buried in long description text
- provenance evidence can help the buyer make an informed decision, but provenance is not equal to third-party certification unless the listing clearly says so
- buyers may purchase unverified autographs at their own disclosed risk, but sellers remain responsible for false, misleading, or unsupported authenticity claims

Variant and parallel resolver requirements:

- TCOS should identify the exact variation or parallel whenever visual evidence and checklist data support it
- InstaComp should query the approved catalog identity layer before trusting comp matches, so base/parallel/refractor/short-print/image-variation/autograph/relic/serial-numbered cards do not get priced as the wrong card
- every selected variant should preserve the matched catalog/checklist ID, source label, source URL, and the visible evidence that made the match
- do not show a long list of near-duplicate options when the evidence resolves the match
- if the card is a refractor, TCOS should not ask whether it is pink, blue, green, gold, base, wave, mojo, cracked ice, shimmer, lava, scope, x-fractor, atomic, numbered, or unnumbered unless the scan/checklist evidence cannot prove the answer
- use front/back scan evidence, border color, foil pattern, refractor effect, serial numbering, card number, set name, year, manufacturer, player, team, logo marks, autograph/relic markers, and visible checklist identifiers
- use online checklists and manufacturer/reference sources only where access is allowed by terms or official API
- compare visual/color clues against checklist variant names and numbering ranges
- use serial numbering to narrow parallels when the checklist defines print runs
- use back-of-card codes and fine print when visible
- collapse candidates into one selected match when confidence is high and source evidence agrees
- if multiple variants remain plausible, ask one targeted question instead of showing dozens of options

- show why TCOS picked the match, including visible clues and source evidence
- show `Needs Review` instead of guessing when evidence is incomplete
- admin approval is required before auto-listing, repricing, or publicly claiming an exact rare variant

Target identification flow:

1. Extract visible facts from images.
2. Normalize title/player/team/year/set/card number/manufacturer.
3. Query approved checklist/catalog sources.
4. Compare variant names, colors, patterns, serial-number ranges, and card numbers.
5. Score candidates by evidence match.
6. Select the exact match if one candidate is clearly supported.
7. Ask one targeted follow-up if the remaining candidates differ by a detail the image cannot prove.
8. Save evidence, confidence score, source URLs, and reviewer decision.

Future data tables should store:

- scan event
- uploaded image references
- AI extracted fields
- matched catalog candidates
- selected catalog match
- source evidence
- value estimate snapshot
- admin approval decision
- variant_resolver_runs
- variant_resolver_candidates
- variant_resolver_evidence
- checklist_source_matches

Future: Expanded Product Detail Collector Intelligence

The first source-link version is implemented on product pages. The expanded version should make each collectable page feel alive by showing verified trend data, saved population-report snapshots, official social context, and current news without inventing hype or scraping restricted sources.

Target route:

```
/product/[id]
```

Goal:

- show trend signals for the player, character, franchise, team, brand, or exact collectable
- show grading population reports when a grade/cert/company is known
- link to official or approved social profiles such as X, Instagram, YouTube, team pages, manufacturer pages, or franchise pages

- show relevant news or milestones when available
- explain the item's current collector story in plain language
- help the buyer understand rarity, demand, timing, and context before purchase

Collector Intelligence panel should evaluate:

- sales velocity from TCOS/eBay/comps when available
- recent sold-price movement
- watch/search activity when available from allowed sources
- player/team news such as big games, awards, trades, records, injuries, call-ups, retirements, or playoff runs
- character/franchise news such as new movie, show, game, comic arc, anniversary, limited release, or manufacturer announcement
- grading population for the exact grade when cert/set/card details are known
- related social profiles and official links
- related collector content such as manufacturer checklist, league/team bio, player page, franchise page, or grading certification page

Trending labels must be evidence-based:

- `Trending Up` only when recent source data supports it
- `Steady Demand` when comps/activity are consistent
- `Cooling Off` when recent source data supports lower demand
- `Not Enough Data` when TCOS cannot prove a trend

TCOS must not show fake urgency, fake scarcity, fake endorsements, or unsourced claims.

Grading/population report requirements:

- support PSA, CGC, SGC, Beckett/BGS, and other grading companies only when lookup access is permitted
- prefer official cert lookup or official population data
- store grading company, grade, cert number, population count, lookup URL, lookup timestamp, and source confidence
- show `population unavailable` instead of guessing
- never imply a cert is verified unless the source confirms it

Social and news requirements:

- use official APIs, official embeds, RSS feeds, licensed news/search APIs, or manually approved links
- never bypass platform limits, scraping protections, login walls, or terms
- never imply a player, team, manufacturer, grading company, league, or franchise endorses TCOS unless there is a written sponsorship/partnership
- public social links should open to the source instead of copying restricted content into TCOS
- summarize news in TCOS language with source links and timestamps
- separate factual news from AI-generated collector commentary

AI collector commentary may help write:

- why this item is interesting
- what recently happened around the player, team, character, franchise, or brand
- what a pop report means in simple terms
- why collectors may care about the grade, parallel, serial number, autograph, patch, rookie status, set, or era

AI collector commentary must:

- cite stored sources
- say when data is missing
- avoid investment advice
- avoid guaranteed future value claims
- avoid medical, legal, or financial claims
- be reviewed or source-gated before public display

Product page display should include:

- trend badge
- short collector story
- latest relevant news link list
- official social/source links
- grading pop report card when grade/cert data exists
- last updated timestamp
- source list
- `Not Enough Data` state when sources are missing

Future data tables should store:

- collectable_intelligence_snapshots
- collectable_trend_signals
- collectable_news_links
- collectable_social_links
- collectable_grading_reports
- collectable_source_evidence
- collectable_intelligence_refresh_jobs
- collectable_intelligence_admin_reviews

Future: Brag Session And Collection Board

This is a future-build community feature. It should not copy eBay-style transactional feedback.

TCOS community ratings should use collector-style tiers:

- `Gold`
- `Silver`
- `Bronze`

These tiers should represent post-purchase collector satisfaction, community trust, and brag-session quality after moderation rules exist. They should not be implemented as a direct eBay positive/neutral/negative clone.

Goal:

- give buyers a fun way to show off what they received
- let collectors post collection photos, mail days, favorites, displays, and stories
- build community around collectables instead of only buyer/seller ratings
- keep transaction issues, chargebacks, and evidence packets separate from public community posts

Buyer post-purchase flow:

- after delivery, invite the buyer to create a `Brag Session`
- buyer can upload photos of the item or collection
- buyer can add title, story, category, tags, and optional order/item link
- buyer can choose public, private, or admin-review-only visibility
- public posts must go through moderation before appearing

Collection board:

- users can show collection shelves, slabs, binders, sealed product, memorabilia, or themed collections
- posts can be grouped by category, sport, franchise, player, set, era, or custom tags
- users can like/save/comment only after community safety controls are built
- admin can feature posts on storefront/community pages

Safety and moderation rules:

- no addresses, tracking labels, order numbers, phone numbers, payment details, or private customer information in public posts
- image uploads must be scanned/moderated before public display
- admin must be able to approve, hide, remove, or feature posts
- reporting tools should exist before public comments go live
- public community posts must not expose chargeback evidence, IP evidence, TOS audit records, or private order records
- community reputation must stay separate from seller compliance, payout, fraud, and transaction evidence workflows

Future data tables should store:

- community profile
- brag session post
- collection board post
- post images
- post moderation status
- tags/categories
- optional order item reference
- likes/saves/comments after moderation is ready
- abuse reports and admin actions

Future: Trades And Collector Swaps

This is a future-build marketplace/community feature. It should let collectors propose trades safely after buyer/seller accounts, identity controls, moderation, shipping evidence, and dispute rules exist.

Goal:

- let collectors mark items as open for trade
- let collectors build trade offers from their collection
- support one-for-one and multi-item trade proposals
- support cash difference only if payments, tax, and dispute rules are designed
- give each side a clear trade review screen before accepting
- protect both parties with identity, address, shipping, condition, and evidence controls
- keep trades separate from normal store inventory until a trade is accepted and locked

Trade platform product model:

- evaluate keeping the trade platform separate from auctions, store checkout, and normal marketplace buying/selling
- consider a simple `2 USD/month` trade membership so collectors have a lightweight billing relationship before they can trade
- the membership should help keep billing address, customer identity, and trade participation more honest without turning trades into a full marketplace checkout flow
- trade membership should increase confidence in the system, discourage ghosting, and make repeat scammers easier to identify, pause, or remove
- the trade platform should still avoid live money, postage, payouts, or checkout behavior until trade-specific billing, cancellation, tax, privacy, and dispute rules are designed

Trade pricing access model:

- the `2 USD/month` trade subscription should include a basic book-price style value reference for trade fairness, not full InstaComp access
- InstaComp should stay reserved for the next paid tier up so collectors have a clear reason to upgrade when they want deeper comps, confidence scoring, and research links
- upgraded pricing access should still respect marketplace limits, usage limits, trade limits, anti-scraping protections, and any collector-intelligence fairness rules
- trade offers should clearly label whether values come from book-price guidance, user-entered declared values, or upgraded InstaComp/collector-intelligence pricing
- the basic book-price reference should help both sides discuss fair value without implying a guaranteed sale price, cash offer, payout amount, tax basis, or appraisal

Trade reputation model:

- trade rankings should use the same collector-facing tier language as auctions: `Gold`, `Silver`, and `Bronze`
- show each collector's career successful trade count on their trade profile
- track completion quality, disputes, ghosting, cancellation history, and admin actions separately from raw completed-trade count

- define a Collecting Hall of Fame milestone for collectors who reach `1000` successful career trades with a `100%` successful-trade record
- Hall of Fame collectors should receive a special ribbon, trophy, or similar badge next to their username across the collectable community
- Hall of Fame status should be revocable or reviewable if later fraud, abuse, counterfeit activity, or serious dispute history is discovered

Trade section entry points to evaluate:

- `/trades` public trade browsing page
- product page `Open To Trade` signal when the owner allows it
- user collection item `Trade This` action
- want ad match to trade offer
- admin trade review queue
- post-purchase prompt to add an item to collection/trade board after delivery

Trade offer data should include:

- initiating user
- receiving user
- offered item list
- requested item list
- optional cash difference only if enabled
- item photos
- condition notes
- grade/cert details when available
- declared value from comps when available
- shipping method
- tracking numbers for both sides
- trade status
- timestamps for offer, counter, acceptance, shipment, delivery, dispute, and completion

Required safety rules:

- require user accounts before trades can exist
- require TOS acceptance for trade terms
- require verified email and recommended multi-factor authentication
- do not expose addresses until both sides accept and shipping flow requires it
- keep address and identity data private
- require item condition photos before acceptance
- record IP/user-agent evidence for trade acceptance events
- require tracking on both shipments
- allow admin freeze/review on suspicious trade activity
- prohibit off-platform payment pressure, scams, harassment, counterfeit claims, and unsafe meetups
- provide report/dispute tools before public trading goes live

Trade fulfillment models to evaluate:

- direct ship: both collectors ship to each other with tracking
- admin-mediated trade: both collectors ship to TCOS first, TCOS verifies, then forwards
- local meetup: future option only if safety, location privacy, and event rules are designed

Admin-mediated trade is safer but operationally heavier. Direct ship is easier but higher risk. TCOS should not enable public trades until the chosen model has clear rules, evidence packets, and dispute handling.

Trade evidence packets should store:

- accepted trade terms
- TOS version
- IP/user-agent evidence
- item photos at acceptance
- declared condition and value
- shipping labels/tracking
- delivery proof
- user messages related to the trade
- admin actions
- dispute notes

Future data tables should store:

- trade_profiles
- trade_items
- trade_offers
- trade_offer_items
- trade_offer_messages
- trade_acceptance_events
- trade_shipments
- trade_evidence_packets
- trade_disputes
- trade_admin_reviews
- trade_terms_acceptance

Future: Collector Map, Card Shows, And Shop Finder

This is a future-build discovery feature. It should help collectors find card shows, collectable shows, local card shops, hobby stores, trade nights, release events, grading events, and community meetups near them.

Goal:

- search by ZIP code, city, state, or current location with consent
- show nearby card shops and collectable shops on a map
- show upcoming card shows and collectable events by radius
- support geofenced regions for promoted local events
- use AI to help discover and normalize show/event information from permitted sources

- help collectors plan where to go, what to bring, and whether an event fits their collecting interests

Card show submission model:

- card show and collectable event posting should be free when the listing includes verified information
- required verified information should include event name, organizer or promoter contact, date, venue, address, city/state/ZIP, hours, admission, category, and a public detail link when available
- event submissions should be reviewed before public display until trusted organizer accounts and automated validation are designed
- private Facebook groups, private message boards, invite-only communities, or non-public event posts must not be copied into TCOS unless the organizer or authorized group admin submits the listing and grants permission to display it on the TCOS website
- submitted private-group listings should store permission evidence, submitter identity, source URL or group name, submission timestamp, and admin review status
- public source discovery should link back to the original source and avoid implying TCOS owns, sponsors, verifies attendance, or guarantees event details unless TCOS has direct organizer confirmation
- collectors should be able to report outdated, wrong, canceled, duplicate, or suspicious event listings
- organizers should eventually be able to claim and maintain their own show profile after identity verification

Card show map MVP scope:

- decision as of July 15, 2026: build the MVP card-show finder first, then preserve and continue the post-MVP roadmap after the MVP is working
- build the MVP version first after the current go-live/live-money goal is finished
- MVP should focus on a useful national card-show finder without overbuilding the full events platform
- MVP should include a `/card-shows` discovery page with ZIP/city/state search, radius filtering, map pins, and a list view
- MVP should support manually/admin-entered events and free promoter-submitted show listings
- MVP submissions should go to an admin review queue before public display
- MVP event cards should show event name, date, venue, address, hours, admission, category, source/detail link, and verification status
- MVP should include a simple `Submit A Show` form, permission confirmation for private-group/private-message-board listings, and a `Report Wrong Info` action
- MVP should link out to official show websites, public source pages, or submitted detail links instead of trying to replace the organizer's event page

Card show map post-MVP roadmap:

- after the MVP works, continue improving the roadmap with organizer claim flows, recurring events, richer filters, duplicate detection, and better geocoding
- add verified organizer badges and trusted organizer accounts after the admin review workflow proves stable
- add source refresh checks so stale or canceled events can be flagged before collectors rely on them
- add saved searches and alerts for collectors who want notified when shows appear near their ZIP code, favorite city, or collecting category

- add city/state SEO pages only after the event data quality and permission rules are reliable
- evaluate promoted local events only after sponsored-result labeling, billing, moderation, and geofencing rules are designed
- preserve the larger shop finder, trade night, grading event, autograph signing, and community meetup roadmap for later phases

Collector search should support:

- ZIP code
- city/state
- radius
- date range
- category, such as sports cards, TCG, comics, coins, toys, memorabilia, autographs, or mixed collectables
- event type, such as show, trade night, signing, release, grading submission event, shop event, or auction preview
- shop/event name
- free/paid admission
- family-friendly flag
- vendor/table information when available

Map and data-source candidates to evaluate:

- Google Maps Platform Places API for shop/place discovery
- Google Maps Geocoding API for ZIP/city lookup
- Mapbox or similar map provider as an alternate map/search provider
- Eventbrite or other event APIs where terms allow event discovery
- promoter-submitted event listings
- shop-owner submitted listings
- admin-entered shows and shops
- Google Programmable Search as a fallback discovery layer
- public show calendars where licensing/terms allow use
- social/event advertising sources only when access is permitted by their terms or an official API

AI event discovery should:

- parse event title, location, dates, times, promoter, admission, vendor tables, categories, and source URL
- deduplicate repeated listings for the same show
- assign confidence scores
- flag missing or conflicting details
- require admin approval before publishing uncertain events
- keep raw source evidence and lookup timestamps

Geofencing and promotion rules:

- geofenced promotions should use ZIP, city, radius, or coarse region targeting by default
- precise location should require clear user consent
- do not store exact user location unless the feature truly needs it and the user has opted in

- allow collectors to search by ZIP without enabling device location
- disclose when a result is sponsored or promoted
- keep sponsored placement separate from organic relevance
- never sell or expose private user location data

Collector-facing event pages should include:

- event/shop name
- address and map
- date/time
- distance from searched area
- event category
- admission cost
- promoter/shop contact link when available
- source link
- confidence/verification status
- last verified timestamp
- notes for collectors
- related nearby shops/events

Future data tables should store:

- shops
- shows/events
- event venues
- promoters
- event categories
- geofenced promotion campaigns
- source evidence
- verification status
- admin approvals
- user saved events
- opt-in location preferences

Future: AI Search, Want Ads, And Site Help

This is a future-build discovery and support feature.

Goal:

- help collectors find the exact item they want
- understand natural-language collector searches
- recommend matching products, categories, saved searches, shows, shops, and content
- create a Want Ad when TCOS does not have the item
- help users navigate the site and answer technical questions
- escalate support questions to a future technical support email inbox when AI cannot answer safely

AI search should support:

- player/person/character/franchise
- year, set, brand, card number, issue number, catalog number, cert number, or serial number
- sport/category
- grade, condition, raw/graded/sealed
- autograph, patch, parallel, variant, refractor, short print, rookie, insert, or limited mark
- price range
- seller-owned inventory when seller accounts exist
- active storefront products
- sold/reference/comps context when useful
- collector intent, such as buying, researching, completing a set, finding a gift, or valuing an item

Search behavior:

- return direct product matches first
- show close matches and explain why they match
- show confidence and missing details
- ask clarifying questions when the request is ambiguous
- do not invent facts or pretend an item is available
- offer to create a Want Ad if no suitable item is found

Want Ads:

- run for 30 days by default
- can be renewed by the user
- should expire automatically if not renewed
- should support status values such as active, matched, expired, renewed, canceled, and fulfilled
- should allow item details, category, desired condition/grade, budget, notes, and optional images
- should notify admin when a new Want Ad is created
- should notify the user when matching inventory appears
- should not publicly expose private email, phone, address, IP, payment, or account details
- should be moderated before public/community display if Want Ads become public

AI site help should:

- answer site navigation questions
- explain how checkout, offers, tracking, orders, TOS, evidence/security, and community features work
- guide collectors to product pages, cart, orders, Brag Sessions, Collection Board, shows/shops, and support
- avoid legal, tax, medical, financial, or authentication-sensitive claims beyond approved policy text
- avoid exposing private admin, customer, seller, payout, IP, or evidence data
- escalate unresolved technical issues to support

Technical support escalation:

- future env var should define the support inbox

- likely name: `TECHNICAL_SUPPORT_EMAIL`
- user should be able to submit name, email, issue type, order ID if relevant, and message
- AI conversation context should be summarized, not forwarded with private hidden data
- support submissions should be saved for admin review
- email forwarding should be added after the support mailbox is chosen

Future data tables should store:

- ai_search_sessions
- ai_search_messages
- want_ads
- want_ad_matches
- want_ad_renewals
- support_requests
- support_request_messages
- support_email_delivery_status

Future: Sponsorships, Advertising, And Partner Outreach

This is a future-build revenue feature. It should help TCOS sell advertising and sponsorship placements to collectable-related companies without creating spam, privacy risk, or low-quality ads.

Goal:

- create advertising inventory on TCOS
- sell logo placements, sponsored placements, newsletter spots, show/shop placements, and category sponsorships
- help collectable companies reach collectors through relevant pages
- create a compliant sponsor outreach workflow
- track leads, conversations, packages, contracts, payment status, creative assets, start/end dates, and performance

Potential sponsor categories:

- card shops
- card show promoters
- grading companies
- supplies companies
- storage/display companies
- auction houses
- breakers and stream sellers
- collectable insurance companies
- memorabilia companies
- comic, coin, toy, stamp, and TCG businesses
- local shops promoted by ZIP, city, state, radius, or event category

Ad inventory to evaluate:

- homepage sponsor block
- shop/category sponsor block
- product-page related sponsor block
- Collector Map sponsor pins
- card show/event sponsor placement
- Brag Session/Collection Board sponsor placements after moderation tools exist
- newsletter/email sponsor slot after subscriber consent tools exist
- admin-approved banner/logo placements

Sponsor packages should define:

- placement
- audience/category
- region or geofence
- price
- duration
- impressions/clicks when tracking is built
- creative/logo requirements
- prohibited content
- approval workflow
- renewal rules

Outreach rules:

- do not scrape or harvest email addresses in ways that violate laws, terms, or platform rules
- do not send deceptive mass email
- use truthful sender identity, subject lines, and offer language
- include Dag Danky Holdings LLC / Truely Collectables contact information
- include a valid physical mailing address when sending commercial outreach
- include a clear opt-out/unsubscribe method
- honor opt-out requests within 10 business days
- keep a suppression list and never email suppressed contacts again except as required for compliance
- separate one-to-one relationship outreach from marketing blasts
- prefer warm leads, opt-in lists, business cards, show contacts, inbound sponsor forms, and manually verified public business contacts

TCOS can help with:

- sponsor media kit copy
- rate card
- sponsorship package definitions
- compliant outreach email templates
- lead tracker
- outreach status pipeline
- follow-up reminders

- opt-out/suppression tracking
- sponsor asset uploads
- ad placement scheduling
- invoice/payment tracking
- performance reporting after analytics is built

TCOS should not send sponsor outreach until:

- `SPONSOR_OUTREACH_FROM_EMAIL` is configured
- `SPONSOR_OUTREACH_REPLY_TO_EMAIL` is configured
- `SPONSOR_OUTREACH_PHYSICAL_ADDRESS` is configured
- opt-out handling is built
- suppression list handling is built
- a reviewed outreach template is approved
- contacts are sourced and verified through allowed methods

Reference:

- FTC CAN-SPAM Act Compliance Guide

Future data tables should store:

- sponsors
- sponsor_contacts
- sponsor_leads
- sponsor_outreach_messages
- sponsor_outreach_suppression_list
- sponsor_packages
- ad_placements
- ad_creatives
- ad_campaigns
- ad_impressions
- ad_clicks
- sponsor_invoices
- sponsor_payments

Future: Marketing Campaigns, Social Posting, And Collectable Of The Day

This is a future-build promotion feature. It should help TCOS advertise the website, promote listings, and keep social channels active without spam, fake engagement, deceptive posting, or platform policy problems.

Goal:

- create organized advertising campaigns for TCOS
- schedule approved promotional posts across connected social channels
- automatically feature a Collectable of the Day
- generate platform-specific captions, hashtags, and safe tags

- promote store listings, card shows, sponsor campaigns, collection posts, want ads, and major inventory drops
- track campaign status, approvals, channels, post history, clicks, conversions, and engagement
- avoid repetitive posting, unwanted tagging, or random website posting that looks like spam

Supported campaign types to evaluate:

- Collectable of the Day
- new inventory drop
- featured category
- local card show or shop spotlight
- sponsor spotlight
- holiday/event promotion
- price drop
- buyer brag/collection highlight after customer permission
- want ad spotlight
- newsletter promotion after subscriber consent tools exist

Collectable of the Day automation should:

- select one eligible product per day from active inventory
- avoid sold, hidden, draft, or blocked products
- prefer items with strong photos, clean title, price, category, and description
- create a short caption for each connected platform
- generate hashtags from product title, category, brand, manufacturer, player, team, set, year, grade, sport, franchise, rarity, and condition
- tag manufacturers, teams, players, leagues, or brands only when the tag is accurate, relevant, and not excessive
- avoid tagging companies or public figures repeatedly just to force attention
- include the product link and clear call to action
- support admin preview and approval before publishing
- record exactly what was posted, where it was posted, and when it was posted

Posting controls should include:

- enabled channels
- posting calendar
- daily/weekly posting limits
- quiet hours
- duplicate detection
- campaign approval status
- link tracking
- image selection
- generated caption review
- generated hashtag review

- generated tag review
- failure/retry handling
- admin override

Web and community promotion rules:

- do not randomly post to unrelated websites
- do not bypass moderation, captchas, posting limits, or site rules
- do not use bots to create fake engagement, fake comments, or fake accounts
- post only where TCOS has permission, an account, an official integration, or clear allowed community participation
- respect each site's terms, community rules, rate limits, and promotional posting policies
- keep a record of where TCOS is allowed to post and what kind of promotion is allowed there
- prefer official APIs, approved social integrations, owned channels, paid advertising accounts, newsletters, and permission-based communities

Social channels to evaluate:

- Facebook page and groups where promotion is allowed
- Instagram business account
- TikTok business account
- X account
- YouTube Shorts
- Pinterest
- LinkedIn company page for sponsor/business updates
- Reddit only in communities where promotional posting is allowed by the subreddit rules
- Discord servers only where the server owner permits promotions
- email newsletter after subscriber consent tools exist

Campaign copy should be generated in the TCOS voice:

- collector-first
- accurate
- excited but not misleading
- no fake scarcity
- no fake endorsements
- no unsupported price claims
- no unauthorized claim that a player, manufacturer, team, league, or brand sponsors TCOS

TCOS should not publish automated posts until:

- at least one official social account is connected
- platform API access or posting method is approved
- admin approval workflow is built
- duplicate/spam controls are built
- channel-specific rules are saved
- link tracking is configured

- image/caption/tag review is enabled
- opt-in newsletter consent exists for email campaigns

Future data tables should store:

- marketing_campaigns
- marketing_campaign_channels
- marketing_campaign_posts
- marketing_campaign_assets
- marketing_campaign_post_approvals
- marketing_campaign_post_metrics
- collectable_of_the_day_candidates
- collectable_of_the_day_posts
- social_accounts
- social_channel_rules
- social_posting_limits
- social_post_failures
- campaign_link_clicks

23. Security And Data Protection

Security is required for owner, buyer, and seller data.

Current implemented protections:

- admin sessions use a signed HTTP-only cookie instead of a plain `true` cookie
- admin session cookies are secure, same-site lax, and expire after 24 hours
- site usage can be blocked when VPN, proxy, Tor, relay, hosting, or anonymous IP risk is detected
- blocked site users see: `Sorry, you must turn off your proxy or VPN to use this website.`
- admin pages are protected by `src/proxy.ts`
- sensitive admin-style API routes are protected by the same admin session
- security headers are applied globally from `src/proxy.ts`
- admin/API responses are marked `no-store`
- Stripe webhooks require Stripe signature verification
- customer TOS acceptance is required before checkout and offer submission
- customer TOS acceptance records the server-observed public IP, user agent, IP risk, and request header evidence
- checkout creates a `tos_acceptance_events` audit row before Stripe checkout is created
- completed transactions create `transaction_evidence_reports` for chargeback/legal packets
- offer submission stores TOS/IP evidence before the offer is accepted
- public offer submission validates product ID, customer name, customer email, offer amount, and current inventory availability before saving the offer
- public checkout is rate-limited to 12 attempts per 10 minutes per IP/account subject when `public_endpoint_rate_limit_events` is migrated

- public offer creation is rate-limited to 8 attempts per 15 minutes per IP/customer/product subject
- collector binding-offer payment setup is rate-limited to 6 attempts per hour per IP/account subject
- seller payout onboarding is rate-limited to 5 attempts per hour per IP/account subject
- `/admin/security` shows recent public money-path events, blocked events, watch events, unique IPs, endpoint counts, identity risk, and header evidence summary
- `/admin/security/ip/[ip]` shows a focused dossier for one IP across login attempts, money-path attempts, TOS events, orders, offers, and transaction evidence reports
- `/admin/security/ip/[ip]` lets admins save a persistent investigation status, severity, and internal notes
- `/admin/launch-readiness` checks whether the public endpoint rate-limit audit table is available
- `/admin/launch-readiness` checks whether the security IP investigation table is available
- buyer account signup starts accounts in `payment_verification_required` status when card verification is required
- Stripe Checkout setup mode collects the buyer card and billing address before TCOS activates the account
- signed Stripe webhook completion marks the account active only when Stripe returns a card payment method with complete United States billing address evidence; otherwise the account stays pending
- TCOS stores Stripe-safe card proof, such as customer ID, setup intent ID, payment method ID, card brand, last 4, expiry, funding type, billing name, billing address fields, billing country, billing postal code, verification check timestamp, and failure reason when applicable
- pending card-verification accounts cannot log in or use authenticated account APIs as active customers
- buyer account signup can proceed to Stripe verification when configured identity checks detect VPN, proxy, Tor, relay, hosting, or anonymous IP use; buyer login and money-path routes remain subject to identity controls
- buyer account signup/login locks out repeated failures after six failed attempts inside 15 minutes
- bank credentials are not stored in TCOS

Buyer-account anti-fraud requirement:

- buyer/customer account signup requires a valid payment card and billing/address evidence through Stripe unless `ACCOUNT_CARD_VERIFICATION_REQUIRED=false`
- card verification requires complete United States billing address evidence before account activation
- failed or canceled card verification prevents account activation and login
- TCOS must never store raw card numbers, CVV, or payment credentials

Important IP rule:

TCOS can log and enforce the server-observed public client IP. If a customer hides behind a VPN, proxy, Tor, relay, or hosting network, TCOS cannot discover the hidden residential IP by itself. Production masking detection requires a third-party IP intelligence provider configured through environment variables.

Routes currently protected by admin session:

- `/admin/*`
- `/ebay/*`
- `/api/admin/*` except `/api/admin/login`

- `/api/ebay/*`
- `/api/orders/*`
- `/api/offers/update-status`
- `/api/offers/counter`

Public routes that must remain public:

- `/shop`
- `/product/[id]`
- `/cart`
- `/terms`
- `/seller-terms`
- `/api/checkout`
- `/api/offers/create`
- `/api/account/collector/binding-offers`
- `/api/account/seller/payout-onboarding`
- `/api/account/seller/marketplace-connections`
- `/api/webhook`
- `/api/stripe/webhook`

Webhook routes are exempt from the site-wide identity gate so Stripe can deliver signed webhook events.

Required future seller-account protections:

- use a real identity/auth provider for buyer and seller accounts
- require email verification before buying or selling account use
- require multi-factor authentication for sellers and admins
- use a third-party provider for bank verification and payouts
- store only provider IDs, verification status, timestamps, and non-sensitive payout metadata
- never store raw bank account numbers, routing numbers, SSNs, tax IDs, passwords, or payment card data in TCOS
- encrypt sensitive internal notes or documents if they are ever added
- restrict seller data so one seller cannot read another seller's account, payout, auction, or customer data
- audit seller-account changes, payout changes, bank verification changes, and admin overrides
- add fraud review and payout hold controls before seller payouts go live

Security files:

```
src/lib/admin-session.ts
src/lib/client-identity.ts
src/lib/public-endpoint-rate-limit.ts
src/lib/tos-acceptance.ts
src/proxy.ts
```

24. Clean Fake Orders

To clear fake local orders, run this in Supabase SQL Editor:

```
truncate table order_items, orders restart identity cascade;
```

This clears only local orders.

It does not delete:

- products
- inventory
- eBay listings
- eBay inventory
- offers
- eBay tokens

If fake orders reduced quantity, run eBay sync afterward.

25. Environment Variables

Core:

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
NEXT_PUBLIC_SITE_URL=
```

`SUPABASE_SERVICE_ROLE_KEY` is server-only and can bypass Row Level Security. Never expose it through a `NEXT_PUBLIC_` name, browser code, screenshots, or public logs.

Admin:

```
ADMIN_PASSWORD=
ADMIN_SESSION_SECRET=
```

Stripe:

```
STRIPE_SECRET_KEY=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_WEBHOOK_SECRET=

STRIPE_TEST_SECRET_KEY=
NEXT_PUBLIC_STRIPE_TEST_PUBLISHABLE_KEY=
STRIPE_TEST_WEBHOOK_SECRET=
```

```
STRIPE_LIVE_SECRET_KEY=
NEXT_PUBLIC_STRIPE_LIVE_PUBLISHABLE_KEY=
STRIPE_LIVE_WEBHOOK_SECRET=

TCOS_LIVE_PAYMENTS_ENABLED=false
TCOS_MONTHLY_SUBSCRIPTION_ENABLED=false
STRIPE_FINANCIAL_EVENTS_VERIFIED=false
STRIPE_LIVE_FINANCIAL_EVENTS_VERIFIED=false
```

The unsuffixed Stripe variables are compatibility fallbacks. Keep test and live credentials separate. Never enable the monthly subscription flag merely because live website payments are approved.

eBay:

```
EBAY_CLIENT_ID=
EBAY_CLIENT_SECRET=
EBAY_ENVIRONMENT=production
MARKETPLACE_TOKEN_ENCRYPTION_KEY=
MARKETPLACE_OAUTH_STATE_SECRET=
EBAY_NOTIFICATION_ENDPOINT_URL=
EBAY_NOTIFICATION_VERIFICATION_TOKEN=
EBAY_NOTIFICATION_ENVIRONMENT=production
```

Email:

```
RESEND_API_KEY=
TRANSACTION_EVIDENCE_EMAIL=
TRANSACTION_EVIDENCE_FROM=
TECHNICAL_SUPPORT_EMAIL=
```

`TRANSACTION_EVIDENCE_EMAIL` is the fallback destination address for transaction evidence PDFs when `store_settings.evidence_email` is not set. `TRANSACTION_EVIDENCE_FROM` is optional and defaults to the store evidence sender. `TECHNICAL_SUPPORT_EMAIL` is the fallback support inbox when `store_settings.support_email` is not set.

AI descriptions:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
OPENAI_MODEL=
INSTACOMP_OPENAI_MODEL=
INSTACOMP_OPENAI_FALLBACK_MODEL=
```

InstaComp OCR:

```
PADDLEOCR_API_URL=http://127.0.0.1:8008/ocr
PADDLEOCR_API_KEY=
PADDLEOCR_TIMEOUT_MS=120000
PADDLEOCR_DEVICE=cpu
PADDLEOCR_CPU_THREADS=8
PADDLEOCR_ENABLE_MKLDNN=false
PADDLEOCR_MAX_CONCURRENCY=1
PADDLEOCR_MAX_PREDICTION_IMAGES=10
PADDLEOCR_MAX_DECODED_PIXELS=40000000

GOOGLE_VISION_API_KEY=
GOOGLE_CLOUD_VISION_API_KEY=
```

Only one Google Vision key name is required. PaddleOCR is the primary local provider when its URL is configured. The web route accepts a Paddle timeout from `1000` through `180000` milliseconds; `120000` is the current local reliability setting for CPU inference.

Optional comps:

```
GOOGLE_SEARCH_API_KEY=
GOOGLE_SEARCH_ENGINE_ID=
PRICECHARTING_API_TOKEN=
SERPAPI_API_KEY=
```

COMC Apify active-listing ingestion is retired from InstaComp comp pricing. COMC remains available as a checklist/reference/search source through ordinary research links and catalog-reference prompts.

Shipping provider readiness:

```
TCOS_SHIPPING_PURCHASE_MODE=dry_run
TCOS_SHIPPING_PROVIDERS_REQUIRED=false
TCOS_STANDARD_ENVELOPE_PROVIDER=
TCOS_STANDARD_ENVELOPE_API_KEY=
IMB_PROVIDER_API_KEY=
TCOS_PARCEL_LABEL_PROVIDER=
EASYPOST_API_KEY=
SHIPPO_API_TOKEN=
TCOS_SHIPPING_COVERAGE_PROVIDER=
TCOS_SHIPPING_COVERAGE_API_KEY=
COVERAGE_API_KEY=
```

Keep `TCOS_SHIPPING_PURCHASE_MODE=dry_run`. The current code deliberately blocks live provider purchase because no live adapter has been approved.

The shipping adapter contract records provider state for each planned label. A planned label captures the method-specific adapter key, provider, service, carrier, purchase mode, missing provider credential groups, missing Coverage credential groups, live-support status, and manual-purchase fallback requirement. This is an audit snapshot only; it does not contact USPS, Coverage, EasyPost, Shippo, or any IMb provider.

Scheduled operations:

```
CRON_SECRET=
```

Use the same protected secret for configured Vercel cron calls to Stripe reconciliation and seller eBay reconciliation. Never put the cron secret in a public URL.

IP intelligence and masked-identity blocking:

```
IP_INTELLIGENCE_API_URL=
IP_INTELLIGENCE_API_KEY=
IP_INTELLIGENCE_REQUIRED=true
```

`IP_INTELLIGENCE_API_URL` may contain `{ip}` as a placeholder. If `IP_INTELLIGENCE_REQUIRED=true`, TCOS blocks site/API usage when the provider is missing, unavailable, or reports VPN/proxy/Tor/hosting/anonymous risk.

Reference links:

- Vercel environment variables
- Supabase project settings
- Stripe docs
- eBay developer keys
- Google Programmable Search JSON API
- OpenAI API docs

26. Database Tables

Legacy compatibility:

- `products`
- `orders`
- `order_items`
- `offers`
- `ebay_tokens`

TCOS V2:

- `stores`
- `store_settings`
- `account_profiles`
- `account_store_memberships`

- `inventory_items`
- `inventory_images`
- `inventory_attributes`
- `ebay_sync_decision_events`
- `sales_comp_snapshots`
- `tos_acceptance_events`
- `transaction_evidence_reports`
- `security_ip_investigations`
- `seller_marketplace_connections`
- `seller_marketplace_connection_tokens`
- `seller_marketplace_import_jobs`
- `seller_marketplace_staged_items`
- `seller_marketplace_webhook_events`
- `seller_marketplace_orders`
- `seller_marketplace_order_items`
- `seller_marketplace_order_events`
- `seller_marketplace_reconciliation_runs`
- `seller_marketplace_reconciliation_events`
- `seller_payout_accounts`
- `seller_payout_ledger_entries`
- `platform_fee_ledger_entries`
- `seller_payout_requests`
- `seller_payout_request_entries`
- `seller_payout_admin_events`
- `order_review_cases`
- `order_review_case_events`
- `order_review_case_packets`
- `instacomp_scans`
- `instacomp_search_cache`
- `checkout_attempts`
- `stripe_webhook_events`
- `stripe_post_payment_objects`
- `financial_adjustment_ledger_entries`
- `stripe_reconciliation_runs`
- `stripe_reconciliation_items`
- `payment_simulation_runs`
- `payment_simulation_scenarios`
- `live_payment_launch_gates`
- `live_payment_launch_events`

- `live_shipping_launch_gates`
- `live_shipping_launch_events`
- `order_shipping_labels`
- `order_shipping_tracking_events`
- `order_shipping_coverage_claims`

See:

```
docs/DATABASE_DOCUMENTATION.md
```

27. Migrations

Migration directory:

```
supabase/migrations
```

Apply every migration in timestamp order. Do not rely on a hand-selected subset. The current production-critical migration tail includes:

```
20260709000000_add_instacomp_search_cache.sql
20260709010000_restore_service_role_database_access.sql
20260710000000_create_secure_seller_marketplace_workflow.sql
20260710030000_create_seller_marketplace_webhook_events.sql
20260710070000_create_seller_ebay_reconciliation.sql
20260710090000_create_seller_ebay_outside_orders.sql
20260710103000_create_stripe_webhook_events.sql
20260710113000_create_checkout_attempts.sql
20260710130000_create_stripe_financial_adjustments.sql
20260710143000_create_stripe_reconciliation.sql
20260710160000_create_dispute_evidence_workflow.sql
20260710170000_create_payment_simulation_runs.sql
20260710180000_create_checkout_e2e_isolation.sql
20260710181000_grant_checkout_audit_access.sql
20260710182000_fix_checkout_e2e_cleanup_uuid.sql
20260710183000_restore_orders_account_link.sql
20260710184000_restore_order_seller_routing.sql
20260710185000_create_live_payment_launch_gate.sql
20260710190000_create_shipping_label_infrastructure.sql
20260711010000_create_instacomp_scan_job_queue.sql
20260711185500_create_live_shipping_launch_gate.sql
20260712174000_add_seller_protection_financial_adjustments.sql
20260716010000_add_instacomp_trade_handoff.sql
```

The authoritative list is the complete `supabase/migrations` directory, including all earlier account, inventory, evidence, security, seller, and payout migrations. Apply migrations before using features that depend on new tables. A missing migration can appear as an unavailable page, `503`, failed draft creation, missing reconciliation data, or an unsafe launch-readiness blocker.

The durable InstaComp queue is unavailable until this migration has been applied to the target Supabase project:

```
supabase/migrations/20260711010000_create_instacomp_scan_job_queue.sql
```

It creates the job/item tables, private `instacomp-job-images` bucket, row-level-security policies, service-role grants, and claim/finish/fail/retry database functions. Applying the file locally does not update the hosted project. Use the Supabase CLI with authenticated database access or paste the complete migration into the target project's SQL Editor, then verify that it completed without errors.

The InstaComp Available-for-Trade button is unavailable until this migration has been applied:

```
supabase/migrations/20260716010000_add_instacomp_trade_handoff.sql
```

It adds `trade_collection_item_id`, `trade_available_at`, an index, and a sell-or-trade check constraint to `instacomp_scan_items` so one scan row cannot create both a sell draft and a trade collection item.

Reference:

- Supabase database migrations

28. Build And Verification

Run:

```
npm run lint
npm run build
npm run verify:instacomp
npm run simulate:instacomp-jobs
npm run simulate:instacomp-catalog-identity
npm run manual:pdf
```

Expected:

- lint succeeds with zero errors and zero warnings
- compile succeeds
- TypeScript succeeds
- route generation succeeds
- the full InstaComp verifier succeeds, including queue state, accuracy, catalog identity, printed-variant identity guard, scan-review, and 100-card trial scorekeeper fixture checks
- all InstaComp queue state simulations succeed

- all InstaComp catalog identity simulations succeed, including exact catalog confirmation, unapproved-source review, ambiguity review, serial-run mismatch review, and missing-candidate review
- `docs/TCOS_OPERATOR_MANUAL_PRINT.html` is regenerated
- `docs/TCOS_OPERATOR_MANUAL.pdf` is regenerated with the ownership watermark
- `TCOS_MANUAL_BROWSER_PATH` can point the manual PDF generator at a custom local browser executable
- `TCOS_MANUAL_PDF_BROWSER_TIMEOUT_MS` can shorten headless-browser shutdown waits while preserving a freshly written PDF

For PaddleOCR service changes, also run:

```
cd C:\Projects\truely-collectables\services\paddleocr-service
.\.venv\Scripts\python.exe -m py_compile app.py
Invoke-RestMethod http://127.0.0.1:8008/health
```

For an InstaComp queue change, also run the direct TypeScript check and verify formatting:

```
npx tsc --noEmit
git diff --check
```

The queue simulation verifies state transitions, leases, idempotency, retries, and completion calculations in the local model. It is not a substitute for applying the migration and running one authenticated upload/scan/recovery test against the target Supabase project.

Use these checks before deploy or after feature changes. Run the relevant payment and shipping simulations after changing money, webhook, reconciliation, seller payout, shipping-policy, provider-adapter, or claim code. `npm run verify:production` includes the non-blocking live-money status report, LetterTrack evidence simulation, provider purchase-attempt audit simulation, and the full twenty-scenario shipping simulation suite, so delivered, not-delivered, exception, returned, override, ignored-provider payout-gate, saved claim evidence-review audit, blocked live-gate purchase-audit text, provider-setup blocker audit text, packet audit lines, Standard Envelope, Ground Advantage, seller-protection reserve, under-$20 cap/allocation/refund-gate math, seller order protection visibility, LetterTrack seller-protection CSV contract, provider-setup evidence contract, provider-adapter, and dry-run guardrail cases are checked before deployment. The production guardrail check also protects the named smoke contracts for launch readiness, Launch Gate Drill, production smoke, live payment/shipping gates, admin shipping controls, shipping simulation API, shipping provider exports, shipping exceptions export, LetterTrack CSV export, the seller marketplace packet intake route, and the seller inventory/order/payout auth gates. It also guards the named `queued-feature smoke manifest` so unknown or duplicate deploy-lag check names fail before launch smoke runs, protects `TCOS_PRODUCTION_PREFLIGHT_ONLY=true` as the no-deploy environment-flag equivalent to `--preflight-only`, protects smoke/deploy/guardrail diagnostic redaction self-tests, and protects the live deploy safety contract for Vercel quota messaging, unwanted `truely-collectables-tt3b.vercel.app` alias removal, clean production aliasing, success-only quota marker clearing, deployed URL output, clean URL output, and the `npm run smoke:production` handoff. The protected live deploy sequence removes the unwanted `truely-collectables-tt3b.vercel.app` alias, sets the clean production alias, clears the local quota marker only after that alias succeeds, prints `DEPLOYED_PRODUCTION=`, prints `CLEAN_PRODUCTION=https://`, then prints the smoke handoff command.

Reference:

- Next.js docs

29. Safe Operating Rules

Do:

- use admin status buttons instead of deleting products
- use eBay sync to restore stock from eBay
- check comps before repricing important cards
- apply suggested price only after reviewing comps
- update tracking before marking shipped
- verify every InstaComp draft against front/back images before activation
- use only real provider references when recording postage, Coverage, claims, or seller payouts
- run reconciliation before live-payment approval
- verify the Transcend backup after material changes

Do not:

- delete eBay tokens casually
- delete inventory rows manually
- assume clearing orders restores quantity
- assume AI descriptions know card facts not entered in TCOS
- scrape pricing sites without permission/API
- claim 100% scan confidence when evidence is incomplete
- mail with a dry-run label or tracking reference
- treat `Mark Paid` as money movement
- restore stock automatically from an outside eBay cancellation or refund
- enable live payments or live shipping by changing only one environment variable
- expose service-role, Stripe, PaddleOCR, provider, cron, or marketplace encryption secrets

30. Troubleshooting

Admin redirects to login

Cookie is missing or expired. Log in again.

Checkout says not enough inventory

Check product status and quantity on `/admin/products/[id]`.

Product does not show in shop

Check:

- status is `active`
- quantity is above zero
- price is above zero

eBay sync fails

Check:

- `EBAY_CLIENT_ID`
- `EBAY_CLIENT_SECRET`
- `ebay_tokens` has a refresh token
- eBay API scopes are valid

Sales comps do not save

Apply:

```
supabase/migrations/20260627160000_create_sales_comp_snapshots.sql
```

Google comps show not configured

Set:

```
GOOGLE_SEARCH_API_KEY=
GOOGLE_SEARCH_ENGINE_ID=
```

PriceCharting shows not configured

Set:

```
PRICECHARTING_API_TOKEN=
```

AI description falls back to local

Check:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
```

PaddleOCR health works but scans fail

The health endpoint is not an inference test. Check:

- `%TEMP%\tcos-paddleocr.stderr.log`
- `PADDLEOCR_API_URL`
- matching `PADDLEOCR_API_KEY` values in TCOS and the worker

- restored `.paddlex-cache`
- `PADDLEOCR_ENABLE_MKLDNN=false` on Windows
- original image/request size

Restart both TCOS and PaddleOCR after environment changes.

InstaComp says request body exceeded 10 MB

This normally indicates the older multipart fallback path, not the durable queue path. Confirm the queue migration is installed. The browser fallback optimizes each full image to approximately `900 KB`, each crop to approximately `180 KB`, and targets a request below `3.75 MB`; still reshoot or resize an unusually large source and retry only the affected row.

InstaComp reports `INSTACOMP_JOB_MIGRATION_REQUIRED`

The web code can reach Supabase, but the durable queue schema is not installed in that project. Apply the entire file below to the same Supabase project used by the running app:

```
supabase/migrations/20260711010000_create_instacomp_scan_job_queue.sql
```

The migration also creates the private image bucket and queue functions. Reload InstaComp after the SQL succeeds. Do not work around this error by making the image bucket public.

InstaComp reports `INSTACOMP_JOB_STORAGE_REQUIRED`

Confirm the migration completed through its Storage statements and that the private bucket named `instacomp-job-images` exists. Confirm `SUPABASE_SERVICE_ROLE_KEY` belongs to the same project as `NEXT_PUBLIC_SUPABASE_URL`, then restart the web app. Never place the service-role key in a browser-exposed `NEXT_PUBLIC_` variable.

A saved InstaComp lot does not continue after closing the browser

This is expected in the current architecture. Supabase preserves the job, uploaded originals, per-row state, results, and retry information, but the open browser tab currently claims and processes work. Reopen `/admin/products/new` or `/admin/instacomp` in the same ownership context and use the recovered job. A detached server worker has not been deployed yet.

InstaComp recovery shows an incomplete upload

Keep the original files until registration and upload finish. InstaComp can confirm objects that finished uploading before a page interruption and can resume registered rows. If the interruption occurred before every row was registered, clear/cancel that partial job and reselect the original lot; do not assume unregistered local files were copied to Supabase.

An InstaComp job stays `cancelling`

An already-processing row may hold a worker lease. Keep the page open briefly and retry `Clear Batch` after the lease is released or expires. Do not delete queue rows or private Storage objects manually; the queue cancellation functions preserve consistent job counts.

InstaComp scans but cannot create drafts

Check:

- the current job is owned by either the admin/store session or the intended active seller session
- the seller session can refresh; if it expired and refresh failed, log in again at `/account/login`
- title is not blank
- price is positive
- quantity is at least one
- inventory migrations are applied
- the persistent row is `completed` or `review_required`

Admin-owned queue jobs create store-owned drafts with no seller account ID. Seller-owned jobs create drafts scoped to that seller. Do not switch ownership context midway through recovery or draft creation.

Serial number is visible but missing

Confirm pairing, inspect OCR diagnostics, reshoot without glare, retry the row, and check final AI serial data/exports. Leave the field blank when the complete fraction cannot be proven.

Live payment checkout remains blocked

Check both locks:

- current database approval at `/admin/live-payment-launch`
- `TCOS_LIVE_PAYMENTS_ENABLED=true` in the running deployment

Then inspect live keys, webhook secret/events, production origin, simulations, reconciliation alerts, test residue, commission percentage, subscription flag, and connected-seller readiness.

Stripe reconciliation has unmatched money

Open `/admin/financial-reconciliation`, compare source IDs and amounts, correct the underlying record, and resolve or ignore only with a specific note. Never hide an alert merely to pass the live gate.

Seller payout cannot be released

Confirm:

- Stripe Connect onboarding is active
- payouts are enabled
- requirements and disabled reason are clear
- order is shipped

- no payment/inventory/shipping review remains
- no active case or appeal hold remains
- a real provider reference exists before marking paid

eBay seller connection is stale or revoked

Open `/seller/marketplaces`, refresh status, reconnect when scopes are missing, and inspect revocation status. Do not reuse Store #1 global tokens for a seller connection.

Shipping purchase is blocked

This is expected when live mode is selected because no live adapter is approved. Return to:

```
TCOS_SHIPPING_PURCHASE_MODE=dry_run
```

For real fulfillment, purchase externally and use `Record Manual Purchase` with real, non-dry-run references.

Dry-run label cannot be marked shipped

This is intentional. Create or record a real label, tracking/IMb, and Coverage policy before saving tracking or marking shipped.

Disaster backup verification fails

Do not restore from an unverified copy. Reconnect the Transcend drive, retry `VERIFY_BACKUP.ps1`, and require zero missing/mismatched files plus the full archive SHA-256 pass. If only the folder is damaged but the archive hash passes, restore from the archive.

31. Developer Map

Inventory:

```
src/modules/inventory
```

eBay, comps, pricing:

```
src/lib/ebay.ts
```

Checkout:

```
src/app/api/checkout/route.ts
```

Terms of Service:

```
src/lib/legal.ts
src/app/terms/page.tsx
```

```
supabase/migrations/20260627170000_add_tos_acceptance_to_orders_offers.sql
```

Stripe webhooks:

```
src/app/api/webhook/route.ts
src/app/api/stripe/webhook/route.ts
```

Transaction evidence:

```
src/lib/evidence-pdf.ts
src/lib/transaction-evidence.ts
src/lib/order-review-case-packet.ts
src/app/admin/files/page.tsx
src/app/api/admin/files/[id]/download/route.ts
src/app/api/admin/order-review-cases/[id]/packet/route.ts
supabase/migrations/20260627180000_create_transaction_evidence_reports.sql
supabase/migrations/20260701220000_create_order_review_case_packets.sql
```

Admin product screens:

```
src/app/admin/products/page.tsx
src/app/admin/products/new/page.tsx
src/app/admin/products/[id]/page.tsx
```

Accounts:

```
src/app/admin/accounts/page.tsx
src/app/account
src/app/api/account
src/lib/account-auth.ts
src/lib/account-profiles.ts
supabase/migrations/20260628213000_create_sports_dashboard_tables.sql
```

Orders:

```
src/app/admin/orders
src/app/api/orders
```

Offers:

```
src/app/admin/offers
src/app/api/offers
```

InstaComp and OCR:

```
src/app/admin/instacomp
src/app/admin/products/new/page.tsx
src/app/api/instacomp
src/lib/instacomp.ts
services/paddleocr-service
docs/INSTACOMP_PADDLEOCR_SERVICE.md
```

Payment reliability and live launch:

```
src/lib/stripe-credentials.ts
src/lib/stripe-webhook-events.ts
src/lib/stripe-post-payment.ts
src/lib/stripe-reconciliation.ts
src/lib/stripe-dispute-evidence.ts
src/lib/payment-simulations.ts
src/lib/checkout-e2e-simulation.ts
src/lib/live-payment-launch.ts
src/app/admin/payment-simulations
src/app/admin/financial-reconciliation
src/app/admin/live-payment-launch
src/app/api/admin/payment-simulations
src/app/api/admin/financial-reconciliation
src/app/api/admin/live-payment-launch
```

Seller payouts:

```
src/lib/seller-payouts.ts
src/lib/seller-payout-ledger.ts
src/lib/seller-payout-review-blocks.ts
src/app/seller/payouts
src/app/admin/seller-payouts
src/app/api/admin/seller-payouts
```

Shipping and Coverage:

```
src/lib/shipping.ts
src/lib/shipping-policy.ts
src/lib/shipping-provider-readiness.ts
src/lib/shipping-provider-adapter.ts
src/lib/shipping-simulations.ts
src/app/admin/shipping
src/app/api/admin/orders/[id]/shipping-labels
src/app/api/admin/orders/[id]/shipping-claims
```

```
src/app/api/admin/shipping-labels
src/app/api/admin/shipping-claims
```

Seller eBay workflow:

```
src/lib/seller-ebay.ts
src/lib/seller-ebay-orders.ts
src/lib/seller-ebay-reconciliation.ts
src/lib/seller-marketplace-connections.ts
src/lib/marketplace-token-crypto.ts
src/app/seller/marketplaces
src/app/api/account/seller/marketplace-connections/ebay
src/app/api/cron/seller-ebay-reconciliation
```

Disaster-recovery snapshot tools:

```
C:\Projects\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-
064539\RESTORE_GUIDE.md
C:\Projects\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-064539\scripts
```

32. InstaComp Production Operation

InstaComp is the current TCOS sports-card image intake, identification, comp-assist, and draft-listing system.

Primary routes:

```
/admin/products/new
/admin/instacomp
/api/instacomp/scan
/api/instacomp/draft-listings
/api/instacomp/jobs
/api/instacomp/jobs/[id]
/api/instacomp/jobs/[id]/claim
/api/instacomp/jobs/[id]/items
/api/instacomp/jobs/[id]/items/[itemId]
/api/instacomp/jobs/[id]/items/[itemId]/complete
/api/instacomp/jobs/[id]/items/[itemId]/fail
/api/instacomp/jobs/[id]/items/[itemId]/retry
/api/instacomp/trade-items
```

Use `/admin/products/new` for normal lot intake. Use `/admin/instacomp` when a dedicated scan-lab view or recent-scan history is easier.

Start InstaComp locally

The local scanner requires two running services:

- TCOS/Next.js on port `3000`
- PaddleOCR on port `8008`

The easiest start procedure uses the newest disaster-recovery snapshot:

```
Set-ExecutionPolicy -Scope Process Bypass
& "C:\Projects\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-064539\scripts\START_TCOS.ps1"
```

The script starts both services, waits up to 45 seconds, and uses the restored local model cache. Verify both services:

```
Invoke-RestMethod http://127.0.0.1:8008/health
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:3000/admin/login
```

Expected PaddleOCR health result:

```
{"ok":"true","provider":"paddleocr"}
```

Important: the health endpoint does not load or run the OCR model. A successful health check proves that the HTTP worker is listening, but the first real scan is the inference test.

Failure logs:

```
%TEMP%\tcos-paddleocr.stderr.log
%TEMP%\tcos-paddleocr.stdout.log
%TEMP%\tcos-next.stderr.log
%TEMP%\tcos-next.stdout.log
```

Required browser sessions and ownership

The admin pages require the admin login at `http://localhost:3000/admin/login`. The queue API then uses one of two ownership contexts:

- if the browser has a valid active seller session and membership, that seller owns the queue job and drafts created from it;
- otherwise, a valid admin cookie owns the queue job on behalf of the active store, and drafts are store-owned with `seller_account_id` left null.

Log in to `http://localhost:3000/account/login` before uploading when the lot must belong to a particular seller. If seller login happened after InstaComp was already open, refresh the page before creating the job. Do not change seller accounts midway through a saved job.

For long-running seller jobs, the browser refreshes the stored Supabase session when it is within five minutes of expiration before authenticated queue, scan, and draft calls. If refresh fails after the token is expired, the

stored session is cleared and the operator must log in again. The admin cookie and seller token remain separate credentials.

Prepare images

Best filename pattern:

```
001-front.jpg
001-back.jpg
002-front.jpg
002-back.jpg
```

Recognized front tokens:

```
front
fr
f
obverse
```

Recognized back tokens:

```
back
bk
b
reverse
rear
```

The side word must be the final filename token before the extension and separated by a space, period, underscore, or dash.

Pairing rules:

- explicit front/back filenames are paired by normalized base name
- multiple fronts/backs with the same base name pair in upload order
- files without a side token pair strictly by upload order: `1+2`, `3+4`, and so on
- an odd final unknown image becomes front-only
- explicitly named extra back-only images are skipped
- explicit-name groups and unknown-name groups are paired independently
- exact duplicate front/back pairs are rejected using filename, byte size, and last-modified signatures

The limit is 500 card rows, not 500 image files. A 500-card front/back lot can contain up to 1,000 original images.

Front-only cards scan, but the back frequently contains the strongest year, set, card-number, manufacturer, copyright, and authenticity evidence. Use both sides whenever possible.

100-card trial run and 94% accuracy scorecard

Use this trial when TCOS needs a real-world InstaComp accuracy check before trusting a new scan batch. The target trial is about `100` cards with front and back images, or about `200` scans.

The trial harness is local and read-only. It does not publish listings, buy postage, create Checkout sessions, deploy, change live-money flags, or call production APIs. It only scores a completed InstaComp run against a ground-truth manifest.

Fast path: run the one-command local intake cockpit first:

```
npm run instacomp:trial:intake
```

That command prints schema `tcos.instacompTrialIntakeCockpit.v1`, ensures the local `instacomp-trial-inbox/` and `instacomp-trial-images/` folders exist, writes `README_TCOS_INSTACOMP_TRIAL.txt` guide files inside both ignored folders, dry-runs image staging, refreshes the prep/preflight receipts, syncs manifest/worksheet image paths from the current image map, and writes `instacomp-trial-intake.local.json`, `instacomp-trial-intake.local.md`, plus ignored local answer-key helper `instacomp-trial-groundtruth-guide.local.md` with one next action. The answer-key guide shows current row progress, first missing rows, examples, commands, and the required answer-key fields: `player`, `year`, `setName`, and `cardNumber`. It does not apply staging, delete source files, scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

1. Put the trial images in a local folder such as `instacomp-trial-images/`.
2. Name each card pair with a stable number, for example `001-front.jpg`, `001-back.jpg`, through `100-front.jpg`, `100-back.jpg`. If the scanner exports plain ordered files such as `scan_0001.jpg`, `scan_0002.jpg`, and so on, keep the folder sorted in intended upload order; the audit treats `1+2`, `3+4`, and onward as front/back pairs when no explicit front/back token is present.
3. If the scanner exports raw files that need to be normalized, drop them in ignored local folder `instacomp-trial-inbox/`, then dry-run the local image staging helper:

```
npm run instacomp:trial:stage-images
```

That command prints schema `tcos.instacompTrialImageStage.v1`, writes `instacomp-trial-stage.local.json` plus `instacomp-trial-stage.local.md`, and shows whether the raw scanner files can be copied into `instacomp-trial-images/` as normalized `001-front`, `001-back`, `002-front`, `002-back` files. It accepts explicit front/back filenames or plain ordered scanner exports. The default is a dry run; when the receipt is clean, apply it:

```
npm run instacomp:trial:stage-images -- --apply
```

The staging helper copies files only; it does not delete originals. Existing target files block apply unless you explicitly add `--overwrite`. It is local/read-only for live systems and does not scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

1. Run the one-command local prep bundle:

```
npm run instacomp:trial:prep
```

That command prints schema `tcos.instacompTrialPrepBundle.v1`, creates `instacomp-trial-manifest.local.json` if it is missing, writes `instacomp-trial-groundtruth.local.tsv` only when the worksheet does not already exist, refreshes `instacomp-trial-image-map.local.json`, refreshes `instacomp-trial-intake-packet.local.md`, and writes `instacomp-trial-preflight.local.json` plus `instacomp-trial-preflight.local.md`. The preflight proof now includes a hard scan permit (`SCAN_PERMIT_GRANTED` or `SCAN_PERMIT_BLOCKED`), a scan warning, and ordered operator next actions so the final tester lot cannot accidentally burn scanner time before the answer key, image pairs, image-map receipt, and intake packet are current. It is safe to rerun while the answer sheet is being filled because it preserves an existing worksheet by default. It is local/read-only for live systems and does not scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

If staged files use non- `.jpg` extensions or the worksheet/manifest needs to match the latest image-map receipt, sync only the front/back image paths:

```
npm run instacomp:trial:sync-images
```

That command prints schema `tcos.instacompTrialImagePathSync.v1`, reads `instacomp-trial-image-map.local.json`, updates only `frontImage` and `backImage` in `instacomp-trial-manifest.local.json`, preserves answer-key identity fields, and updates only the image columns in `instacomp-trial-groundtruth.local.tsv` when that worksheet exists. It writes `instacomp-trial-image-path-sync.local.json` plus `instacomp-trial-image-path-sync.local.md`. It is local/read-only for live systems and does not scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

While loading images or filling the answer key, run the local readiness monitor:

```
npm run instacomp:trial:monitor
```

That command prints schema `tcos.instacompTrialReadinessWatch.v1` and shows answer-key row progress, image-file count, complete front/back pairs, stale image-map/intake receipts, the raw scanner inbox, the normalized image drop zone, the visual answer-key HTML review sheet, answer-key validation status, readiness gates, ordered operator next actions, the first missing rows/files, and the exact next command. When the local HTML sheet is current, the monitor points the operator to use `instacomp-trial-answer-key.local.html` beside `instacomp-trial-groundtruth.local.tsv`, then run `npm run instacomp:trial:answer-key:validate` before applying ground truth. The broader `npm run status:instacomp-final-tester` checkpoint also reports whether `instacomp-trial-groundtruth.local.tsv` exists, how many worksheet rows have the required core fields filled, whether the validation receipt matches the current worksheet, whether `instacomp-trial-groundtruth-guide.local.md` exists, matches the current ground-truth audit, needs a refresh from `npm run instacomp:trial:intake`, and how many accepted scanner image files are waiting in raw drop zone `instacomp-trial-inbox/` before staging into normalized `instacomp-trial-images/`. Use `npm run instacomp:trial:monitor:json` for automation, or `node scripts/watch-instacomp-trial-readiness.mjs --watch --interval-ms 5000` if you want it to keep refreshing while the scanner files are copying. It is local/read-only for live systems and does not scan cards,

deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

If you need only the manifest, create the local manifest:

```
npm run instacomp:trial:init
```

1. Open `instacomp-trial-manifest.local.json` and fill the `expected` fields from the physical card before using the scan result. Required ground-truth fields for scoring are player/subject, year, set, and card number. Important recommended fields are brand/manufacturer, parallel, variation, team, sport, autograph/relic/rookie flags, exact serial number such as `07/50`, and serial run such as `/50`.

If editing JSON for 100 cards is too slow, write the local spreadsheet-style answer sheet:

```
npm run instacomp:trial:groundtruth:sheet
```

That command writes ignored local file `instacomp-trial-groundtruth.local.tsv` with columns for `trialCardId`, image paths, player, year, setName, cardNumber, brand, parallel, variation, serial number, team, sport, rookie/autograph/relic flags, and notes. Open it in Numbers, Excel, or Google Sheets, fill the answer key from the physical cards, save it as TSV, then apply it back to the manifest:

For faster visual filling, write the local answer-key HTML review sheet:

```
npm run instacomp:trial:answer-key-html
```

That command writes ignored local file `instacomp-trial-answer-key.local.html` with front/back thumbnails, editable current TSV identity fields, missing-field status, and the exact apply/recheck commands. Use `Copy updated TSV` or `Download updated TSV` after editing, save the result over `instacomp-trial-groundtruth.local.tsv`, then run `npm run instacomp:trial:groundtruth:apply`. It is a local guide only: it does not scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, mutate images, write the TSV automatically, or apply worksheet values.

Before applying a copied or downloaded TSV, validate the answer key:

```
npm run instacomp:trial:answer-key:validate
```

That command prints schema `tcos.instacompTrialAnswerKeyValidation.v1`, checks the local worksheet against the manifest for required columns, duplicate/missing IDs, row-count drift, missing `player` / `year` / `setName` / `cardNumber` fields, image-path drift, row-order drift, and odd rookie/autograph/relic boolean values. It writes ignored local receipts `instacomp-trial-answer-key-validation.local.json` and `instacomp-trial-answer-key-validation.local.md`. It does not apply worksheet values, mutate images, scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

```
npm run instacomp:trial:groundtruth:apply
```

The apply command only updates the local manifest from matching `trialCardId` rows. It does not scan, deploy, publish listings, buy postage, create Checkout, or touch production APIs.

1. Before scanning, audit the ground-truth manifest:

```
npm run instacomp:trial:groundtruth
```

That command is local and read-only. It prints schema `tcos.instacompTrialManifestAudit.v1` and fails until every `trial-card-###` row has player/year/set/card number filled, no `trialCardId` rows are blank, and no duplicate `trialCardId` values exist. This prevents a fake `94%` pass when the answer key is still empty.

1. Before scanning, audit the folder so missing fronts/backs, duplicate images, unknown filenames, and extra files are caught before the scanner wastes time:

```
npm run instacomp:trial:audit
```

The audit is read-only and writes nothing. It prints schema `tcos.instacompTrialImageAudit.v1`, confirms the expected 100 cards / 200 images, reports ordered-pair candidate files and complete pairs, and fails until every card has exactly one front and one back image.

To save a local front/back receipt before scanning, run:

```
npm run instacomp:trial:map
```

That command writes ignored local file `instacomp-trial-image-map.local.json` with schema `tcos.instacompTrialImageMap.v1`. Use it to confirm which scanner file became the front and back for each `trial-card-###` row before uploading the lot.

To save a readable operator intake packet before scanning, run:

```
npm run instacomp:trial:packet
```

That command writes ignored local files `instacomp-trial-image-map.local.json` and `instacomp-trial-intake-packet.local.md`. The packet is a Markdown receipt with schema-backed audit counts, the local image folder, ready/not-ready status, the first pairing-preview rows, problem counts, accepted filename patterns, and the next commands. It uses `--allow-not-ready`, so it can document missing files before the folder is complete without pretending the lot is ready to scan.

To run the one-shot final tester preflight, run:

```
npm run instacomp:trial:preflight
```

That command prints schema `tcos.instacompTrialPreflight.v1` and fails until the ground-truth answer key is ready, the image folder has 100 complete front/back pairs, the image-map receipt matches the current audit, and the readable intake packet is current. It is the fastest operator gate before spending scanner time. It is local/read-only and does not scan cards, deploy, publish listings, buy postage, create Checkout, call production APIs, approve live money, release payouts, or change runtime switches.

For the normal pre-scan gate, run the one-shot readiness command:

```
npm run instacomp:trial:ready
```

That command runs the ground-truth manifest audit, runs the image audit, writes the front/back image-map receipt, and prints `status:instacomp-final-tester`. It must pass before the 100-card lot should be scanned.

1. Run the lot through `/admin/instacomp` or `/admin/products/new` using the safest durable batch workflow below.
2. From `/admin/instacomp`, check the `Final Tester Gate` HUD before export. It shows visible result count, timing coverage, average speed, p95 speed, slowest rows, and the `FAF PASS` / `NOT READY` status against the final tester targets.
3. Use `Export Trial Results` or `Copy Trial Results` after the batch finishes, then save the exported JSON as `instacomp-trial-results.local.json`. The export uses schema `tcos.instacompTrialResults.v1`, preserves row-stable trialCardId values such as `trial-card-001`, includes the detected `actual` fields, carries consensus/review status, includes per-row timing evidence, and only includes completed visible scan rows. If you manually build the file instead, each row must use the same `trialCardId` as the manifest and can put detected fields under `actual`, `result`, `predicted`, or `ai`; to satisfy the speed gate, include timing under `timing.elapsedMs`, `scanTiming.elapsedMs`, or `scanElapsedMs`.
4. Score the trial with the official accuracy + FAF timing gate:

```
npm run instacomp:trial:score
```

To save the miss list as a durable fix queue, run:

```
npm run instacomp:trial:failures
```

That command writes `instacomp-trial-failures.local.json` with schema `tcos.instacompTrialFailureReport.v1`. The file includes missing result rows, mismatched fields, multi-scanner consensus-review rows, speed misses, suggested actions, and the read-only no-money/no-postage/no-deploy side-effect boundary.

The report prints:

- card count and declared scan count
- front/back pair count
- card-identity exact accuracy
- field-level identity accuracy
- exact serial-number accuracy
- serial-run accuracy
- combined identity-and-serial accuracy
- timing-evidence coverage
- average seconds per completed card
- p95 seconds per completed card

- the slowest trial rows
- any cards still blocked by multi-scanner consensus review
- the exact trial card IDs and fields that missed
- the optional failure-report path, failure-row count, and consensus-review-row count when `--write-failure-report` is used

The trial is considered a pass only when combined identity-and-serial accuracy is at least `94%`, card-identity exact accuracy is at least `94%`, exact serial-number accuracy is at least `94%` when serial-numbered cards are present, every completed row includes timing evidence, average scan speed is `15` seconds per card or faster, p95 scan speed is `45` seconds per card or faster, no manifest row is missing a result, and no card still has `review_required` from InstaComp™ Multi-Scanner Consensus. Critical player, set, card-number, year, and serial-number disagreements must be resolved by checklist/catalog evidence or operator review before the tester can honestly pass.

Use `npm run simulate:instacomp-trial` to verify the scorekeeper itself with committed fixture data. The fixture intentionally stays tiny; the real 100-card manifest and results are local files ignored by Git because they can include personal inventory paths and scan results.

Durable queue prerequisite

Apply this migration to the Supabase project used by TCOS before using the saved queue:

```
supabase/migrations/20260711010000_create_instacomp_scan_job_queue.sql
```

The queue requires `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, and the server-only `SUPABASE_SERVICE_ROLE_KEY`. The migration creates:

- `instacomp_scan_jobs` and `instacomp_scan_items`;
- a private `instacomp-job-images` Storage bucket;
- seller-scoped read policies plus service-role access used by authenticated admin/seller server routes;
- atomic claim, completion, failure, retry, cancellation, and counter-refresh functions.

If the migration is missing, the API intentionally returns `503` with `INSTACOMP_JOB_MIGRATION_REQUIRED`. A service-role key cannot create the database schema by itself through the normal data API.

Safest durable batch workflow

Use this controlled workflow:

1. Open `/admin/products/new`.
2. Drop the front/back images into the batch area.
3. Confirm the displayed pairing before scanning.
4. Leave `Parallel Scans` at the default `4`. The allowed range is `1` through `6`; use `1` or `2` if the laptop becomes unstable or Supabase reports connection pressure.
5. Click `Run Batch InstaComp`.
6. Wait while the browser creates an idempotent job, registers no more than 50 rows at a time, creates bounded high-resolution derivatives, uploads them to private Storage with up to 6 concurrent signed

uploads, confirms each registered chunk, and queues the job.

7. Keep the InstaComp tab open while it claims and scans rows. The browser claims up to 5 queue rows per worker mini-pack to reduce database round-trips. `Pause` stops new work only after current requests finish.
8. Inspect every `completed` and `review_required` row. A completed request is not a promise that the card identity is exact.
9. Correct the title, positive listing price, quantity, identity fields, and price evidence.
10. Filter to `Clean Ready` when possible.
11. Select only cards whose photos and printed facts you personally verified.
12. Create drafts; the browser sends persistent draft requests one card at a time with limited parallelism instead of one oversized lot request.
13. Use `Open InstaComp Drafts` from the success message, or `Open in InstaComp drafts` on an individual row, to open `/seller/inventory?status=draft&source=instacomp` with the relevant search/filter already applied.
14. In Seller Inventory, keep the `Source` filter on `InstaComp`, inspect each draft, and fix any remaining readiness blockers before activation.
15. For a trade-only card, use `Add to Available for Trade` instead of creating a sell draft. The trade handoff creates or reuses a seller-owned collection item marked Available for Trade. Do not use both sell-draft and trade handoff on the same scan row.
16. For cross-listing prep only, select verified ready drafts and use `Copy Marketplace Packet` or `Download Marketplace CSV`. These files do not publish to eBay, Whatnot, or another external storefront.
17. Activate only after the seller inventory readiness check, photos, title, price, shipping, authenticity, and platform-specific requirements are verified.

`Run InstaComp Auto-Pilot` scans unfinished rows and attempts draft creation only for rows that pass both technical draft readiness and the queue review gate. A row marked `review_required` is not automatically drafted. Auto-Pilot never publishes a live listing.

If Auto-Pilot is paused, it finishes current scan requests but does not run its draft-creation phase.

Failed rows can be retried individually, all at once, or through the current visible filter.

Use the thumbnail zoom and rotate controls before retrying a row whose image is sideways or hard to inspect. Each rotate click applies a 45-degree left or right correction from the original local image and updates the preview. Rotation requires the original file to still be present in the browser; recovered remote-only saved-lot images must be reselected locally before rotation can be applied.

If a row or claim request reports database connection pressure, InstaComp pauses before claiming more work, reduces browser concurrency by one, and displays the pressure message. Wait for Supabase to catch up, then resume. Do not repeatedly click resume while the database is still reporting `Too many connections`.

Registration, upload, recovery, and resume

The durable flow separates image transfer from expensive OCR/AI work:

1. The browser creates a job with a stable client batch ID so an identical create request can be replayed safely.

2. It registers rows in chunks of at most 50.
3. Supabase returns short-lived, non-overwriting signed upload targets for the private bucket. Every registered image requires a browser-computed SHA-256 digest.
4. The browser uploads the bounded card-side derivatives with up to 6 concurrent signed uploads and bulk-confirms no more than 50 registered rows per request.
5. The browser marks the complete job `queued`, claims rows with leases in up to 5-card mini-packs, and calls the JSON scan path using job/item IDs instead of resending image bytes.
6. The server downloads the private derivatives, verifies their registered size and SHA-256 digest, performs OCR/AI/comp work, and atomically records either `completed`, `review_required`, a retry, or a terminal failure.

The current browser stores the active job reference locally and also checks the server for the newest recoverable job. Reloading or reopening InstaComp can restore registered rows, signed image previews, saved results, counts, and retry state. If an object uploaded immediately before a crash but its confirmation did not finish, recovery can confirm the existing object. If the browser crashed before later files were registered or uploaded, those local file bytes do not exist on the server; cancel/clear the partial job and reselect the original files.

Important limitation: the browser is still the worker. Closing all InstaComp tabs stops new claims and therefore stops scanning. The job remains durable and resumable in Supabase, but it does not continue autonomously until a detached worker is deployed.

Queue job statuses:

- `uploading`: rows and private image derivatives are still being registered/confirmed;
- `queued`: uploads are complete and rows are ready to claim;
- `processing`: one or more rows have active work or retries;
- `completed`: every row completed without a review/failure/cancellation outcome;
- `completed_with_errors`: all rows are terminal, but one or more need review, failed, or were cancelled;
- `failed`: the job could not reach a normal terminal result;
- `cancelling`: new work is stopped while an active lease is released or expires;
- `cancelled`: cancellation is complete.

Queue item statuses:

- `awaiting_upload`, `queued`, `processing`, `retry_wait`;
- `completed`, `review_required`, `failed`, `cancelled`.

Rows default to at most three attempts. Claims use expiring leases and the database selects available rows with row locks, which prevents two workers from owning the same live attempt. A retryable failure enters `retry_wait`; an expired lease can be reclaimed until attempts are exhausted. Manual retry returns an eligible terminal row to the queue. Cancellation immediately cancels unclaimed work and waits for any still-valid processing lease before finalizing the job.

Queue creation guardrails currently allow no more than three active jobs and 1,500 submitted card rows in a rolling 24-hour period for the same ownership context. Finish or cancel an existing lot instead of repeatedly creating replacements. Each individual job still has the 500-row maximum and configured scan concurrency from 1 through 6.

The claim API caps one browser worker claim at 5 rows. Keep the browser and server cap aligned when changing scan throughput; raising browser concurrency without aligning claim limits only increases polling pressure without increasing completed-card throughput.

What InstaComp reads

Per card, InstaComp can display:

- player or subject
- year
- brand/manufacturer
- set and subset
- card number
- parallel or finish
- serial number such as `087/250`
- serial OCR rejects impossible fractions such as `0/25` or `99/25`; valid listing and search labels use the print run (`/25`) while preserving `1/1`
- the production InstaComp accuracy gate also verifies that invalid serials cannot constrain comp searches, exact results keep the same print run, excluded lots/graded cards stay out of raw-card results, and guidance prices identify serial-run adjustments
- the printed-variant identity guard prevents OCR-visible or AI-vision-note-visible Limited Red, Clear Cut, Outliers, Future Watch, Spectrum FX, acetate/clear-stock, insert, subset, color foil, refractor, prizm, holo, wave, shimmer, ice, laser, scope, pulsar, mojo, and mosaic cues from staying as `Base`; Upper Deck clear-stock cards with centered/ghosted back-logo cues are promoted to `Clear Cut`, exact printed cues are promoted into the parallel/set fields, and uncertain insert cues are marked for review
- exact comps for a named non-base parallel must also contain full or strong partial parallel evidence, preventing a same-card/same-print-run listing in the wrong color from entering exact market value
- exact comps must contain the detected player and card number whenever those identifiers are available, preventing another card from the same set/parallel/print run from clearing the score threshold
- exact comps must also contain the detected year and brand when available, preventing strong but wrong-year or wrong-brand listings from entering exact market value
- when InstaComp identifies an autograph or relic card, exact comps must contain matching autograph or relic title evidence; ordinary base copies remain available only as broader guidance when otherwise relevant
- autograph and relic/patch signals
- team and sport
- condition clue
- confidence
- AI notes
- OCR provider, checked-image count, OCR text excerpt, and OCR serial
- consensus speed lane and scanner-council mode, including whether the row stayed `fast_lane` / `fast_lane_council`, escalated to `escalated_multi_ai` / `full_council`, the risk tier, and the scanner plan used for that row
- comp-provider status, included comps, suggested price, and research links

- market price basis, including active listings, sold comps when available, same-run guidance, and serial-adjusted guidance

The multipart fallback creates targeted serial-stamp, edge, band, contrast, and inverted crops in the browser. Durable jobs normally send only the stored front/back derivatives to the scan route. When PaddleOCR receives no more than two card-side images, its worker creates five grayscale, auto-contrast serial regions per image (top-right, top-left, middle-right, bottom-right, and bottom-left) until the configured prediction-image cap is reached.

Provider order:

1. PaddleOCR when `PADDLEOCR_API_URL` or `INSTACOMP_PADDLEOCR_API_URL` is configured.
2. Google Vision fallback when `GOOGLE_VISION_API_KEY` or `GOOGLE_CLOUD_VISION_API_KEY` is configured.
3. OpenAI vision structured card identification.
4. Dedicated OpenAI serial-number inspection when external OCR did not prove the serial.

Limits per card:

- the persistent queue accepts JPEG, PNG, and WebP inputs and stores a bounded high-resolution derivative up to `3600` pixels on the longest side and `3 MB` per image
- each registered derivative includes a SHA-256 digest; Storage size/type is checked before queueing and the digest is verified before OCR
- the scan route accepts detail crops up to `512 KB` each and at most `20 MB` total source-plus-detail input
- the multipart browser fallback targets about `900 KB` per full image and `180 KB` per crop, with a request target below `3.75 MB`
- PaddleOCR defaults to at most 10 prediction images and can be configured from 2 through 24
- Google Vision checks at most 16 images
- main OpenAI identification sees front/back plus the first eight detail crops
- dedicated OpenAI serial inspection sees front/back plus all submitted detail crops
- Paddle timeout defaults to 120 seconds in code and is clamped between 1 and 180 seconds
- saved queue results are compacted before persistence and may omit oversized diagnostic detail; the displayed identity and recovery state remain the operational record

When front and back disagree, the prompt tells the identifier to prefer printed back evidence for card number, year, set, and manufacturer and to explain the conflict.

Windows PaddleOCR requirements

The working local environment uses Python 3.12, PaddleOCR 3.7, PaddlePaddle 3.3, and PP-OCRv6 models. Keep these Windows CPU settings:

```
PADDLEOCR_DEVICE=cpu
PADDLEOCR_CPU_THREADS=8
PADDLEOCR_ENABLE_MKLDNN=false
PADDLEOCR_MAX_CONCURRENCY=1
PADDLEOCR_MAX_PREDICTION_IMAGES=10
PADDLEOCR_MAX_DECODED_PIXELS=40000000
```

`PADDLEOCR_ENABLE_MKLDNN=false` avoids the released PaddlePaddle Windows oneDNN/PIR inference crash. The recovery start script also sets:

```
PADDLE_PDX_CACHE_HOME=<project>\services\paddleocr-service\.paddlex-cache
PADDLE_PDX_DISABLE_MODEL_SOURCE_CHECK=True
```

The service contract and manual startup commands live in:

```
docs/INSTACOMP_PADDLEOCR_SERVICE.md
services/paddleocr-service/README.md
```

Paddle worker safety and capacity defaults:

- one OCR prediction request runs at a time; `PADDLEOCR_MAX_CONCURRENCY` is bounded from 1 through 4;
- model initialization is locked so concurrent first requests cannot load multiple model instances;
- at most 10 original/generated prediction images are processed by default; the configured value is bounded from 2 through 24;
- each decoded image is limited to 40,000,000 pixels by default; the configured value is bounded from 1,000,000 through 100,000,000 pixels;
- the Next scan route enforces the byte limits before PaddleOCR; the local worker should remain bound to `127.0.0.1` because it does not replace the upstream aggregate-byte controls.

Draft readiness and review warnings

A row is technically draft-ready when it has:

- a nonblank title
- a positive listing price
- quantity of at least `1`

Review warnings include:

- front only
- confidence below 85%
- missing player/subject, year, brand/set, or card number
- uncertain parallel
- weak adjacency-only pairing
- no listing price
- no market price
- no usable comps
- scan failure
- draft failure

Important: technical draft readiness and automatic drafting are separate decisions. Auto-Pilot excludes a queue row marked `review_required`, even if that row has a title, positive price, and quantity. An operator may

review/correct such a row and deliberately create a non-public draft, but every draft still requires manual verification before activation.

Draft creation facts:

- requires an authenticated active seller or admin/store job owner
- creates a seller-owned draft for a seller job and a store-owned draft for an admin job
- accepts at most 500 items server-side
- requires the persistent job row to be `completed` or `review_required`
- reads the persistent AI/comp result from the server job record rather than accepting client-supplied persistent identity evidence
- reserves each persistent queue row in the database before creating inventory, so simultaneous draft clicks cannot create two listings for the same saved row
- copies the saved high-resolution derivatives from private job Storage into inventory media
- rechecks saved image size and SHA-256 immediately before draft promotion, so listing photos cannot silently differ from the analyzed photos
- builds server-side draft titles with the shared InstaComp title helper, so numbered cards use the print run such as `/50`, invalid serial fractions are omitted, and true one-of-one cards keep `1/1`
- enforces a 3 MB per-saved-image limit after high-resolution browser normalization
- sends persistent draft rows individually with browser concurrency limited to two
- checks SKU, dedupe key, client ID, and scan ID before reusing an existing draft
- can report a metadata/back-image warning after the base draft succeeds
- returns `503` when required inventory or InstaComp queue migrations are missing

A new browser re-upload/rescan can produce new client and scan IDs. Do not assume cross-session duplicate prevention is perfect; check Seller Inventory before retrying a large draft operation.

Available for Trade handoff

Seller-owned InstaComp rows can be sent to the trade side instead of the sell side with `Add to Available for Trade`. This route is:

```
/api/instacomp/trade-items
```

Trade handoff facts:

- requires the card owner to be signed in as a seller account;
- requires a completed or reviewable persistent InstaComp row;
- creates or reuses an `account_collection_items` row for the seller account and active store;
- marks the collection item as Available for Trade and stores an InstaComp evidence snapshot in metadata;
- saves the resulting collection item ID back to `instacomp_scan_items.trade_collection_item_id`;
- records `trade_available_at`;
- refuses trade handoff when the scan row already created a sell draft;
- refuses sell-draft duplication when the row is already marked Available for Trade.

The sell/trade exclusivity is enforced by:

```
supabase/migrations/20260716010000_add_instacomp_trade_handoff.sql
```

The database constraint allows `draft_inventory_item_id` or `trade_collection_item_id`, not both. If the route returns a migration-required error, apply that migration before using `Add to Available for Trade`.

Use the `Buy Me on TCOS` and `Trade For Me on TCOS` buttons as search/discovery handoffs. They do not create a listing or trade item by themselves; they help operators and future customers search for the detected card on the TCOS sale and trade surfaces.

Seller Inventory InstaComp lane and marketplace export packets

Seller-created InstaComp drafts appear in Seller Inventory with:

- `Source = InstaComp`
- an InstaComp badge on the item card
- scan ID
- detected serial number when present
- listing price source such as `instacomp_market` or `manual`
- front/back image indicator

Every Seller Inventory row also shows a default single-unit shipping plan derived from the listing price:

- cards at `$20.00` or less default to Standard Envelope
- cards at `$20.01` or higher default to Ground Advantage
- the row shows the estimated postage, estimated ounces, Coverage provider, Coverage requirement, and Coverage type
- Standard Envelope rows also show whether the seller opted into TCOS Under-$20 Seller Protection, the 2% reserve estimate, the protected item cap, and the not-insurance reminder that LetterTrack/USPS IMb is delivery evidence while shipping is excluded from reimbursement
- if the rules force a method change, the reason appears under the shipping plan

This Seller Inventory plan is a listing handoff aid. Final checkout/order shipping can still recalculate for multi-card carts or operator-entered package details.

When one or more rows are selected, the Selection Summary also shows the selected shipping mix: Standard Envelope count, Ground Advantage count, Priority Mail count, Coverage-required count, forced-method count, and estimated total postage for the selected rows. Use this before activation or marketplace packet export to catch obvious shipping-policy surprises.

The Selection Summary also shows a Selected Activation Check with the activatable count, draft-needs-work count, already-active count, archived count, and the top selected readiness blockers. Use it before `Activate Ready` so bulk activation only moves rows that are actually eligible.

After a bulk activation or archive attempt, use `Copy Bulk Report` or `Download Bulk Report` to preserve a compact JSON report with successes, failures, blocker groups, and retry-ready counts. Use `Clear Bulk Report` after the report has been copied/downloaded and the result card is no longer needed. The Bulk Action Follow-Up card groups failed rows by blocker reason. Click a blocker group to keep only that failed subset

selected for cleanup, then fix the row data. When any failed rows become eligible for the same action, use `Retry Corrected` to rerun only those corrected failed rows.

Seller Inventory supports the URL-safe lane:

```
/seller/inventory?status=draft&source=instacomp
```

Use the `Readiness` filter to separate `Ready` drafts from `Needs work` drafts. The Seller Command Center shows `InstaComp Ready` and routes to ready InstaComp drafts when no higher-priority needs-work draft pressure exists.

Marketplace packet controls in Seller Inventory:

- `Copy Marketplace Packet` copies a JSON packet for selected ready rows
- `Download Marketplace Packet` downloads the same JSON packet for selected ready rows
- `Download Marketplace CSV` downloads selected ready rows in spreadsheet form
- the Bulk Controls panel displays visible guardrails before export: cross-list prep only, no external publishing, no postage purchase, and the ready-row export count
- only activation-ready selected rows are included
- the export contains TCOS inventory ID, SKU, title, price, quantity, category, condition, description, image URL, shipping method, postage estimate, Coverage fields, Standard Envelope delivery-evidence requirement, under-$20 seller-protection provider/rate/max/cap/claim/refund/not-insurance fields, shipping-purchase guardrail fields, InstaComp scan ID, serial number, market/listing price evidence, and readiness blockers
- copied/downloaded JSON packets also include packet-level `crosslist_prep_only`, `externalPublishingApproved = false`, `shippingPurchaseIncluded = false`, shipping warning metadata, seller-protection warning metadata, an operator checklist, a prohibited-action manifest, and export context with selected count, ready count, visible count, active filters, and search text
- downloaded marketplace CSV rows include matching prohibited-action columns for external publishing, postage purchase, Coverage policy creation, seller payout release, and order fulfillment
- marketplace exports do not opt the seller into TCOS Under-$20 Seller Protection, create insurance, buy postage, or reimburse shipping; the opt-in must exist before fulfillment and a seller-protection reimbursement requires LetterTrack/USPS IMb delivery evidence that does not show delivered under TCOS rules
- downloaded marketplace packet/CSV filenames include ready-row count plus current status, readiness, source, and search context so saved files can be matched back to the seller inventory view that produced them

These packet controls are outbound preparation only. They do not publish to eBay, Whatnot, Shopify, COMC, or another external storefront. Before any real external publishing connector is enabled, implement and test platform-specific listing rules, seller authorization, idempotency, duplicate prevention, fee/shipping mapping, image upload rules, and external-listing reconciliation.

`/seller/marketplaces` now includes a Marketplace Packet Intake card that repeats those guardrails — cross-list prep only, no external publishing, no postage purchase, no Coverage policy creation, no seller payout release, no order fulfillment, not insurance, and no automatic TCOS Under-$20 Seller Protection activation —

and routes sellers back to ready or needs-work Seller Inventory rows. Use it as the receiving-side explanation for packet files created in Seller Inventory.

The same Seller Connections surface also shows safe marketplace API receipts after auth, status, sync-control, disconnect, preview, staging, reconciliation, outside-order import, and staged-promotion actions. Use `Copy Safe Receipt` or `Download Safe Receipt` to hand off the latest event, or `Copy Trail` / `Download Trail` for the recent session-saved receipt trail when debugging a multi-step marketplace sequence. Use `Clear Trail` after the handoff is captured or when the seller/operator starts a new sequence.

Multipart fallback request limits

The durable job path avoids relaying original images through a large browser-to-Next multipart request: derivatives upload directly to private Storage, and the scan request carries job/item/lease identifiers as JSON. The older fallback path still uses multipart form data and can hit a platform body limit. Multipart fallback draft creation is development-only; production draft creation requires a completed persistent queue row so database reservation and image-integrity guarantees cannot be bypassed.

Failure text:

```
Request body exceeded 10MB
Failed to parse body as FormData
```

Workaround:

1. Confirm `20260711010000_create_instacomp_scan_job_queue.sql` is applied so the durable path can start.
2. Use `Run Batch InstaComp` for large lots.
3. Let the browser optimize a fallback full image to about `900 KB` and a detail crop to about `180 KB`.
4. If parsing still fails, reduce the affected source resolution and retry only that row.
5. Never increase platform request limits as a substitute for the private direct-upload queue.

Serial-number troubleshooting

If a serial is visible but missing:

1. Confirm both front and back were paired correctly.
2. Inspect the original photo at full size; glare, sleeve reflections, tilt, blur, or aggressive compression can erase thin foil numbers.
3. Confirm OCR diagnostics says `paddleocr` and shows checked images/text.
4. Confirm the PaddleOCR worker token matches `PADDLEOCR_API_KEY` in `.env.development.local`.
5. Check the Paddle stderr log for lazy model-load or Windows inference errors.
6. Crop or reshoot the serial area with even lighting and the stamp square to the camera.
7. Retry the row.
8. Check final `Serial #` and the batch CSV/JSON export. External OCR can report no serial while the dedicated OpenAI pass finds one in final AI data.
9. If no source proves the complete fraction, leave it blank and keep the draft in review. Never invent a denominator.

Comp limitations

- suggested price is the median of included live matches
- the configured eBay Browse provider returns active asking prices, not completed-sales proof
- registered sold-data sources may appear as research links without live ingestion
- COMC active-price ingestion has been removed from the comp equation; COMC remains a checklist/reference/search source only
- Sportlots, Panini America, Upper Deck, Cardboard Connection, Blowout Cards, TCDB, and COMC checklist pages can help identity review, but checklist/reference sources must not be treated as sold comps unless a permitted sold-price provider supplies transaction evidence
- provider failure can leave the identification result usable while comps remain incomplete
- Supabase scan-save failure can leave `scanId` empty even when the browser displays the result

Never describe a suggested price as verified sold value unless the included comp set actually contains verified sold transactions.

InstaComp failure lookup

`OCR configured but did not return usable text`:

- verify the OCR URL and matching token
- restart Next after environment changes
- inspect the Paddle logs
- reduce request size

`401 Invalid OCR service token`:

- the Paddle worker token and TCOS `PADDLEOCR_API_KEY` do not match

Health works but first scan fails:

- the lazy model load or model cache failed; inspect Paddle stderr

`Missing OPENAI_API_KEY`:

- restore `.env.local` and restart Next

`No market price` or `No usable comps`:

- verify the card manually and enter a positive listing price only when the operator can support it

`Sign in to a seller account before creating drafts`:

- this message belongs to the older seller-only flow; for a seller-owned durable job, log in at `/account/login` and refresh InstaComp; for a store-owned job, confirm the admin cookie is valid

`Unauthorized`:

- the seller token and admin cookie are both missing/invalid, or the seller session could not refresh; log in again in the intended ownership context

`INSTACOMP_JOB_MIGRATION_REQUIRED`:

- apply `supabase/migrations/20260711010000_create_instacomp_scan_job_queue.sql` to the hosted Supabase project and reload the scanner

Job remains `uploading` after recovery:

- inspect which registered rows still lack confirmed front/back objects; if local files were never uploaded, cancel/clear the partial job and reselect the originals

Job remains `cancelling`:

- an active worker lease may still exist; wait for the request/lease to finish, then retry clear instead of manually deleting rows

Unexpected pairing:

- rename each file with final `-front` or `-back` tokens and reload the batch

33. Payment Reliability, Reconciliation, Disputes, And Seller Payouts

TCOS payment policy remains an 8% platform commission on the total website sale amount, including item price plus allocated buyer-paid shipping. The proposed `$5` monthly subscription is disabled and must stay disabled until separately approved, implemented, disclosed, and tested.

Payment safety architecture

Current payment controls include:

- Stripe-hosted checkout and seller onboarding
- no raw card or bank numbers stored by TCOS
- signed webhook verification
- idempotent webhook journal and duplicate-event protection
- duplicate-checkout prevention
- immutable transaction and financial-adjustment ledgers
- refund, dispute, reversal, and negative-balance accounting support
- seller payout holds during unresolved cases or shipment review
- daily Stripe-versus-TCOS reconciliation
- admin alerts for unmatched money
- Stripe dispute evidence staging and final submission controls
- isolated test-mode simulations

Daily money checklist

1. Open `/admin/launch-readiness` and inspect blocked database/configuration checks.
2. Open `/admin/financial-reconciliation` and verify the previous UTC day.
3. Resolve every unmatched-money alert with a required note after correcting the underlying record.
4. Open `/admin/order-review-cases` and handle payment, dispute, return, authenticity, shipping, and seller holds.
5. Open `/admin/seller-payouts`, refresh Stripe Connect state, and handle only eligible cash-out rows.

6. Do not approve live payments while reconciliation alerts, simulation failures, test residue, missing webhook events, or seller payout blockers remain.

Payment simulation runbook

Open:

```
/admin/payment-simulations
```

Run in this order:

1. Run `No-Money Suite`.
2. Confirm a real Stripe `sk_test_` key and test webhook signing secret are configured.
3. Run `Stripe Sandbox Suite`.
4. Type exactly:

```text
RUN STRIPE TEST
```

1. Run `Full Checkout E2E`.
2. Type exactly:

```text
RUN CHECKOUT E2E
```

1. Require zero failed scenarios.
2. Inspect every skipped scenario; skipped is not the same as passed.
3. Confirm no `[TCOS TEST]` product, test order, or test inventory remains.

The sandbox suite creates tagged Stripe test objects. The checkout E2E drill creates a disposable product/order, exercises checkout through refund, and removes the fixture. Tagged simulation webhooks are quarantined from production financial ledgers. Subscription renewal simulation is intentionally excluded while the monthly fee is disabled.

Live payment dual-lock runbook

Open:

```
/admin/live-payment-launch
```

Live checkout requires both independent locks:

1. Current database approval for `tcos-live-payments-v1`.
2. `TCOS_LIVE_PAYMENTS_ENABLED=true` in the deployed runtime.

Approval:

1. Enter the real operator name.
2. Review every launch gate.
3. Type exactly:

```text
APPROVE LIVE PAYMENTS
```

1. Confirm the immutable approval event appears.
2. Set the environment switch only after database approval and deploy/restart the runtime.

Emergency revocation:

1. Enter the operator name and reason.
2. Type exactly:

```text
REVOKE LIVE PAYMENTS
```

1. Confirm the revocation event. Database revocation blocks live checkout even if the environment switch is still true.

Required live conditions include:

- matching live secret/publishable key pair
- live webhook secret
- HTTPS production origin
- exactly 8% configured store commission
- latest full checkout E2E with at least eight scenarios and zero failures
- zero open reconciliation alerts
- zero test-order/test-product residue
- monthly subscription flag disabled
- verified live refund/dispute event delivery
- valid Stripe platform business details
- enabled live webhook at `{production origin}/api/webhook`
- live and payout-enabled connected sellers where seller routing applies

Required Stripe events:

```
account.updated
checkout.session.completed
refund.created
refund.updated
refund.failed
charge.dispute.created
charge.dispute.updated
```

```
charge.dispute.closed
charge.dispute.funds_withdrawn
charge.dispute.funds_reinstated
```

`/admin/launch-readiness` is a broad advisory configuration/database report. `/admin/live-payment-launch` is the actual database half of the two-lock checkout control. Neither page proves that live postage purchasing works.

If the live-payment approval migration is missing, the runtime gate fails closed and tells the operator to apply `supabase/migrations/20260710185000_create_live_payment_launch_gate.sql`. The approval button is disabled when either the gate table or immutable event table cannot be checked, and `/api/admin/live-payment-launch` returns a blocked `409` response instead of recording approval or surfacing an unclear write error. Missing approval tables are a migration problem, not an operator override problem.

`/admin/launch-readiness` now also includes a first-class Live Payment Launch Gate row plus database checks for `live_payment_launch_gates` and `live_payment_launch_events`. Live buyer payments are not configuration-ready when the live-payment audit tables are unavailable or when Stripe live mode is staged but the approval report still has blockers.

The top Launch Readiness payment banner follows the dedicated Live Payment Launch Gate, not the whole-page blocked count. This prevents unrelated shipping-provider, marketplace, identity, email, or future-feature blockers from making an already-open payment runtime look closed. Use the separate full-launch banner and checklist for those broader launch blockers and review items; that banner reports blocked and review counts separately.

Launch Readiness now also includes a Live money runway panel. It mirrors the dedicated live-payment gate summary with approval-blocker, launch-lock, warning, and live Checkout state counts, plus the first live-money next actions. The same panel, the `/admin` Launch Locks card, `/admin/live-payment-launch`, and `/api/admin/live-payment-launch` show the Live Money JSON Evidence contract: schema `tcos.liveMoneyGoNoGo.v1`, the post-smoke raw JSON command `npm --silent run status:live-money:json`, the final-window raw preflight command `npm --silent run preflight:live-money:json`, the timestamped archive helpers `npm run archive:live-money` and `npm run archive:live-money:preflight`, Supabase bootstrap and final live-payment runtime environment checklists, accepted go-live states, halt states, and the read-only no-money/no-postage guarantee. Use it to answer “how much until full live money?” from the main launch dashboard, then open `/admin/live-payment-launch` for the authoritative approval controls and immutable history.

Launch Readiness also includes an Emergency Backup Evidence panel and exports the same contract through `/api/admin/launch-readiness`, the Markdown brief, and the launch handoff bundle. The contract points operators at `npm run status:nightly-backup`, `npm --silent run status:nightly-backup:json`, `npm run verify:nightly-backup`, `npm --silent run verify:nightly-backup:json`, `npm run archive:nightly-backup-status`, `npm run archive:nightly-backup-verification`, `npm run status:go-live`, and `npm run archive:go-live-runway`. Before go-live, preserve current schedule health, scheduler proof, freshest backup timestamp, approximate age, current-for-last-scheduled-run status, retention keep count, over-retention count, launchd loaded/runs/last-exit evidence, verification ok, failed-check count, verified archive path, and computed SHA-256. Treat `scheduleHealth: current`, `verification.ok: true`, and either `schedulerProof: automatic_proven` or a documented first-run/manual-backup exception while launchd is loaded as the

accepted backup posture. These helpers must not deploy, upload, create Checkout, buy postage, release payouts, approve launch, revoke anything, create a new backup archive, or push Git.

The same live-payment evaluator is available from the terminal. `npm run status:live-money` is the read-only recurring-block check: it prints the live-money go/no-go state, approval-blocker count, launch-lock count, warning count, database approval state, runtime switch state, live Checkout state, first next actions, missing local bootstrap environment, local Supabase bootstrap status, local final live-payment runtime status, and a read-only guarantee while exiting cleanly even when blocked. Use `npm --silent run status:production:json` for raw quota evidence with schema `tcos.productionQuotaStatus.v1`, then use `npm run status:go-live` for the single read-only runway view: local Git `HEAD` / `origin/main` /working-tree cleanliness, go-live readiness state, blocker count, watch item count, blocker action categories, per-blocker action commands, next actionable step, next deploy step, next operator step, Vercel quota status with UTC plus local retry time, production deploy safety with clean production domain, unwanted alias, protected deploy sequence, launch command when quota opens, smoke command, Vercel scope rule, emergency backup schedule health, scheduler proof, freshest backup timestamp, approximate age, current-for-last-scheduled-run status, retention count, backup verification result, live-money state, missing bootstrap environment, local live-payment runtime readiness, and safe next commands, including both `npm run archive:nightly-backup-status` and `npm run archive:nightly-backup-verification`, without starting deploys, uploads, archive creation, Git push, Checkout, postage, payouts, launch approvals, or revocations. Live-money blockers point to the one-command operator evidence packet `npm run prepare:go-live-evidence`, which archives runway proof, archives nightly-backup status and verification proof, prints the no-secret live-money packet, archives it with a `.sha256` sidecar, verifies the packet, archives verifier evidence, prints bootstrap-only Vercel commands, prints the Supabase-only local template, reruns `npm run status:live-money`, archives the go-live evidence verifier result after the packet proves clean, and refreshes runway proof again so the final runway archive can show the current verifier proof. Run `npm run verify:go-live-evidence` or `npm --silent run verify:go-live-evidence:json` after a clean pushed packet to confirm the latest runway, backup, and live-money evidence was captured at `HEAD=origin/main` with a clean tree; run `npm run archive:go-live-evidence-verification` to preserve that verifier result under `.codex-run/go-live-evidence-verification/`. The targeted live-money handoff remains available as `npm run prepare:live-money-bootstrap`, the current-evidence operator shortcut is `npm run live-money:bootstrap-handoff`, and the expanded command chain remains available as `npm run live-money:env-packet`, `npm run archive:live-money-env-packet`, `npm run verify:live-money-env-packet`, `npm run archive:live-money-env-packet-verification`, `npm run live-money:vercel-bootstrap-commands`, `npm run live-money:bootstrap-template`, then `npm run status:live-money`. Use `npm --silent run status:go-live:json` when the combined runway status needs raw archivable evidence, or `npm run archive:go-live-runway` for a timestamped file under `.codex-run/go-live-runway/`; the JSON schema is `tcos.goLiveRunwayStatus.v1` and includes Git, go-live-readiness, quota, production-deploy-safety, emergency-backup, live-money, safe-next-command, and read-only-guarantee fields without scraping the text output. `npm run verify:production` also runs that non-blocking status command so every production verification carries the live-money posture without preventing a normal code deploy while live Checkout is intentionally locked. Before staging Supabase or Stripe live values, run `npm run prepare:go-live-evidence` when operators need the full launch-safe evidence packet for the current runway; it chains `npm run archive:go-live-runway`, `npm run archive:nightly-backup-status`, `npm run archive:nightly-backup-verification`, and `npm run prepare:live-money-bootstrap` without deploying, reading secrets, creating Checkout, buying postage, releasing payouts, or changing runtime switches. Then run `npm run verify:go-live-evidence` or `npm --silent run verify:go-live-evidence:json` to verify that the latest

local packet has all required runway, backup, and live-money proof, was captured at the pushed Git tip, has clean working-tree evidence, preserves seven-backup retention proof, and keeps deploy/money/postage side effects closed. Run `npm run archive:go-live-evidence-verification` when you need a timestamped verifier proof under `.codex-run/go-live-evidence-verification/`; `npm run prepare:go-live-evidence` now includes that archive step after the packet verifies cleanly, then refreshes runway proof so the latest runway archive carries the current verifier proof. Run `npm run prepare:live-money-bootstrap` when the blocker is only the missing Supabase bootstrap environment; it chains the no-secret checklist, timestamped packet archive, checksum/no-secret verification, verifier evidence archive, bootstrap-only Vercel command printout, Supabase-only local template printout, and live-money status check without reading secrets, deploying, calling Stripe/Supabase, buying postage, creating Checkout, or flipping runtime switches. Run `npm run live-money:bootstrap-handoff` when evidence is already current and operators only need the bootstrap-only Vercel command checklist, Supabase-only local template, and follow-up `status:live-money` check. Vercel env commands stage deployed runtime values only; local `npm run status:live-money` reads local `.env` or shell variables, so mirror the same values locally before expecting local status to clear, then redeploy only when quota is open and verify deployed runtime with smoke/live-money evidence. For individual steps, run `npm run live-money:env-packet` for the no-secret checklist, `npm --silent run live-money:env-packet:json` for schema `tcos.liveMoneyEnvPacket.v1` raw packet evidence, `npm run archive:live-money-env-packet` for a timestamped no-secret packet plus `.sha256` sidecar under `.codex-run/live-money-env-packet/`, `npm run verify:live-money-env-packet` or `npm --silent run verify:live-money-env-packet:json` for schema `tcos.liveMoneyEnvPacketVerification.v1` checksum, no-secret, and local/deployed-boundary verification, `npm run archive:live-money-env-packet-verification` for a timestamped verifier evidence file under `.codex-run/live-money-env-packet-verification/`, `npm run live-money:bootstrap-template` for Supabase-only local placeholders, `npm run live-money:env-template` for full placeholder values, `npm run live-money:vercel-bootstrap-commands` for prompt-based Vercel commands that include only Supabase bootstrap keys, or `npm run live-money:vercel-commands` for the full Supabase plus final live-payment runtime command set; those helpers do not read secrets, call Stripe or Supabase, deploy, buy postage, create Checkout, or flip `TCOS_LIVE_PAYMENTS_ENABLED`, and the Vercel command helper safety path rejects malformed `VERCEL_SCOPE` values before printing commands. Their command output is pinned to `vercel@56.2.0` through `npm exec` with `--cwd "$PWD"` so operators do not rely on an unverified global Vercel CLI. For raw JSON evidence, use `npm --silent run status:live-money:json` or final-window `npm --silent run preflight:live-money:json`; the JSON schema is `tcos.liveMoneyGoNoGo.v1` and includes the same state, counts, next actions, read-only guarantee, plus a `liveMoneyEvidence` contract with accepted go-live states, halt states, archive requirement, Supabase bootstrap environment checklist, final live-payment runtime environment checklist, companion status/preflight commands, and `localEnvironmentStatus` local environment readiness without npm's command banner. For the archivable timestamped files operators should preserve, run `npm run archive:live-money` after production smoke passes and `npm run archive:live-money:preflight` during the final go-live window; both write under `.codex-run/live-money-evidence/`, preserve the underlying status/preflight exit behavior, and add archive metadata for timestamp, command, local `HEAD`, local `origin/main`, and working-tree cleanliness. `npm run preflight:live-money` is the final-window gate: it exits nonzero until the state is `READY_FOR_RUNTIME_SWITCH` or `LIVE_MONEY_OPEN`. Both commands are read-only and must not create Checkout Sessions, Customers, PaymentIntents, refunds, disputes, payouts, labels, postage purchases, Coverage policies, launch approvals, or revocations.

For the 30-minute operator handoff, `npm run status:go-live` also lists the checkpoint helpers `npm run status:build-block`, `npm --silent run status:build-block:json`, `npm run prepare:build-block-checkpoint`, `npm run archive:build-block-checkpoint`, `npm run verify:build-block-checkpoint`, and `npm --silent run verify:build-block-checkpoint:json`, the selected-lane helpers `npm run next:build-block`, `npm --silent run next:build-block:json`, `npm run archive:next-build-block-action`, `npm run verify:next-build-block-action`, `npm --silent run verify:next-build-block-action:json`, and `npm run prepare:next-build-block-action`, plus the compact-history helpers `npm run status:build-block-history`, `npm --silent run status:build-block-history:json`, `npm run prepare:build-block-history`, `npm run archive:build-block-history`, `npm run verify:build-block-history`, and `npm --silent run verify:build-block-history:json`. Use those commands to preserve the current checkpoint, next local build lane, compact half-hour history, quota retry/remaining evidence, and configured deploy-timeout evidence while live money, postage, Checkout, payouts, and production deploys stay gated.

When the latest go-live evidence is already clean at `HEAD=origin/main`, `npm run status:go-live` advances the live-money blocker from “rerun the full packet” to the actual Supabase bootstrap handoff: run `npm run live-money:bootstrap-handoff` to print the bootstrap-only Vercel commands, print the Supabase-only local template, and rerun `npm run status:live-money` after the same values are staged in Vercel and mirrored locally.

The Live Payment Launch Gate also shows an operator summary, approval-blocker count, launch-lock count, ordered next-action list, and a final Live Money JSON Evidence packet. Approval blockers are the remaining payment-readiness checks that must pass before the database approval can be recorded. Launch locks are the two intentional final controls — auditable database approval and `TCOS_LIVE_PAYMENTS_ENABLED` — that keep live Checkout closed until the go-live window even after the rest of the payment stack is ready. Do not approve the database lock or change the runtime switch unless the final-window JSON evidence shows `READY_FOR_RUNTIME_SWITCH` or `LIVE_MONEY_OPEN`.

The Launch Attention Board near the top of `/admin/launch-readiness` pulls the current blocked and review items out of the full checklist and shows the first ten by severity. It is a triage shortcut over the same readiness data, not a separate approval gate. When TCOS can infer the right admin surface, each attention card includes an `Open related page` shortcut.

The same page links to `/api/admin/launch-readiness` for a compact JSON launch brief and `/api/admin/launch-readiness?format=markdown` for a Markdown handoff brief. These exports are read-only and summarize payment posture, live-money approval blockers, live-money launch locks, the live-money next-action list, shipping posture, Standard Envelope evidence readiness, provider purchase-attempt audit suite status/count/key coverage, dry-run cleanup, launch drill counts, top attention items with inferred admin `href` and absolute `url` targets, plus an operator-facing overall status and next recommended step. The JSON brief includes `brief.payment.liveMoneyEvidence` with the `tcos.liveMoneyGoNoGo.v1` schema, `npm --silent run status:live-money:json` post-smoke raw JSON command, `npm --silent run preflight:live-money:json` final-window raw preflight command, `npm run archive:live-money` and `npm run archive:live-money:preflight` timestamped archive helpers, accepted go-live states `READY_FOR_RUNTIME_SWITCH` and `LIVE_MONEY_OPEN`, halt states, archive requirement, and read-only no-money/no-postage side-effect guarantee. It also includes `brief.emergencyBackupEvidence` with the `tcos.nightlyBackupStatus.v1`, `tcos.nightlyEmergencyBackupVerification.v1`, and `tcos.goLiveRunwayStatus.v1` schemas, backup status/verification/archive commands, `~/Backups` default

folder guidance, seven-backup retention, accepted launch proof, and side-effect boundary. It also includes `brief.deploySafety` with the clean production domain, unwanted `truely-collectables-tt3b.vercel.app` alias, `api-deployments-free-per-day` quota code, rolling 24-hour quota reset instruction, simple Vercel team slug requirement for `VERCEL_SCOPE`, deployed/clean URL output contract, `brief.deploySafety.sequence` protected deploy order, and `npm run smoke:production` handoff command. It also includes `brief.sellerMarketplaceReceiptHandoff` with the `/seller/marketplaces` route, proof text, required controls, operations, and safe-use boundary for copied/downloaded marketplace API receipt handoffs. The Markdown brief and the deeper launch handoff bundle both include Live Money JSON Evidence, Emergency Backup Evidence, and Seller Marketplace Receipt Handoff sections requiring `/seller/marketplaces` proof text for `Copy Safe Receipt`, `Download Safe Receipt`, `Copy Trail`, `Download Trail`, and `Clear Trail`, and remind operators that the receipt trail is not an audit ledger, payment record, fulfillment proof, or provider reconciliation source of truth. The Markdown brief also includes a `Production Deploy Safety` section with the Vercel quota reset, clean-domain, unwanted-alias, Vercel scope rule, deployed/clean URL output, and `npm run smoke:production` handoff reminders.

When shipping is intentionally in `dry_run` mode with live shipping disabled and the shipping approval database is available, the launch brief treats live-shipping lock checks as `review` items rather than overall launch blockers. This keeps the handoff aligned with the Launch Locks card: live payments can be open while shipping stays safely locked.

`/admin/launch-gate-drill` runs a no-money runtime smoke over the payment and shipping launch locks. For payments, it verifies that test-mode Checkout remains available for simulations, invalid Stripe secrets fail closed, and the current live runtime state matches the live-payment launch report. It also shows the live-money runway with approval-blocker, launch-lock, warning, live Checkout state, and next-action counts so operators can reconcile runtime-smoke safety with remaining payment launch work. For shipping, it also carries the provider purchase-attempt audit suite status, expected five-scenario count, key-coverage result, missing/unexpected purchase-audit key lists, and the shared Shipping Provider Unlock Action Plan from the live-shipping launch report/provider setup packet. It uses synthetic key strings and does not create Checkout Sessions, Customers, PaymentIntents, refunds, disputes, labels, postage purchases, or Coverage policies.

The drill report now includes a Side-effect Guardrails section plus a Shipping Provider Unlock Action Plan section. The JSON API returns `sideEffectPolicy` and `report.shipping.providerSetupActionPlan`, and `/api/admin/launch-gate-drill?format=markdown` downloads a Markdown operator report with the same sections. This explicitly lists the read/evaluate operations the drill may perform, the no-secret shipping unlock sequence, and the forbidden operations it must not perform: Stripe money-object creation, postage quote/buy/void/record actions, seller Coverage purchase, external claim/policy creation, seller payout release, or marking orders shipped.

`/admin/production-smoke` is the admin-facing production smoke report map. It does not run Vercel or external provider actions; it shows the launch command, clean production target, smoke coverage, common failure meanings, the post-smoke manual verification checklist, and the manual follow-up links operators should check after `npm run launch:production` succeeds. The checklist tells operators to capture proof for the Git tip/clean domain, Launch Gate Drill evidence, live-money runway proof, live-money JSON evidence, live-shipping lock posture, seller-protection money trail, shipping operations exports, Seller Connections Marketplace Packet Intake card, and Seller Connections receipt-handoff controls, then halt the launch lane at the first blocker instead of assuming a green smoke means every operator artifact is ready. The live-money runway proof must show approval-blocker count, launch-lock count, warning count, live Checkout state, and next live-money

actions before any runtime switch changes. The live-money JSON evidence should be archived with `npm run archive:live-money` after smoke passes and, during the final go-live window, `npm run archive:live-money:preflight` showing schema `tcos.liveMoneyGoNoGo.v1` with `READY_FOR_RUNTIME_SWITCH` or `LIVE_MONEY_OPEN` before any runtime switch changes; raw commands remain `npm --silent run status:live-money:json` and `npm --silent run preflight:live-money:json` when operators need terminal-only output. It also surfaces the deploy-live safety contract, including Vercel quota messaging, Vercel scope validation, unwanted `truely-collectables-tt3b.vercel.app` alias removal, clean production aliasing, deployed URL output, clean URL output, and the `npm run smoke:production` handoff. The smoke suite also verifies the admin dashboard, launch readiness page/JSON/Markdown, Launch Gate Drill page/JSON/Markdown including live-money runway, live payment gate, live shipping gate, admin shipping LetterTrack controls, the dashboard Shipping Provider Unlock Action Plan, the drill Shipping Provider Unlock Action Plan, the live-shipping Shipping Provider Unlock Action Plan, live-shipping purchase-audit key-drift card, Shipping Simulation Lab coverage for twenty policy/adapter scenarios plus five provider purchase-audit scenarios, shipping purchase-attempt audit simulations for live-gate/missing-setup/dry-run/packet text, shipping simulation API drift fields, launch handoff purchase-audit key-drift reminders, shipping provider exports, ranked shipping exceptions CSV shape, LetterTrack CSV export, the production smoke page's seller marketplace packet-intake and receipt-handoff coverage lines, `/seller/marketplaces` itself for Marketplace Packet Intake guidance and safe receipt handoff wording, and unauthenticated `/seller/inventory`, `/seller/orders`, and `/seller/payouts` for login gates before seller-owned data can render. Smoke requests default to 15 seconds, and `SMOKE_REQUEST_TIMEOUT_MS` must be integer milliseconds from `1000` through `120000`; malformed, infinite, fractional, zero, negative, or too-large values fail before admin authentication, Git fetch, or network requests. If any queued launch feature fails while the pushed Git tip is newer than production, treat the smoke output's queued-feature warning as deploy lag, read the `Queued launch feature failure(s): ...` line for the exact failed checks, and rerun deploy/smoke after Vercel accepts deployments.

The production smoke page and launch handoff bundle also include a Production Go/No-Go Ladder: verify the pushed stack with `npm run verify:production`, launch only when quota is open with `npm run launch:production`, halt if Vercel reports `api-deployments-free-per-day`, avoid rapid-fire deploy retries because Vercel can still upload files before returning the quota error, let the deploy helper's `.codex-run/vercel-quota-block.json` cooldown marker stop later attempts before upload unless `TCOS_VERCEL_QUOTA_RETRY_OVERRIDE=true` or `--force-quota-retry` is used intentionally, split deploy/smoke only intentionally, and ship only after smoke passes the clean production domain while the unwanted alias stays absent.

The Launch Gate Drill also shows Payment Launch Posture and Shipping Launch Posture cards. These cards separate runtime-smoke safety from operator launch readiness: the drill can pass while shipping remains `Locked Safe` because TCOS is still in dry-run postage mode or provider setup is incomplete. Treat `Locked Safe` as an intentional hold, not permission to buy postage.

Live Checkout runtime also probes the immutable live-payment event table after approval is verified. If the audit table becomes unavailable after an approval was recorded, live Checkout fails closed until the migration/table problem is fixed.

Financial reconciliation runbook

Open:

```
/admin/financial-reconciliation
```

1. Click `Run Previous UTC Day`.
2. Inspect critical and high alerts first.
3. Compare Stripe source ID, TCOS record ID, Stripe amount, TCOS amount, and difference.
4. Correct the underlying Stripe or TCOS record before closing the alert.
5. Click `Resolve` or `Ignore With Note`.
6. Enter a specific note; a note is mandatory.
7. Rerun or confirm the window after correction.

Rules:

- tolerance is one cent
- a run processes at most 1,000 Stripe balance transactions
- reaching that limit opens a critical alert
- rerunning a completed window replays the stored run
- scheduled reconciliation runs daily at 18:00 UTC
- cron requires `Authorization: Bearer {CRON_SECRET}`
- matching covers charges, refunds, disputes, transfers/payouts, Stripe fees, seller payables, and the 8% platform ledger

Dispute and chargeback runbook

1. Open `/admin/order-review-cases` or the case from `/admin/orders/[id]`.
2. Confirm case type, severity, Stripe dispute ID, evidence deadline, order, and seller scope.
3. Download the case packet.
4. Add evidence notes and move through the appropriate states: `open`, `evidence_gathering`, `waiting_on_buyer`, `waiting_on_seller`, and `under_review`.
5. Click `Generate And Stage`. Staging remains editable.
6. Review every evidence field against the order, tracking, policies, buyer communication, and Stripe record.
7. For final submission, click `Final Submit To Stripe` and type exactly:

```text
SUBMIT TO STRIPE
```

1. Final Stripe submission cannot be amended.
2. Record the Stripe outcome and TCOS outcome summary.
3. Apply payout resolution separately.

Payout-resolution choices:

- seller favorable: `release_to_seller`
- buyer favorable: `reverse_for_buyer` or `cancel_no_payout`
- appeal/continued review: `hold_for_appeal`

Seller release still requires shipment and clearance of payment, inventory, shipping, and case holds. Active or paid cash-out requests can block destructive payout-row changes.

Seller payout runbook

Seller surface:

```
/seller/payouts
```

Admin surface:

```
/admin/seller-payouts
```

1. Seller accepts `/seller-terms`.
2. Seller starts Stripe-hosted Express onboarding from `/account` or `/seller/payouts`.
3. Stripe—not TCOS—collects bank, identity, and payout credentials.
4. Admin opens `/admin/seller-payouts` and clicks `Refresh Stripe Status`.
5. Treat the account as ready only when onboarding is active, payouts are enabled, details are submitted, no current/past-due requirements remain, and no disabled reason exists.
6. Website checkout creates the 8% platform-fee row and seller-payable row using item price plus allocated buyer-paid shipping.
7. Seller payable starts at `hold_pending_fulfillment`.
8. Release only after shipment and after all cases/reviews clear.
9. Seller requests cash-out only against remaining `eligible` rows.
10. Admin advances `requested -> approved -> processing -> paid`.
11. Complete the real money movement in the approved provider first.
12. While `processing`, record the real provider payout reference and final processor fee.
13. Only then click `Mark Paid`.
14. Reconcile the provider reference in `/admin/financial-reconciliation`.

Critical warning: `Mark Paid` records TCOS state. It does not send money. Current TCOS payout movement is not automated.

34. Shipping, Postage, Coverage, Tracking, And Claims

Primary routes:

```
/admin/orders/[id]
/admin/shipping
/admin/shipping/simulations
```

Shipping policy

Standard Envelope is eligible when:

- merchandise subtotal is at most `$20.00`
- estimated weight is at most `3 oz`
- current estimate is one ounce per card
- USPS IMb delivery evidence is required through the approved Standard Envelope provider

TCOS automatically resolves to Ground Advantage at `$20.01` or more or above three estimated ounces.

Standard Envelope rate table:

Weight	Before 2026-07-12 07:00 UTC	At/after 2026-07-12 07:00 UTC
1 oz	$0.74	$0.78
2 oz	$1.03	$1.07
3 oz	$1.32	$1.36

Parcel rules currently embedded in TCOS:

- Ground Advantage: `$6.99` for the first five cards, then `$0.25` per additional card; free at `$149`
- Priority: `$12.99` for the first five cards, then `$0.25` per additional card; free at `$500`
- TCOS Under-$20 Seller Protection is the optional internal Standard Envelope program; it is not third-party insurance and depends on IMb delivery evidence such as Out for Delivery / Delivered in Mailbox when USPS data is available
- when a seller opts in for a Standard Envelope shipment, TCOS withholds a 2% seller-protection reserve from the seller payout row and caps reimbursement exposure at `$20.00` of item sale amount
- seller account, command-center, payout, order, admin payout, and financial reconciliation workspaces expose the withheld reserve, protected/liability row counts, protected item amount, internal reimbursement credits, and shipping-excluded amount so sellers and operators can see how the optional internal protection affected cash-out math
- if an opted-in under-$20 Standard Envelope shipment requires a buyer refund because delivery evidence does not show delivered under TCOS claim rules, TCOS reimburses the seller for the protected item sale amount up to `$20.00`; shipping is excluded and is not reimbursed
- when an eligible TCOS Under-$20 Seller Protection claim is marked `paid`, TCOS records an idempotent `seller_protection_reimbursement` financial adjustment that credits seller payable for the protected item amount only and saves a reimbursement allocation plan showing eligible payable seller rows, operator-readable skipped-row reasons, and excluded shipping amounts
- Stripe reconciliation run summaries now preserve TCOS internal seller-protection reimbursement context in JSON fields for reimbursed protected item amount, shipping excluded, adjustment count, and allocation count, so operators can distinguish internal seller-payable credits from actual Stripe payout movement
- Launch readiness treats Seller Protection Financial Adjustments as its own database capability, pointing operators to `20260712174000_add_seller_protection_financial_adjustments.sql` when TCOS internal seller-protection reimbursement support is missing
- the launch-readiness JSON, Markdown, and hand-off exports carry the same seller-protection operating contract so deployment handoffs preserve the internal-only model, evidence rule, reserve rate, item cap, and `financial_adjustment_ledger_entries` dependency

- production smoke coverage names the Under-$20 Seller Protection launch handoff and points manual follow-up to the handoff bundle, `/admin/financial-reconciliation`, and `/admin/shipping`
- `src/lib/seller-protection-launch-contract.ts` is the shared launch-handoff source for the Under-$20 Seller Protection contract; update that file first before changing launch-readiness or production-smoke wording
- `/admin/shipping` renders an always-visible Under-$20 Seller Protection Guardrails note so refund-proof and payout-blocker controls remain discoverable even before any claim creates a live exception row
- the production guardrail script checks that `/admin/shipping` keeps the Under-$20 Seller Protection Guardrails source text, so the production smoke expectation remains backed by a page-level contract
- failed production smoke rows now include a `missingText` field for required text checks, starting with the `/admin/shipping` LetterTrack and seller-protection controls smoke
- the `missingText` smoke diagnostics now cover `/admin/production-smoke`, `/api/admin/launch-readiness?format=handoff-bundle`, and `/admin/shipping/simulations` as well as the admin shipping controls check
- the launch-readiness Deployment Source section uses non-secret Vercel/Git metadata (`VERCEL_GIT_COMMIT_SHA`, `VERCEL_GIT_COMMIT_REF`, `VERCEL_GIT_REPO_OWNER`, `VERCEL_GIT_REPO_SLUG`, `VERCEL_URL`, and `VERCEL_ENV`) so operators can tell whether production is behind GitHub after a quota-delayed deploy
- production smoke refreshes `origin/main`, reads its full SHA, and requires `/api/admin/launch-readiness` to report the same `deployment.gitCommitSha`, short SHA, `main` ref, and clean production domain before treating the deployment as current
- when that Deployment Source assertion fails, production smoke prints `Deployment source mismatch` details for each mismatched value instead of forcing operators to inspect the JSON payload manually
- `/admin/production-smoke` now includes a post-smoke manual verification checklist with proof targets and blocked-action instructions for Git/domain freshness, Launch Gate Drill evidence, live-money runway proof, live-money JSON evidence, live-shipping lock posture, seller-protection money trail, shipping operations exports, Seller Connections Marketplace Packet Intake guardrails, and Seller Connections receipt handoff controls
- production smoke now verifies unauthenticated seller inventory, order, and payout workspaces render login gates before seller-owned drafts, orders, payout holds, cash-out blockers, or seller hold context can appear
- `/api/admin/shipping/provider-setup` now carries a Standard Envelope evidence/protection contract in JSON, CSV, env-template, Vercel-command, and operator-checklist exports, making clear that LetterTrack / USPS IMb is the trackable delivery-evidence provider while TCOS Under-$20 Seller Protection is an optional internal seller program, not third-party insurance; those exports also include the shared runtime gate validator result as `standardEnvelopeEvidenceContractReady` / `Runtime gate validator: ready` so operators can see whether the current contract is launch-safe. The JSON and export responses also include `X-TCOS-Shipping-Provider-Decision`, `X-TCOS-Shipping-Provider-Missing-Groups`, `X-TCOS-Shipping-Provider-Live-Blockers`, `X-TCOS-Shipping-Provider-Contract-Ready`, and `X-TCOS-Shipping-Provider-Summary` headers so smoke output can capture provider-readiness posture without parsing each file body.
- when a seller does not opt in for a Standard Envelope shipment, no 2% reserve is withheld and the seller is responsible for refunding the buyer in full if the shipment is lost or cannot satisfy TCOS delivery-

evidence rules
- parcel Coverage is required for Ground Advantage and Priority shipments
- current buyer charge for Coverage is zero

Critical live-postage warning

TCOS currently has only a dry-run shipping provider adapter. No live postage-provider adapter is approved. Setting `TCOS_SHIPPING_PURCHASE_MODE=live` deliberately blocks/throws instead of buying postage.

`Prepare Label + Coverage Record` now saves a shipping adapter profile on the label metadata. `/admin/orders/[id]` and `/admin/shipping` display that adapter profile so the operator can see the configured provider, service, carrier, purchase mode, missing credentials, and live-block reason without opening raw metadata.

`/admin/shipping` also includes a Provider Setup Checklist. Use it to inspect the three setup lanes before any shipping launch decision:

- Standard Envelope / IMb
- Ground Advantage / Priority
- Shipment Coverage

The checklist displays provider names, service/carrier labels, purchase mode, live-purchase support state, and required secret names. It does not display secret values and it does not call any live provider. Standard Envelope currently uses the LetterTrack / USPS IMb handoff path: TCOS can export eligible Standard Envelope rows as a LetterTrack import CSV, but the operator must import/print in LetterTrack and then record the assigned IMb or tracking reference back into TCOS before marking the order shipped.

`/admin/shipping` exposes `Export LetterTrack CSV`, which downloads `/api/admin/shipping/lettertrack-export`. The export includes only Standard Envelope labels in `planned`, `purchase_pending`, or `rate_selected` status. Rows missing recipient name, address line 1, city, state, or postal code are skipped so the import file is not polluted with unshippable envelopes; the response headers include row count, skipped count, and an operator-readable skipped-reason summary for missing order rows or incomplete recipient/address data. LetterTrack provides IMb delivery evidence; TCOS Under-$20 Seller Protection remains internal, item-only, optional per seller shipment, and not third-party insurance. Each CSV row also carries the seller-protection contract fields operators need beside the mailpiece: opt-in required, 2% reserve rate, `$20.00` item-only cap, `item_sale_amount_excluding_shipping` basis, no shipping reimbursement, and the IMb/LetterTrack delivery-evidence requirement.

The provider setup exports also repeat this Standard Envelope evidence/protection contract and print the shared runtime gate validator result. Use those exports during provider onboarding to prevent a false dependency on third-party insurance for cards under `$20.00`: the must-have provider requirement is trackable IMb delivery evidence that can show delivered, while the 2% / `$20.00` seller protection is a TCOS internal reserve and reimbursement workflow. If the seller did not opt in for that shipment, TCOS does not reimburse the seller and the seller remains responsible for the buyer refund when delivery evidence fails TCOS rules.

After the LetterTrack import/print step, use the `/admin/shipping` `LetterTrack IMb Recording` panel to paste the assigned IMb or LetterTrack mailpiece reference back into TCOS. This writes `lettertrack_imb_recorded`, changes the label to `printed`, stores the IMb as the tracking and coverage evidence reference, and copies the tracking reference onto the order row. Only mark the order shipped after the envelope is actually mailed.

After USPS/LetterTrack scan history appears, use the `/admin/shipping` `LetterTrack Delivery Evidence` panel to record `Delivered`, `Out for Delivery`, `Not Delivered`, `Delivery Exception`, `Returned`, or other IMb status evidence. These events are stored in `order_shipping_tracking_events` and are included in label and claim packets. When an under-$20 Standard Envelope claim draft is opened, TCOS snapshots the latest LetterTrack evidence into the claim metadata and packet. Delivered evidence supports closing the shipment trail and blocks seller-protection payout unless an operator documents a current or previously saved explicit override note; not-delivered, exception, or returned evidence supports TCOS Under-$20 Seller Protection claim review. Each submitted, under-review, approved, paid, or denied seller-protection status change also records `latest_lettertrack_delivery_evidence_review` so the claim has an audit trail of the current IMb evidence even before payout.

Before `Mark Paid` creates a seller-protection reimbursement, TCOS reloads the latest LetterTrack evidence and refuses payout unless claim-review evidence is present or the current note or a previously saved status note contains an override reason. `Mark Paid` also refuses eligible under-$20 seller-protection payout unless the current note or a previously saved status note confirms buyer/customer refund evidence or a refund reference, because TCOS only reimburses the seller after buyer refund proof is documented. `/admin/shipping` and `/admin/orders/[id]` show the saved claim snapshot, latest saved status-review, buyer-refund proof readiness that updates from the typed note plus saved metadata, saved buyer-refund gate, and current LetterTrack evidence recalculated from loaded tracking events; the order page loads up to 100 tracking events and the shipping cockpit loads up to 500 so older IMb evidence remains visible during review.

Claim evidence packets include the saved claim snapshot, Latest Saved LetterTrack Evidence Review, Seller-Protection Buyer Refund Evidence Gate, and a Current LetterTrack Evidence Review recalculated from the current tracking events in the packet. When a payout gate decision is recorded, the claim evidence packet also includes a dedicated LetterTrack Seller-Protection Payout Gate section with allowed/blocked, override, reason, latest status, and latest tracking. If `Mark Paid` creates or reuses an internal seller-protection reimbursement, the claim action card and claim evidence packet also include a Seller-Protection Reimbursement Allocation section with inserted credit count, requested/reimbursed/remaining plan amounts, per-row allocation amounts, operator-readable skipped-row reasons, and shipping-excluded amounts. Approved seller-protection claims missing refund proof appear in the `/admin/shipping` priority board and the `/api/admin/shipping/exceptions` CSV as `seller_protection_refund_proof_missing`; claims blocked by LetterTrack delivery-evidence rules appear as `seller_protection_payout_blocked`.

Provider setup packets are available from `/admin/shipping`:

- `Setup JSON` opens `/api/admin/shipping/provider-setup`
- `Setup CSV` opens `/api/admin/shipping/provider-setup?format=csv`

These exports are safe to use for operator setup review because they include secret names and configuration status only, not secret values.

The provider setup packet now includes a Shipping Provider Unlock Action Plan in JSON, CSV, env-template, Vercel-command, operator-checklist, `/admin/shipping`, `/admin/launch-readiness`, `/admin/live-shipping-launch`, and launch handoff bundle outputs. Work it in order:

1. Choose provider accounts for Standard Envelope / IMb, parcel labels, and shipment Coverage.
2. Stage the selected Vercel environment names without putting secret values in Git, chat, screenshots, tickets, or exported packets.

3. Keep shipping runtime locked with `TCOS_SHIPPING_PURCHASE_MODE=dry_run` and `TCOS_LIVE_SHIPPING_ENABLED=false`.
4. Prove live adapter evidence for quote, buy, void, Coverage purchase, webhooks, reconciliation, and audit packets.
5. Approve, deploy, and smoke only after the live-shipping gate is ready.

The provider setup packet includes an operator decision:

- `dry_run_only` means provider credential groups appear staged, but TCOS will still only simulate purchases.
- `needs_provider_setup` means at least one Standard Envelope, parcel-label, or Coverage credential group is missing.
- `live_blocked` means live purchase mode is enabled while TCOS still has no approved live adapter.
- `ready_for_live_adapter_build` means the setup packet is clear enough to begin live-adapter implementation work, but not to buy postage yet.

The same decision appears in the `/admin/shipping` Provider Setup Checklist as the Shipping setup verdict, so an operator can see the current go/no-go state without opening the export.

`/admin/shipping` now also shows a Live Shipping Runway under the setup verdict. This board separates:

- what is allowed now: dry-run planning, setup exports, external label purchase, and manual recording of real tracking/Coverage references
- what must be finished next: missing provider credential groups
- what must be built before TCOS can buy postage: quote, buy, void, Coverage purchase, webhook reconciliation, and audit-packet proof
- what must not happen: mailing dry-run labels, marking dry-run tracking as shipped, or enabling live mode before launch readiness and simulations are clean

The Live Shipping Runway now includes a Live Adapter Approval Checklist. The provider setup packet and `/admin/shipping` must show all of these gates ready before TCOS treats live postage or Coverage purchase as approved: provider credentials, live adapter implementation, quote/buy/void tests, Coverage purchase tests, provider webhook plus reconciliation approval, twenty-scenario shipping simulation pass evidence, five-scenario provider purchase-attempt audit pass evidence, and explicit admin live-shipping approval. Secret presence alone is not enough to enable live postage.

`/admin/live-shipping-launch` is the auditable live-shipping database lock. It evaluates the current live-shipping approval version, `TCOS_LIVE_SHIPPING_ENABLED`, `TCOS_SHIPPING_PURCHASE_MODE`, provider setup, the Standard Envelope evidence/protection contract, live requirement checklist, the twenty-scenario shipping simulation suite, the five-scenario provider purchase-attempt audit suite, live approval report, and dry-run cleanup status. Approval writes to `live_shipping_launch_gates` and appends immutable `live_shipping_launch_events`; revocation clears the approval side of the lock. Live shipping still requires both this database approval and the environment/runtime switches, and the current dry-run-only provider adapter still blocks live postage execution.

The live-shipping report now includes a dedicated Provider Purchase-Attempt Audit Suite check. It must pass all five expected scenarios and key coverage before `approvalReady` can become true. This prevents a live-

shipping approval from relying only on the twenty policy/adapter shipping scenarios while the blocked live-gate, missing-setup, dry-run sentence, empty-packet, or packet-line purchase-attempt audit text has drifted.

The live-shipping report now includes a dedicated Standard Envelope Evidence Contract check. It must show that LetterTrack / USPS IMb is delivery evidence that can show delivered, while TCOS Under-$20 Seller Protection is an optional internal seller program with seller opt-in required, 2% reserve, `$20.00` item-only cap, no shipping reimbursement, and no third-party insurance claim.

The live-shipping launch page and `/api/admin/live-shipping-launch` JSON report also expose the shared Standard Envelope runtime gate validator result, so operators can confirm the same `standardEnvelopeEvidenceContractReady` state that appears in the provider setup exports before approving live postage.

If the live-shipping approval migration is missing, the runtime gate fails closed and tells the operator to apply `supabase/migrations/20260711185500_create_live_shipping_launch_gate.sql`. Do not switch `TCOS_SHIPPING_PURCHASE_MODE` to `live` until `/admin/launch-readiness` shows the live-shipping gate and immutable event tables available and `/admin/live-shipping-launch` shows the approval report is safe.

The live-shipping approval button is disabled when the approval database check cannot run, and `/api/admin/live-shipping-launch` returns a blocked `409` response instead of recording approval or surfacing an unclear write error. Missing approval tables are a migration problem, not an operator override problem.

Provider purchase attempts in `/api/admin/orders/[id]/shipping-labels` now check the live-shipping runtime gate before calling the provider adapter. Manual external label purchase/void recording remains available, but live provider purchase is blocked unless the database gate, immutable event table, `TCOS_LIVE_SHIPPING_ENABLED`, `TCOS_SHIPPING_PURCHASE_MODE`, Standard Envelope evidence/protection contract, live requirements, and dry-run cleanup checks all pass.

When a provider purchase attempt is blocked by the live-shipping gate or missing provider setup, TCOS now stores the Standard Envelope evidence validator snapshot in the blocked purchase event and the label's latest purchase-attempt metadata. The ranked shipping exceptions CSV repeats that validator state in the blocked-purchase issue detail so an operator can tell whether the under-$20 card-shipping contract was intact when the attempt was stopped. The CSV response also includes `X-TCOS-Shipping-Exceptions-Rows`, `X-TCOS-Shipping-Exceptions-Critical`, `X-TCOS-Shipping-Exceptions-Warning`, `X-TCOS-Shipping-Exceptions-Watch`, and `X-TCOS-Shipping-Exceptions-Summary` headers so smoke output and operators can spot the active exception posture without parsing the whole CSV. `/admin/shipping` shows that same snapshot on blocked purchase attempt cards, `/admin/orders/[id]` shows the latest provider purchase attempt on each shipping label card, and `/api/admin/shipping-labels/[id]/packet` includes a Provider Purchase Attempt Audit section plus blocked-event audit lines so a downloaded label packet carries the live gate, missing setup, and evidence-validator context.

`/admin/launch-readiness` also includes the same Shipping Setup Verdict, the live-shipping launch gate status, the Standard Envelope evidence validator state, and database checks for `live_shipping_launch_gates` plus `live_shipping_launch_events`. Treat this as the production-readiness warning surface; it does not mean live postage buying is enabled.

The same `/admin/launch-gate-drill` report also checks the live-shipping runtime lock without quoting, buying, voiding, or recording a provider label. In dry-run mode, the expected safe result is that the dry-run

shipping path remains available while live postage remains locked. In live mode, the drill expects the runtime gate to match the live-shipping launch report. The drill page, JSON, and Markdown report also show the Standard Envelope evidence validator state so operators can confirm the under-$20 card-shipping contract stayed intact during the no-postage runtime smoke.

The drill's Shipping Launch Posture card lists the blocked live-shipping checks and next actions, such as configuring provider credentials, building the live quote/buy/void adapter, proving Coverage purchase, and wiring provider webhook reconciliation. Keep shipping in `Locked Safe` until those items are actually ready.

The admin command center (`/admin`) also shows a Launch Locks card in the side rail with the current no-money gate drill result, payment posture, live-money approval-blocker/launch-lock/warning counts, the first live-money next action, the Live Money JSON Evidence archive/preflight commands and accepted/halt states, Emergency Backup Evidence commands for `npm run archive:go-live-runway`, `npm run archive:nightly-backup-status`, and `npm run archive:nightly-backup-verification`, the seven-backup retention reminder, read-only/no-backup-creation/no-Git-push boundary, shipping posture, Standard Envelope evidence validator state, and direct links to Gate Drill, Launch Readiness, Live Payment Gate, Live Shipping Gate, and the launch brief JSON/Markdown exports. It also shows the Shipping Setup verdict, the provider setup Standard Envelope evidence validator state, and the first no-secret Shipping Provider Unlock Action Plan steps, and includes that verdict in operator alerts, so blocked shipping-provider setup is visible from the first admin landing page.

Never mail with references beginning:

```
IMB-TCOS-DRYRUN-
USPS-TCOS-DRYRUN-
dryrun-
```

Dry-run labels cannot be mailed, marked shipped, saved as real tracking, entered as manual purchases, or used for real Coverage claims.

The shipping Coverage policy save endpoint also rejects dry-run policy IDs and refuses to mark a dry-run label as covered. To attach a real Coverage policy, first record a real external label/manual purchase or void the dry-run record and create the correct fulfillment record.

The tracking save endpoint also rejects dry-run tracking references and refuses to save tracking while the active shipping label is still a dry-run simulation. Save tracking only after a real external label has been recorded or the dry-run record has been voided.

The mark-shipped endpoint uses the same broad dry-run reference detection before changing fulfillment status or sending shipment email.

Dry-run shipping detection for fulfillment-critical backend routes is centralized in `src/lib/shipping-dry-run.ts`. Use that helper instead of copying one-off string checks when adding new label, tracking, Coverage, claim, or shipment actions.

Buyer and seller account order APIs also use the shared dry-run detector before showing tracking or carrier values, so dry-run references stay hidden from account-facing order views.

Admin order detail and packing-slip printing also use the shared detector before treating label, tracking, shipment, or Coverage references as printable fulfillment data.

Admin Fulfillment Center hides dry-run order tracking/carrier values behind an operator warning, and Command Center alerts if any order rows still carry dry-run shipping references.

Transaction evidence reports, order review case packets, and Stripe dispute evidence staging suppress dry-run tracking/carrier values as shipment proof and add dry-run evidence warnings where applicable.

Seller order list/detail APIs already suppress dry-run tracking/carrier values, and the seller order UI now shows an explicit dry-run shipping warning instead of presenting hidden simulated references as missing real tracking.

Seller payout request summaries include a dry-run shipping flag without exposing simulated tracking, and payout-linked order cards keep dry-run shipped rows routed to Shipping Orders instead of completed/cash-out-only flows.

Seller Home consumes the same dry-run shipping flag for payout pressure and order workspace cards, keeping dry-run shipped rows in shipping follow-through until real fulfillment proof is recorded.

Admin Seller Payout Review also checks the shared dry-run shipping detector before releasing ledger rows. A row cannot be moved to eligible while the order only has TCOS dry-run tracking, even if the order status says shipped.

Seller payout release guards now use the order-level dry-run shipping proof helper in `src/lib/shipping-dry-run-cleanup.ts`, so release checks inspect the order row, shipping label rows, and tracking-event rows before treating fulfillment proof as real.

Seller cash-out request review blockers now include dry-run shipping rows, so requests cannot move to approved, processing, or paid while any linked payout row only has simulated TCOS tracking.

Order review case payout resolution uses the same detector before releasing held rows to seller eligibility, and the admin case UI lists dry-run shipping as the skipped-row reason.

Admin Shipping and the ranked shipping exception CSV now use the shared detector for dry-run label, tracking, shipment, and Coverage references while preserving event-based simulated-purchase detection.

Shipping label packets, Coverage claim evidence packets, and Coverage claim create/update routes also use the shared detector, with packets still preserving event-based simulated-purchase detection for audit evidence and dry-run safety notices.

Shipping label audit packets also flag dry-run evidence when the order snapshot or tracking events contain dry-run references, not only when the label row itself was created by the simulated adapter.

Launch Readiness now includes a fail-closed dry-run shipping cleanup gate. Live buyer payments stay blocked when recent label, tracking-event, or order rows still contain TCOS dry-run shipping references or when the cleanup check cannot run.

The Live Payment Launch Gate uses the same dry-run shipping cleanup check inside the approval report, so `/api/admin/live-payment-launch` cannot record a new live-payment approval while sampled shipping, tracking-event, or order rows still contain TCOS dry-run references.

Live Checkout runtime also rechecks dry-run shipping cleanup after the environment switch and database approval pass, so an older approval cannot keep live Checkout open if new dry-run shipping residue appears later.

The shared dry-run shipping cleanup scanner lives in `src/lib/shipping-dry-run-cleanup.ts`; use it for launch, live-payment, or future money/fulfillment gates instead of duplicating label/event/order cleanup queries.

`/admin/shipping#dry-run-cleanup` is the Dry-Run Shipping Cleanup Center. It lists the exact order rows, shipping label rows, and tracking-event rows that are still blocking launch or seller payout release because they contain TCOS dry-run proof. Use `Retire Dry-Run Proof` only for simulated TCOS rows: it clears fake order tracking, voids simulated label records, retires simulated tracking events, and preserves audit metadata showing who retired the proof and why. It does not buy postage, void real postage, purchase Coverage, or prove shipment. After retiring dry-run proof, record a real external label, real carrier/IMb tracking, and real Coverage policy before shipping or releasing seller funds.

The preferred cleanup test flow is `Retire + Record Real Label` from `/admin/shipping#dry-run-cleanup`. That retires simulated proof, then opens `/admin/orders/[id]?shippingAction=manualPurchase` with the manual label + Coverage form already expanded. Save the real provider, carrier, tracking/IMb, postage, provider IDs, label URL/PDF if available, Coverage provider, Coverage policy ID, and coverage amount. Use `Retire Only` only when you are intentionally clearing simulated proof before recording real proof later.

Real shipment runbook

1. Open `/admin/orders/[id]`.
2. Click `Prepare Label + Coverage Record`.
3. Confirm the resolved method is correct under the value/weight rules.
4. Buy the real label and real Coverage policy externally.
5. Click `Record Manual Purchase`.
6. Enter the real provider, carrier, tracking or IMb, postage, provider label/shipment ID, label/PDF URL where available, Coverage provider, policy ID, and covered amount.
7. Confirm no dry-run reference is present.
8. Open `/admin/shipping`.
9. Clear missing-tracking, missing-policy, blocked-purchase, and other priority exceptions.
10. Download the label/coverage audit packet when needed.
11. Save tracking.
12. Mark shipped only after the real label, real tracking, and real policy are recorded.

The shipping queue also supports priority sorting, external void records, claim status, label/coverage packets, manual label records, and ranked exception CSV export.

Coverage claim runbook

1. Confirm a real, non-dry-run label and Coverage policy exist.
2. Open a Coverage claim draft from the order.
3. Download the claim evidence packet.
4. Submit the claim to the external provider. TCOS does not submit it.
5. Record the external provider claim ID and note.
6. Advance only through valid states:

```text
draft -> submitted
submitted -> under_review / approved / denied / cancelled
```

under_review -> approved / denied / cancelled

approved -> paid / denied / cancelled

```

1. Treat paid, denied, and cancelled claims as audit-locked.

### Shipping simulation runbook

Open `/admin/shipping/simulations` or run `npm run verify:shipping`. Require all twenty policy/adapter assertions plus the five provider purchase-attempt audit assertions. The page and `POST /api/admin/shipping/simulations` now expose both the shipping scenario suite and a `purchase_audit` suite so operators can confirm blocked live-gate, missing-setup, dry-run, empty-packet, and packet-line audit text before launch. The page shows missing and unexpected keys for both the shipping manifest and the purchase-audit manifest so drift can be diagnosed without reading raw JSON first. It also shows a first-class Under-$20 Seller Protection Allocation Contract panel, and the API returns `seller_protection_allocation_contract`, so operators can verify item-only reimbursement, shipping exclusion, and non-opted-in seller liability without digging through raw scenario assertions. `/admin/launch-readiness`, `/api/admin/launch-readiness`, and the no-money Launch Gate Drill repeat the purchase-audit missing/unexpected key lists so live-shipping drift is visible from the main launch surfaces too.

- `$19.99` and 3 oz stays Standard Envelope
- `$20.01` forces Ground Advantage
- more than 3 oz forces Ground Advantage
- Standard Envelope requires delivery evidence and records whether the seller opted into TCOS Under-$20 Seller Protection
- opted-in under-$20 Standard Envelope claims reimburse item sale amount only and exclude shipping
- non-opted-in under-$20 Standard Envelope claims reimburse `$0.00` and leave refund liability with the seller
- mixed protected/unprotected under-$20 claim rows cap TCOS reimbursement at `$20.00`, exclude shipping, and leave unprotected rows outside reimbursement
- Mark Paid reimbursement allocation creates seller credits only for eligible payable seller rows, stops at the `$20.00` cap, records operator-readable skip reasons for forged, unprotected, missing-seller, zero-covered, or cap-reached rows, and keeps shipping excluded
- Mark Paid buyer-refund gate accepts a current or previously saved internal note confirming buyer/customer refund evidence or a refund reference before TCOS seller-protection reimbursement
- provider setup exports state that LetterTrack / USPS IMb supplies trackable delivery evidence while TCOS Under-$20 Seller Protection remains an optional internal, item-only, non-insurance program
- Coverage is required for parcel shipping
- shipping adapter profiles expose provider, carrier, credential, Coverage, live-support, and manual-fallback state
- Standard Envelope labels can export to a LetterTrack import CSV with recipient address, order reference, declared value, and IMb recording instructions
- LetterTrack CSV rows carry the under-$20 seller-protection contract: opt-in required, 2% reserve, `$20.00` item-only cap, shipping excluded, and IMb delivery-evidence requirement
```

- LetterTrack delivery evidence snapshots distinguish delivered shipments from not-delivered claim-review support before TCOS Under-$20 Seller Protection reimbursement
- under-$20 seller-protection payout blocks delivered LetterTrack evidence, allows not-delivered review evidence, and accepts a current or previously saved explicit override note for exceptions
- under-$20 seller-protection status changes save LetterTrack evidence-review audit records on submitted, under-review, approved, paid, and denied statuses
- dry-run Standard Envelope and Ground Advantage adapter purchases behave as dry runs
- provider purchase-attempt audit lines preserve Standard Envelope evidence readiness, live-gate reasons, missing credential blockers, dry-run purchase status, and packet fallback text

The dry-run Standard Envelope purchase assertion uses the active Standard Envelope rate table at run time, so it should follow the July 12, 2026 rate change without hardcoded stale postage.

The page also shows Purchase Attempt Audit Coverage and a Live Shipping Approval Report. Live shipping must remain blocked unless the report says `ready_to_request_live_mode`, the setup verdict is acceptable, every live requirement is ready, and there are zero blockers. This page does not contact USPS or Coverage and does not buy postage.

35. Seller eBay Connection, Staging, Outside Orders, And Reconciliation

Seller-scoped eBay tokens and workflows are separate from the Store #1 global `ebay_tokens` connection.

Connect and stage inventory

1. Admin enables the correct store-wide eBay environment and sync policy in `/admin/settings`.
2. Seller opens `/seller/marketplaces`.
3. Seller connects eBay through OAuth.
4. Reconnect old accounts when identity or fulfillment scopes are missing.
5. Run preview first; preview performs no inventory write.
6. Stage the first/next 25 rows or all remaining rows.
7. Review ready rows, needs-review rows, blocked conflicts, missing SKU/listing ID, authenticity fields, and activation blockers.
8. Promote only reviewed rows into seller-owned TCOS drafts.
9. Open `/seller/inventory` and activate separately after payout verification and listing readiness pass.

Reconcile and import outside orders

1. Import outside eBay orders from `/seller/marketplaces`.
2. Run reconciliation for linked inventory.
3. Review quantity reductions, sold rows, price mismatches, missing remote items, and failed rows.
4. Resolve conflicts instead of forcing promotion or quantity changes.

Safety rules:

- reconciliation may lower TCOS quantity or mark sold
- reconciliation never raises local quantity automatically

- outside eBay sales do not enter TCOS checkout, seller payout, or 8% platform-fee ledgers
- eBay refunds/cancellations are review-only and never restore stock automatically
- pausing retains credentials, staging, history, and inventory
- disconnecting deletes stored seller eBay tokens but retains staging/history/inventory
- eBay revocation or account-deletion notifications must disable future access without deleting audit history

Scheduled seller reconciliation runs at 09:00 UTC and requires `CRON_SECRET`. The current cron processes one connected seller per invocation and one reconciliation/import batch. Multiple connected sellers may require a more frequent schedule or a future scaling change.

36. Local Startup, Backup Verification, And Laptop-Failure Recovery

MacBook nightly emergency backup

The MacBook nightly emergency backup is a fast source recovery path for daily work. Run it manually with:

```
npm run backup:nightly
```

By default it creates `~/Backups/truely-collectables-nightly-*.tar.gz`, a `.sha256` checksum file, and a `.manifest.json` evidence file on the MacBook. On Windows, the matching drive-root example is `C:\Backups\`. Modern macOS blocks unprivileged creation of a true `/Backups` drive-root folder; if an admin creates and owns `/Backups`, reinstall the scheduler with `npm run backup:nightly:install -- --backup-dir /Backups`. The archive preserves the working repository, `.git` history, uncommitted working-tree files, and ignored `.env*` files for local emergency restore. It intentionally excludes rebuildable or noisy folders such as `node_modules`, `.next`, `.next-*`, `out`, `build`, `.codex-run`, `TCOS_BACKUP`, PaddleOCR virtual environments/caches, `coverage`, `.venv`, `.DS_Store`, and `*.tsbuildinfo`.

Retention is a seven-backup rolling window. Before day 8 is written, the oldest dated backup is removed so the new dated backup replaces day 1. Day 9 replaces day 2, and the cycle continues. The default `TCOS_NIGHTLY_BACKUP_KEEP` value is `7`.

The same command also attempts `git push origin HEAD:main` when the current branch is `main`. That Git leg only pushes already-committed source. It does not run `git add`, does not create an automatic commit, and does not publish ignored `.env*` files or random untracked files. If the working tree is dirty, the dirty files are captured by the local Mac archive and the manifest notes that Git only received committed source.

Install the nightly macOS LaunchAgent with:

```
npm run backup:nightly:install
```

The installer writes `~/Library/LaunchAgents/com.truelycollectables.nightly-emergency-backup.plist`, schedules the backup for 02:30 local time, passes the selected backup folder through `TCOS_NIGHTLY_BACKUP_DIR`, and writes logs under `~/Backups/logs` unless `--backup-dir` is supplied. Use `npm run backup:nightly:install -- --no-load` to write the plist without loading it. Use `npm run backup:nightly -- --local-only` for a no-network emergency archive, or `npm run backup:nightly -- --backup-dir .codex-run/nightly-backup-test --local-only` for a workspace-local test.

Check the nightly scheduler and seven-backup rotation without creating an archive:

```
npm run status:nightly-backup
npm --silent run status:nightly-backup:json
npm run verify:nightly-backup
npm --silent run verify:nightly-backup:json
npm run archive:nightly-backup-status
npm run archive:nightly-backup-verification
```

The JSON schema is `tcos.nightlyBackupStatus.v1`. The helper only reads the LaunchAgent plist, launchd runtime state, selected backup folder, and log metadata; it creates no archive, starts no Git push, deploy, Checkout, postage, payout, launch approval, or revocation. It also reports the freshest backup timestamp, UTC plus local timestamps for the latest backup and scheduled runs, approximate age, current-for-last-scheduled-run status, retention keep count, and over-retention count; schedule health such as `current`, `pending_first_run`, or `overdue_or_failed`; plus scheduler proof states such as `automatic_unproven`, `automatic_proven`, or `automatic_failed` with launchd loaded/runs/last-exit evidence so a missed 02:30 run or never-fired scheduler is visible. If the automatic run misses but `npm run backup:nightly` creates a fresh verified archive for the current scheduled window, the backup runway records `manual_current_after_automatic_failure`, accepts the backup posture for launch safety, and keeps operator watch required until a later automatic run is proven. The backup verifier emits `tcos.nightlyEmergencyBackupVerification.v1` JSON and reads only the newest archive, manifest, `.sha256` file, and tar listing to prove the archive is restorable without creating a new archive or pushing Git. The archive helpers write timestamped status evidence under `.codex-run/nightly-backup-status/` and verification evidence under `.codex-run/nightly-backup-verification/` with Git head, origin, working-tree metadata, the current schedule-health state, the current scheduler-proof state, freshest backup timestamp, current-for-last-scheduled-run status, over-retention count, the current launchd runtime state, and backup integrity results.

Treat the backup folder like a password vault because local archives can include production-capable `.env*` secrets. Do not upload the tarballs to public cloud storage or commit them to Git.

The current disaster-recovery snapshot created on 2026-07-11 is fully verified in the local location below. The second path is the intended Transcend target:

```
C:\Projects\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-064539
D:\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-064539  (copy pending)
```

The `D:` Transcend copy is the laptop-failure copy. The 2026-07-11 external copy was blocked by the automation environment's elevated-action quota after the local folder and archive verified successfully. The earlier `20260710-230103` snapshot remains on `D:`. Do not treat the new snapshot as laptop-failure-safe until its folder, archive, and SHA-256 file have been copied to `D:` and the external verifier passes. Do not rely only on the `C:` copy because an internal-drive failure can destroy both the working project and a same-drive backup.

Fast source checkpoint after the GitHub `main` sync, Vercel production deploy verification, and seller inventory export/activation hardening:

```
C:\Projects\TCOS_DISASTER_RECOVERY\code-snapshots\tcos-code-snapshot-20260711-152107
C:\Projects\TCOS_DISASTER_RECOVERY\code-snapshots\tcos-code-snapshot-20260711-152107.zip
C:\Projects\TCOS_DISASTER_RECOVERY\code-snapshots\tcos-code-snapshot-20260711-
152107.zip.sha256
SHA-256: DA4851DC757CC2A9A2A95E7FEB165BB6AA9D4277B4063B68EC56481388BBCD01
Git HEAD: 4a3645c4c8e8741b69b0cb5472ac81f2d39712a0
```

This fast checkpoint preserves the current source tree, `.env*` files, and `.git` history at the same commit deployed by Vercel production. It intentionally excludes rebuildable/generated folders such as `node_modules`, `.next`, `.codex-run`, `.venv`, `TCOS_BACKUP`, `tsconfig.tsbuildinfo`, and Paddle model caches. Use it as a quick source-code recovery point, not as a complete laptop-failure restore. The full disaster-recovery snapshot still needs the Transcend copy and external verification before it is laptop-failure-safe.

Snapshot contents include:

- complete working repository and all untracked files
- `.git` history and an independent Git bundle
- ignored `.env*` files and production-capable secrets
- `node_modules` and `.next`
- portable Node/npm and Python runtimes
- PaddleOCR virtual environment and downloaded PP-OCRv6 models
- Supabase API-level table/schema/Auth/Storage export available to the service-role key
- older local backups
- restore, start, and verification scripts
- critical-file checksum manifest
- full `tar.gz` archive and SHA-256 file

Authoritative file counts, byte totals, Git head, cloud-export status, and component sizes are stored inside `manifest\backup-manifest.json`. Every snapshot file except the checksum CSV itself is recorded in `manifest\critical-files.sha256.csv`. Both local and external verification must report zero missing and zero mismatched files, and the external archive SHA-256 must match the local archive. Do not copy older numeric totals into this manual; always read the manifest belonging to the snapshot being restored.

Verify the Transcend backup

1. Connect the Transcend drive.
2. Open PowerShell.
3. Run:

```powershell
Set-ExecutionPolicy -Scope Process Bypass
& "D:\TCOS_DISASTER_RECOVERY\TCOS_FULL_DISASTER_RECOVERY_20260711-
064539\scripts\VERIFY_BACKUP.ps1"
```

1. Require:

```text
Missing: 0
Mismatched: 0
Backup verification passed.
Complete archive SHA256 passed.
```

Do not erase the working laptop until the external verification passes.

Restore on a replacement Windows laptop

1. Copy the entire snapshot folder from `D:` onto the replacement computer.
2. Open PowerShell inside the copied snapshot folder.
3. Run verification first:

```powershell
Set-ExecutionPolicy -Scope Process Bypass
.\scripts\VERIFY_BACKUP.ps1
```

1. Confirm no critical file is missing or mismatched.
2. Confirm `C:\Projects\truely-collectables` does not already contain a different working project. Rename an existing folder first.
3. Restore:

```powershell
.\scripts\RESTORE_TCOS.ps1
```

1. The default restore targets are:

```text
C:\Projects\truely-collectables
C:\Projects\Python312
C:\Projects\NodeRuntime
```

1. Start both services:

```powershell
.\scripts\START_TCOS.ps1
```

1. Open:

```text
http://localhost:3000/admin/login
```

1. Log in and run a known-card InstaComp test before doing new production work.

The restore script refuses to merge into an existing app folder unless `-Force` is supplied. Avoid `-Force` unless an intentional merge is required and the old folder has already been preserved.

Manual service verification after restore

```
Invoke-RestMethod http://127.0.0.1:8008/health
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:3000/admin/login
```

If startup fails, inspect:

```
%TEMP%\tcos-paddleocr.stderr.log
%TEMP%\tcos-next.stderr.log
```

Git-only fallback

If the paste-ready app copy is damaged but the independent Git bundle is good:

```
git clone .\git\truely-collectables.bundle C:\Projects\truely-collectables
```

The Git bundle restores committed history only. It does not replace the paste-ready copy because Git does not contain ignored `.env*`, dependencies, model caches, or untracked work.

Cloud recovery boundary

The snapshot is sufficient to continue TCOS after a laptop failure while the existing provider accounts remain active. It is not a complete independent backup of every provider.

Current gaps:

- Supabase export is service-role API level, not a PostgreSQL `pg_dump`
- a full database dump still requires the Supabase database password/direct connection and must preserve private schemas, roles, grants, extensions, Auth password hashes, and remote-only schema drift
- Vercel Development, Preview, and Production environment sets were not exported because no Vercel personal access token/project link was available
- Stripe, eBay, Resend, OpenAI, Coverage, and other provider-side account state remains with those providers

Local credentials are preserved inside `.env*`, so the restored app can reconnect to the existing provider accounts. Add a PostgreSQL dump and Vercel environment exports to a future snapshot when those credentials are available.

Backup security

The snapshot contains production-capable secrets in plaintext so it can be paste-ready. Treat the Transcend drive like a password vault:

- keep it physically secure
- encrypt the drive where possible
- do not upload the unencrypted snapshot to public cloud storage
- do not email or share `.env*`
- rotate provider keys immediately if the drive is lost or stolen

Refresh the disaster backup after material changes

After code, migrations, environment variables, OCR models, or this manual changes:

1. Commit the intended project changes locally.
2. Stop TCOS and PaddleOCR for a consistent snapshot.
3. Refresh the paste-ready local snapshot without omitting hidden/ignored files.
4. Regenerate the Git bundle.
5. Regenerate the manual HTML and PDF.
6. Regenerate checksum manifests and the full archive.
7. Copy the refreshed folder, archive, and SHA-256 file to the Transcend drive.
8. Run the external `VERIFY_BACKUP.ps1`.
9. Restart TCOS and PaddleOCR.
10. Record the new snapshot timestamp and hash in this manual.

37. Maintenance Rule

When a feature changes, update this manual in the same work session.

PDF generation can wait until the end of a completed module or slice so development can move faster. Keep the Markdown manual current during implementation, then run `npm run manual:pdf` at the module checkpoint.

If the local browser is installed in a nonstandard location, run `TCOS_MANUAL_BROWSER_PATH=/path/to/browser npm run manual:pdf`. If Chrome writes the PDF and then hangs in updater or crash-handler cleanup, rerun with a shorter `TCOS_MANUAL_PDF_BROWSER_TIMEOUT_MS`; the generator verifies the PDF timestamp and size before accepting that timeout as success.

Every generated manual PDF, including the future separate mobile app manual PDF, must watermark each page with `Property of Dag Danky Holdings LLC.`.

Checklist for future changes:

1. Update feature behavior section.

2. Update route list if routes changed.
3. Update environment variables if new keys are added.
4. Update database docs if tables/fields change.
5. Update the mobile app manual when the change affects the mobile app.
6. Run `npm run lint` and `npm run build`.
7. Run affected payment, shipping, marketplace, OCR, and recovery checks.
8. Regenerate the correct HTML/PDF manual with `npm run manual:pdf`.
9. Commit the manual with the feature changes.
10. Refresh the paste-ready local disaster snapshot.
11. Refresh the Transcend folder, archive, and SHA-256 file.
12. Run the external `VERIFY_BACKUP.ps1` and require a clean result.

The app should not get ahead of the documentation.

Recent InstaComp catch-up through commit `087bda1`:

- Batch image rotation now works for single-card and batch rows with 45-degree left/right clicks. Rotation is based on the original local image plus cumulative degrees, strips repeated rotated filename suffixes, and avoids repeated-preview shrinkage. Recovered remote-only saved-lot images must be reselected locally before rotation.
- Batch thumbnails can be clicked for larger image review, with zoom-oriented controls so operators can inspect wrong pairings, Clear Cut backs, serial stamps, and variant clues before retrying.
- The scanner no longer prints `Front` and `Back` labels under thumbnails; it groups paired images by upload/pair order while retaining internal front/back pairing state.
- Clear Batch and cancellation recovery now handle active worker leases more safely. If cancellation is waiting for an existing lease, the saved lot remains recoverable and the operator should retry Clear Batch shortly instead of manually deleting queue rows.
- InstaComp title generation suppresses generic `Base`, strips duplicate player/release/parallel/card-number echoes, treats Upper Deck as manufacturer for O-Pee-Chee Platinum, preserves Upper Deck Series 1/2/Extended as release names, preserves true one-of-one display, and uses serial print runs such as `/50` instead of exact copy numbers except for `1/1`.
- Printed variant/parallel identity is guarded so OCR-visible or AI-vision-note-visible Limited Red, Clear Cut, Outliers, Future Watch, Spectrum FX, acetate/clear-stock, insert, color foil, refractor, prizm, holo, wave, shimmer, ice, laser, scope, pulsar, mojo, mosaic, and similar cues cannot stay as generic base. Upper Deck clear-stock cards with centered/ghosted back-logo cues are promoted to `Clear Cut`; ambiguous printed cues stay in review rather than being overclaimed.
- InstaComp Multi-Scanner Consensus now has visible fast/full council policy with adaptive second-reader escalation. High-confidence complete cards stay in the `fast_lane` / `fast_lane_council`; low-confidence, incomplete, front-only, weak-pairing, serial-numbered, uncertain, or printed-variant-signal cards move to `escalated_multi_ai` / `full_council`, run a second independent AI identity reader, and preserve the risk tier, scanner plan, and escalation reasons in OCR diagnostics.
- InstaComp trial intake now writes ignored local guide `instacomp-trial-groundtruth-guide.local.md` with answer-key row progress, first missing rows, examples, required fields, and the exact commands to apply/recheck the 100-card answer key.

- InstaComp final tester status now reports whether the ignored local answer-key guide exists, matches the current ground-truth audit, and needs to be refreshed before the 100-card tester run.
- InstaComp final tester status now shows raw `instacomp-trial-inbox/` accepted image counts separately from normalized `instacomp-trial-images/`, so operators can see whether scanner files have been copied before staging.
- InstaComp final tester status now reads ignored worksheet `instacomp-trial-groundtruth.local.tsv` directly and reports row count, core-ready rows, required-column health, and the exact apply/recheck next step.
- InstaComp now has a local visual answer-key HTML guide: `npm run instacomp:trial:answer-key-html` writes ignored `instacomp-trial-answer-key.local.html` with front/back thumbnails beside TSV fields, missing-field status, and apply/recheck commands so the 100-card answer key can be filled faster before the final tester run.
- The 100-card final tester now has a local ground-truth TSV worksheet flow: `npm run instacomp:trial:groundtruth:sheet` writes `instacomp-trial-groundtruth.local.tsv`, the operator can fill the answer key in a spreadsheet, and `npm run instacomp:trial:groundtruth:apply` applies matching `trialCardId` rows back to the ignored local manifest before audit/scoring.
- Clear Cut detection now treats clear/acetate stock, Clear Cut text, and matching back-logo evidence as variant signals that can override generic set/base guesses.
- COMC active-price ingestion is removed from the comp equation. COMC, Sportlots, Panini America, Upper Deck, Cardboard Connection, Blowout Cards, TCDB, and similar sources remain reference/checklist context for identity review when usage rights allow.
- External provider failures and dead providers are skipped during batch scans so one failed provider does not stall the entire batch.
- Upload/scan throughput was raised for the durable queue: row registration and confirmation chunks are 50, signed upload concurrency is 6, default browser scan concurrency is 4, maximum browser scan concurrency is 6, and queue claim mini-packs are 5 rows with a matching server cap.
- Front/back image optimization and serial-detail crop preparation run in parallel where safe, and the JSON scan route converts front/back/detail image data concurrently before OCR/AI work.
- Database connection pressure now trips a browser-side pressure governor: scanning pauses before claiming more work, browser concurrency backs down by one, and the operator sees a specific pressure message instead of repeated generic failures.
- InstaComp Auto-Pilot branding now includes `InstaComp™`.
- `Buy Me on TCOS` and `Trade For Me on TCOS` buttons were added as sale/trade search handoffs for detected cards.
- `Add to Available for Trade` was added for seller-owned scans. It creates or reuses a collection item marked Available for Trade, writes the handoff back to the scan row, and is mutually exclusive with sell-draft creation.
- The trade handoff is protected by `20260716010000_add_instacomp_trade_handoff.sql`; apply it before using Available for Trade.
- Lint is now clean with zero warnings after removing dead COMC active-provider code and aligning the claim route with the five-card mini-pack throughput.

Recent seller workspace wording cleanup:

- InstaComp draft success links now open the Seller Inventory InstaComp lane directly through `Open InstaComp Drafts` and `Open in InstaComp drafts`.
- Seller Inventory now has a `Source` filter with an `InstaComp` lane, InstaComp item badges, scan/serial/price-source details, and ready-row marketplace packet export controls.
- InstaComp draft titles now prefer serial-run display such as `/50` instead of exact copy-number display such as `07/50`; true one-of-one cards remain `1/1`. Admin scanner, test scanner, server draft creation, and comp-search title generation share the same draft-title/serial-run helpers so the behavior does not drift.
- Seller Inventory rows now show the default Standard Envelope/Ground Advantage shipping plan, estimated postage, Coverage requirement, Coverage type, and Standard Envelope under-$20 seller-protection warning; selected rows now show a shipping mix summary before activation or marketplace export, and selected ready-row marketplace packets include self-contained 2% reserve, `$20.00` cap, claim trigger, not-insurance, and shipping-excluded reimbursement fields.
- Seller Inventory Selection Summary now shows a selected-row activation check with activatable, needs-work, active, archived, and top-blocker counts before `Activate Ready`.
- Seller Inventory Bulk Action Follow-Up now groups failed rows by blocker reason and lets operators keep only a specific blocker group selected for cleanup.
- Seller Inventory Bulk Action Follow-Up now shows `Retry Corrected` when failed rows have become eligible for the same activation/archive action.
- Seller Inventory bulk action result cards now include `Copy Bulk Report` and `Download Bulk Report` for a JSON audit/debug handoff with success, failure, blocker, and retry-ready details.
- Seller Inventory bulk action result cards now include `Clear Bulk Report` so old result/follow-up cards can be intentionally dismissed after audit handoff.
- Seller Inventory marketplace JSON/CSV exports now include explicit prep-only, no-external-publishing, no-shipping-purchase, and standard `3-day auction` guardrail fields so exported files cannot be mistaken for live marketplace or postage actions; selected ready rows can now download the JSON marketplace packet directly instead of relying on clipboard copy, downloaded marketplace filenames include row count plus active filter context, and JSON packets embed the active export context.
- Seller Inventory Bulk Controls now show the marketplace export guardrails on-screen before copy/download actions.
- Seller Connections now includes a Marketplace Packet Intake card explaining that Seller Inventory marketplace packets are cross-list prep only, auction prep defaults to the TCOS standard `3-day auction`, and packets do not publish externally, buy postage, create Coverage policies, release seller payouts, fulfill orders, create insurance, or activate TCOS Under-$20 Seller Protection; it routes sellers back to ready and needs-work Seller Inventory rows.
- Seller Connections now keeps a session-saved safe marketplace API receipt trail with `Copy Safe Receipt`, `Download Safe Receipt`, `Copy Trail`, `Download Trail`, and `Clear Trail` controls for auth/import/staging/reconcile/order-import/promotion handoffs.
- Seller Command Center now shows `InstaComp Ready` and routes to ready InstaComp drafts when that is the safest inventory shortcut.
- Seller order surface labels now use `Seller Order Workspace`, `Search orders`, `Order views`, and `Reset Order View` wording instead of the older workflow phrasing.

- Seller dashboard order signal chips now read `Shipping Orders`, `Cash-Out Orders`, `Action Orders`, and `Completed Orders`.
- Seller payout shortcuts now use `Blocked Payouts`, `Cash-Out Payouts`, `Attention Payouts`, and `Paid Payouts`, and seller inventory order follow-up labels now use `Shipping Orders`.
- The seller home payout-pressure card now uses `Open Payouts`, `Blocked Payouts`, and `Paid Payouts` summary labels so its totals match the seller payout workspace wording.
- The seller order workspace summary now uses `Open Payouts` for its cash-out pressure metric so order-level payout totals match the seller payout workspace wording.
- Seller marketplace connector actions now use `Open Seller Connections`, and the future Shopify interest control now reads `Request Shopify`.
- The seller marketplace page now titles the surface `Seller Connections` so the page heading matches the `Open Seller Connections` wording used across seller shortcuts and connector cards.
- Seller marketplace handoff buttons now use `Open Review Rows`, `Search Review Rows`, and `Search Marketplace Rows` wording so review-stage jumps match the marketplace row language already used by import-run controls and cross-workspace links.
- Seller marketplace handoff buttons now use `Open Ready Rows` and `Search Ready Rows` wording instead of the older `Ready Stage` labels, so ready-state marketplace jumps match the rest of the import-run controls.
- Seller inventory and marketplace recovery controls now use `Open Active Inventory`, `Open Archived Inventory`, `Open Seller Inventory`, `Open Failed Inventory`, and `Open Failed Promotions`.
- Seller signal summaries now refer to the seller workspace, and seller order detail action cards now use `Return View`, `Shipping Orders`, and `Cash-Out Payouts`.
- The seller order detail cash-out section now uses `Cash-Out Payouts` wording in both its section title and empty state so the order-detail surface matches the seller payout workspace language.
- Seller home action-order cards now label order-linked cash-out counts as `Open Payouts` instead of `Open Claims` so those pressure summaries match the rest of the seller payout workspace wording.
- The seller order workspace now labels per-order cash-out sections as `Cash-Out Payouts` so order-list payout panels match the seller payout workspace wording.
- The seller order detail payout action card now uses `Cash-Out Payouts` wording so its order-level handoff matches the rest of the seller payout workspace language.
- Seller order detail return links now use `Return To Seller Orders`, `Return To Action Orders`, and matching payout return wording instead of the older `Back To ...` labels.
- Account-level seller payout shortcuts now use `Seller Payout Setup`, `Open Cash-Out Payouts`, and `Open Seller Payouts` wording so account dashboard handoffs match the seller workspace language.
- Account-level Seller Cash-Out and Seller Command Center payout-pressure cards now expose TCOS Under-$20 Seller Protection reserve visibility, including `Protection Reserve`, `Under-$20 Protection Reserve`, the 2% reserve, protected item amount, protected/liability row counts, and shipping excluded from reimbursement.
- Admin Seller Payouts now exposes `Admin Under-$20 Protection Reserve` and row-level `Under-$20 Protection` chips so payout operators can see TCOS internal Standard Envelope reserve math, protected/liability row counts, and shipping-excluded reimbursement limits before payout release work.

- Financial Reconciliation now exposes `Seller-Protection Reimbursement Adjustments` under `TCOS Internal Money Context`, including latest-run reimbursement and shipping-excluded summary fields, so money operators can distinguish TCOS internal seller-payable credits from Stripe payout movement while reviewing unmatched money.
- Account-level seller marketplace shortcuts now use `Blocked Marketplace Rows`, `Ready Marketplace Rows`, `Mapped Marketplace Rows`, and `Marketplace Rows` wording so account handoffs match the seller marketplace workspace language.
- Seller home payout signal buttons and seller payout shortcut actions now use `Open Cash-Out Payouts` wording instead of mixing request-based labels into the same payout workspace handoff.
- Seller inventory and marketplace bulk guidance now refer to cleanup work, current selections, seller draft inventory views, and ready, review, or conflict rows instead of older workflow and stage wording.
- Marketplace import-run buttons now use `Open Ready Rows`, `Open Review Rows`, `Open Blocked Rows`, and `Open Mapped Rows`.
- Seller signal, cash-out request, and fallback order or payout buttons now use full `Open Seller Payouts`, `Open Blocked Payouts`, `Open Cash-Out Payouts`, `Open Paid Payouts`, `Open Attention Payouts`, and `Open Seller Orders` wording.
- Blocked-payout order cards on seller home and seller payouts now also use `Open Order Detail` wording.
- Seller-home signal buttons plus seller order and payout shortcut cards now also use full `Open Shipping Orders`, `Open Action Orders`, `Open Cash-Out Orders`, `Open Completed Orders`, `Open Seller Orders`, `Open Attention Payouts`, `Open Blocked Payouts`, `Open Cash-Out Payouts`, `Open Paid Payouts`, and `Open Seller Payouts` wording when those labels render directly.